TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN: CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

ĐỀ TÀI: NGHIÊN CỨU NGÔN NGỮ GO

Giảng viên hướng dẫn: Ths. Phạm Thi Vương

Nhóm sinh viên thực hiện: Phạm Thiên Phú - 3122560059

Nguyễn Quốc Tuấn - 3122560087

Phạm Thanh Sự - 3122560065

Đào Thanh Tú - 3122560085

TP.HCM tháng 04/2026

Mục lục

CHƯƠNG 1: GIỚI THIỆU VỀ NGÔN NGỮ LẬP TRÌNH GO 4
1.1. Lịch sử hình thành và đặc điểm ngôn ngữ 4
1.2. Sức mạnh của Go trong xử lý đồng thời (Concurrency) 10
1.3. Lý do lựa chọn Go - Phân tích từ góc độ đặc điểm ngôn ngữ 16
CHƯƠNG 2: CƠ HỘI NGHỀ NGHIỆP VÀ NHU CẦU THỊ TRƯỜNG 23
2.1. Nhu cầu tuyển dụng lập trình viên Backend Go: 23
2.2. Mức lương và lộ trình phát triển: 24
2.3. Lợi thế cạnh tranh: 25
CHƯƠNG 3: THIẾT LẬP MÔI TRƯỜNG VÀ CẤU TRÚC DỰ ÁN 27
3.1. Thiết lập môi trường phát triển 27
3.2. Quản lý dependency với Go Modules 33
3.3. Xây dựng cấu trúc thư mục theo Clean Architecture 35
CHƯƠNG 4: CÁC KHÁI NIỆM NỀN TẢNG - MIDDLEWARE TRONG GO 50
4.1. Middleware là gì? Phân tích từ góc độ ngôn ngữ học 50
4.2. Phân tích chuyên sâu ba Middleware cốt lõi 56
4.3. Tổ chức Middleware trong kiến trúc Clean Architecture của CinemaBooking 75
4.4. Rate Limiting Middleware - Kết hợp Goroutine và Channel 78
CHƯƠNG 5: CÁC KỸ THUẬT NÂNG CAO VÀ TỐI ƯU HÓA 82
5.1 Xử Lý Đồng Thời - Bài Toán Đặt Ghế Trùng Nhau (Double Booking) 82
5.2. Connection pool Postgres (tối ưu kết nối DB) 86
5.3 Tối Ưu API Response - Phân Trang Danh Sách Phim 88
5.4 Bảo Mật Với JWT 90
5.5 Idempotency Cho Booking & Payment 93
CHƯƠNG 6. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 97
6.1. Thu thập và đặc tả yêu cầu 97
6.4. Thiết kế RESTful API (API Contract) 101
6.5. Sơ đồ ERD 104
CHƯƠNG 7: TRIỂN KHAI VÀ KIỂM THỬ API 105
7.1. Công Cụ Kiểm Thử - Postman 105
7.2. Tích Hợp Swagger - Tài Liệu API Tự Động 109
7.3. Bảng Kiểm Thử Tổng Hợp 111
7.4. Tổng Kết Kết Quả Kiểm Thử 122

CHƯƠNG 8: ỨNG DỤNG NGÔN NGỮ GOLANG THỰC HIỆN WEB BOOKING CINEMA 123
8.1. Xác Thực Và Phân Quyền Người Dùng (Jwt + Middleware Pattern) 123
8.2. Xử Lý Đồng Thời Và Race Condition (Concurrency Control) 127
8.3. Hệ Thống Real-Time Và Broadcast Trạng Thái Ghế (Websocket) 131
8.4. Kiến Trúc Phần Mềm (Clean Architecture & Depend 135
ency Injection) 135
8.5. Tích Hợp Cổng Thanh Toán & Idempotency (Payment Safety) 137
8.6. Xử Lý Webhook Bất Đồng Bộ (Async Processing) 141
8.7. Domain Modeling Và Struct Tags 143
8.8. Graceful Shutdown Và Quản Lý Vòng Đời Ứng Dụng 145
8.9. CẤU TRÚC LỖI VÀ STATE MACHINE TRẠNG THÁI ĐẶT VÉ 148
CHƯƠNG 9: KẾT QUẢ ĐẠT ĐƯỢC 151
9.1 Mức Độ Hoàn Thành Các Chức Năng 151
9.2. Hệ Thống Đáp Ứng Kiến Trúc Clean MVC 152
9.3. Đánh Giá Quá Trình Làm Việc Nhóm 155
9.4. Khó Khăn Và Cách Khắc Phục 158
CHƯƠNG 10: HƯỚNG PHÁT TRIỂN 162
10.1. Nâng Cấp Tính Năng Ứng Dụng 162
10.2. Nâng Cấp Kiến Trúc Phần Mềm 164

CHƯƠNG 1: GIỚI THIỆU VỀ NGÔN NGỮ LẬP TRÌNH GO

1.1. Lịch sử hình thành và đặc điểm ngôn ngữ

1.1.1. Bối cảnh ra đời

Ngôn ngữ lập trình Go (tên chính thức) hay Golang (tên thông dụng để phân biệt khi tìm kiếm) được khởi xướng nội bộ tại Google vào năm 2007 bởi ba kỹ sư: Robert Griesemer, Rob Pike và Ken Thompson - trong đó Ken Thompson là đồng tác giả của ngôn ngữ C và hệ điều hành Unix. Go được công bố mã nguồn mở vào năm 2009 và phát hành phiên bản ổn định đầu tiên (Go 1.0) vào tháng 3 năm 2012.

Sự ra đời của Go không xuất phát từ mục tiêu học thuật thuần túy, mà từ một vấn đề kỹ thuật thực tế: cơ sở hạ tầng phần mềm tại Google vào cuối những năm 2000 đang phải vận hành các hệ thống quy mô cực lớn với hàng triệu dòng code C++ và Java. Ba điểm đau chính được xác định là:

- Thời gian biên dịch quá chậm với C++, làm giảm năng suất lập trình viên.
- Mô hình concurrency phức tạp và dễ lỗi với Thread/Mutex truyền thống trong Java và C++.
- Thiếu một ngôn ngữ hệ thống hiện đại kết hợp được tốc độ của C với sự an toàn và năng suất của các ngôn ngữ hiện đại hơn.

Go được thiết kế để giải quyết đồng thời cả ba vấn đề trên, với triết lý thiết kế cốt lõi: *"Make it simple, but not simpler."*

1.1.2. Đặc điểm ngôn ngữ học cơ bản

Từ góc độ phân loại ngôn ngữ lập trình, Go sở hữu một tập hợp đặc điểm khá độc đáo, không hoàn toàn thuộc về một trường phái nào:

a) Kiểu tĩnh biên dịch tĩnh (Statically Typed, Compiled)

Go là ngôn ngữ kiểu tĩnh (*statically typed*): kiểu dữ liệu của mọi biến phải được xác định tại thời điểm biên dịch (*compile time*), không phải lúc chạy (*runtime*). Điều này khác biệt cơ bản với Python hay JavaScript (kiểu động - *dynamically typed*).

Tuy nhiên, Go bổ sung cơ chế type inference thông qua toán tử :=, cho phép trình biên dịch tự suy luận kiểu từ giá trị khởi tạo, giúp code ngắn gọn hơn mà không mất đi sự an toàn kiểu:

```go
// Khai báo tường minh — Go biết x có kiểu int
var x int = 10

// Type inference — trình biên dịch tự suy luận y có kiểu string
y := "cinema"

// Lỗi biên dịch — Go phát hiện ngay, không phải lúc runtime
// x = "text" // cannot use "text" (type string) as type int
```

b) Biên dịch ra mã máy độc lập (Statically Linked Binary)

Go biên dịch thẳng ra file nhị phân mã máy (*native binary*), không yêu cầu máy ảo (JVM của Java) hay trình thông dịch (Python interpreter). Toàn bộ dependencies được liên kết tĩnh (*static linking*) vào một file thực thi duy nhất. Điều này tạo ra lợi thế triển khai (*deployment*) đáng kể: chỉ cần copy một file binary là có thể chạy trên bất kỳ máy nào cùng kiến trúc CPU.

Tiêu chí	Go	Java	Python	C++
Kiểu hệ thống	Tĩnh	Tĩnh	Động	Tĩnh
Thực thi	Native binary	JVM bytecode	Interpreter	Native binary

Thời gian biên dịch	Rất nhanh	Trung bình	N/A	Chậm
Quản lý bộ nhớ	GC tự động	GC tự động	GC tự động	Thủ công
Concurrency built-in	Goroutine/Channel	Thread/Lock	GIL (hạn chế)	Thread/std::async

c) Garbage Collection thế hệ mới

Go sử dụng bộ thu gom rác (*Garbage Collector - GC*) với thuật toán tri-color mark-and-sweep không dừng toàn bộ chương trình (*concurrent GC*). Kể từ Go 1.5, thời gian dừng của GC (*stop-the-world pause*) được giảm xuống dưới mức mili giây, khắc phục điểm yếu lớn nhất của các ngôn ngữ GC truyền thống trong các hệ thống yêu cầu độ trễ thấp.

d) Không có kế thừa - Composition qua Interface

Go loại bỏ hoàn toàn cơ chế kế thừa lớp (*class inheritance*) - một trong những tính năng trung tâm của Java và C++. Thay vào đó, Go sử dụng mô hình composition và structural typing thông qua interface. Một kiểu dữ liệu được coi là thỏa mãn một interface nếu nó cài đặt đủ các phương thức của interface đó - không cần khai báo tường minh. Đây là cơ chế duck typing nhưng được kiểm tra tại compile time.

1.1.3. Code minh họa - Các đặc điểm cơ bản của ngôn ngữ

Đoạn code dưới đây minh họa đồng thời bốn đặc điểm ngôn ngữ học vừa trình bày: type inference, interface structural typing, zero value, và multiple return values - tất cả thông qua một cấu trúc dữ liệu đơn giản liên quan đến ngữ cảnh CinemaBooking.

```go
package main

import (
    "errors"
    "fmt"
)

// --- ĐẶC ĐIỂM 1: INTERFACE STRUCTURAL TYPING ---
// Priced là interface; bất kỳ kiểu nào có phương thức Price() float64
// đều tự động thỏa mãn interface này — không cần từ khóa "implements"
type Priced interface {
    Price() float64
}

// --- ĐẶC ĐIỂM 2: STRUCT + METHOD (thay cho class) ---
// Go không có class; struct + method là cách tổ chức dữ liệu và hành vi
type Seat struct {
    Row     string
    Col     int
    SeatType string // "standard", "vip", "couple"
}

// Seat thỏa mãn interface Priced nhờ cài đặt phương thức Price()
// Không cần khai báo "Seat implements Priced" như Java
func (s Seat) Price() float64 {
    // Go dùng map literal, không cần khởi tạo phức tạp
    pricing := map[string]float64{
        "standard": 75_000,  // Dấu _ trong số giúp dễ đọc — Go spec §Literals
        "vip":      120_000,
        "couple":   200_000,
    }
    // Zero value: nếu key không tồn tại, map trả về 0.0 (zero value của float64)
    return pricing[s.SeatType]
}

// --- ĐẶC ĐIỂM 3: MULTIPLE RETURN VALUES ---
// Go cho phép hàm trả về nhiều giá trị; quy ước là (kết_quả, error)
// Đây là cách Go xử lý lỗi thay vì exception (Java, Python)
func calculateTotal(seats []Priced, discountPct float64) (float64, error) {
    // Validation — lỗi được trả về tường minh, không "ném" exception
    if discountPct < 0 || discountPct > 100 {
        return 0, errors.New("discount phải trong khoảng [0, 100]")
    }

    var total float64 // zero value của float64 là 0.0 — Go đảm bảo điều này
    for _, item := range seats {
        total += item.Price()
    }

    discount := total * (discountPct / 100)
    return total - discount, nil // nil là zero value của error interface
}
```

```go

func main() {
    // --- ĐẶC ĐIỂM 4: TYPE INFERENCE với := ---
    seats := []Priced{ // slice của interface — Go cho phép điều này
        Seat{Row: "A", Col: 1, SeatType: "vip"},
        Seat{Row: "A", Col: 2, SeatType: "standard"},
        Seat{Row: "B", Col: 5, SeatType: "couple"},
    }

    // Nhận hai giá trị trả về — bắt buộc phải xử lý error (hoặc dùng _)
    total, err := calculateTotal(seats, 10)
    if err != nil {
        fmt.Println("Lỗi:", err)
        return
    }
    fmt.Printf("Tổng tiền sau giảm giá 10%%: %.0f VNĐ\n", total)
    // Output: Tổng tiền sau giảm giá 10%: 355500 VNĐ
}
```

Phân tích các điểm ngôn ngữ học trong code:

- Type inference (:=): Go suy luận seats là []Priced và total, err là float64, error - không cần khai báo tường minh.
- Structural typing: Seat thỏa mãn Priced tự động vì có phương thức Price() float64 với đúng signature - trình biên dịch kiểm tra tại compile time.
- Zero value: var total float64 được khởi tạo thành 0.0 tự động - Go đảm bảo không có biến uninitialized.
- Multiple return values: calculateTotal trả về (float64, error) - buộc người gọi xử lý cả kết quả lẫn khả năng lỗi.

1.1.4. Bài toán minh họa nhỏ - Structural Typing & Interface

Đề bài: Viết một hàm printSummary(items []Describable) nhận vào một slice bất kỳ, trong đó Describable là interface có phương thức Describe() string. Định nghĩa hai struct Movie và Cinema, mỗi struct tự cài đặt Describe() theo cách riêng. Gọi printSummary với danh sách hỗn hợp chứa cả Movie và Cinema.

Mục tiêu: Quan sát cơ chế structural typing - Go xử lý đa hình (*polymorphism*) không thông qua kế thừa mà thông qua interface, và kiểm tra toàn bộ tại compile time.

```go
package main

import "fmt"
// Interface — chỉ quan tâm đến hành vi, không quan tâm đến kiểu cụ thể
type Describable interface {
    Describe() string
}
type Movie struct {
    Title string
    Genre string
}
// Movie thỏa mãn Describable — tự động, không cần khai báo
func (m Movie) Describe() string {
    return fmt.Sprintf("Phim: %s | Thể loại: %s", m.Title, m.Genre)
}

type Cinema struct {
    Name string
    City string
}
// Cinema cũng thỏa mãn Describable — cùng interface, khác kiểu
func (c Cinema) Describe() string {
    return fmt.Sprintf("Rạp: %s | Thành phố: %s", c.Name, c.City)
}

// Hàm này không biết gì về Movie hay Cinema — chỉ biết Describable
// Đây chính là sức mạnh của interface: tách biệt "hợp đồng" khỏi "triển khai"
func printSummary(items []Describable) {
    for i, item := range items {
        fmt.Printf("[%d] %s\n", i+1, item.Describe())
    }
}
func main() {
    items := []Describable{
        Movie{Title: "Inception", Genre: "Sci-Fi"},
        Cinema{Name: "CGV Vincom", City: "Hà Nội"},
        Movie{Title: "Interstellar", Genre: "Sci-Fi"},
        Cinema{Name: "Lotte Cinema", City: "TP.HCM"},
    }
    printSummary(items)
}
// Output:
// [1] Phim: Inception | Thể loại: Sci-Fi
// [2] Rạp: CGV Vincom | Thành phố: Hà Nội
// [3] Phim: Interstellar | Thể loại: Sci-Fi
// [4] Rạp: Lotte Cinema | Thành phố: TP.HCM
```

Output nhận được:

```
PS C:\Users\Admin> go run "c:\Users\Admin\turtorial.go"
[1] Phim: Inception | Thể loại: Sci-Fi
[2] Rạp: CGV Vincom | Thành phố: Hà Nội
[3] Phim: Interstellar | Thể loại: Sci-Fi
[4] Rạp: Lotte Cinema | Thành phố: TP.HCM
```

1.2. Sức mạnh của Go trong xử lý đồng thời (Concurrency)

1.2.1. Lý thuyết - Mô hình Concurrency của Go

a) Phân biệt Concurrency và Parallelism

Trước khi phân tích cơ chế của Go, cần làm rõ hai khái niệm thường bị nhầm lẫn trong tài liệu kỹ thuật:

- **Concurrency** (đồng thời): Khả năng *cấu trúc* chương trình để xử lý nhiều tác vụ trong một khoảng thời gian, nhưng không nhất thiết tại cùng một thời điểm vật lý. Đây là tính chất của *thiết kế chương trình*.
- **Parallelism** (song song): Các tác vụ thực sự *chạy đồng thời* trên nhiều CPU core tại cùng một thời điểm vật lý. Đây là tính chất của *phần cứng và scheduling*.

Rob Pike - một trong ba tác giả của Go - phát biểu: *"Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once."* Go được thiết kế để hỗ trợ concurrency một cách tự nhiên ở cấp độ ngôn ngữ, và khi có nhiều CPU core, Go runtime tự động tận dụng parallelism.

b) Goroutine - Luồng xanh (Green Thread) của Go

Goroutine là đơn vị thực thi đồng thời của Go. Về bản chất, goroutine là **green thread** - luồng được quản lý bởi Go runtime, không phải bởi hệ điều hành (OS thread).

Cơ chế hoạt động bên trong:

Go runtime sử dụng mô hình M:N scheduling: M goroutine được ánh xạ lên N OS thread, trong đó N thường bằng số CPU core (kiểm soát bởi GOMAXPROCS). Bộ lập lịch (*scheduler*) của Go là work-stealing scheduler - các OS thread rảnh rỗi có thể "đánh cắp" goroutine từ hàng đợi của các thread khác để tối ưu hóa sử dụng CPU.

Goroutine được khởi tạo với stack cực nhỏ (mặc định 2KB trong Go 1.4 trở đi, so với 8MB mặc định của OS thread trên Linux), và stack này có thể tự mở rộng (*growable*

stack) khi cần thiết, tối đa lên đến 1GB (có thể cấu hình). Đây là lý do một chương trình Go có thể khởi tạo hàng triệu goroutine mà không cạn kiệt bộ nhớ.

Tiêu chí	OS Thread (Java/C++)	Goroutine (Go)
Stack khởi tạo	~1–8 MB	~2 KB
Quản lý bởi	Hệ điều hành	Go runtime
Chi phí context switch	Cao (kernel mode)	Thấp (user space)
Số lượng thực tế	Hàng nghìn	Hàng triệu
Tạo goroutine mới	N/A	go func() - một từ khóa

c) Channel - Cơ chế giao tiếp giữa các Goroutine

Go theo triết lý thiết kế CSP (*Communicating Sequential Processes*) - một mô hình toán học được Tony Hoare đề xuất năm 1978. Nguyên tắc cốt lõi của CSP trong Go được đúc kết thành một câu:

"Do not communicate by sharing memory; instead, share memory by communicating."

Channel là cơ chế cụ thể hóa nguyên tắc này: thay vì nhiều goroutine cùng truy cập một vùng nhớ chung (và cần mutex để đồng bộ), các goroutine giao tiếp bằng cách gửi/nhận dữ liệu qua channel.

Channel có hai dạng:

- **Unbuffered channel** (make(chan T)): người gửi bị block cho đến khi người nhận sẵn sàng - đồng bộ hóa tự nhiên.
- **Buffered channel** (make(chan T, n)): người gửi chỉ bị block khi buffer đầy - tách rời nhà sản xuất và người tiêu thụ.

1.2.2. Code minh họa - Goroutine và Channel

Đặc điểm concurrency của Go được minh họa qua bài toán kiểm tra ghế trống: hệ thống CinemaBooking cần kiểm tra trạng thái nhiều ghế trong cơ sở dữ liệu đồng thời thay vì tuần tự.

```go
package main

import (
    "fmt"
    "math/rand"
    "sync"
    "time"
)

// SeatStatus mô phỏng kết quả kiểm tra một ghế từ DB
type SeatStatus struct {
    SeatID    string
    Available bool
}

// checkSeat mô phỏng truy vấn DB để kiểm tra một ghế
// Đây là hàm blocking — giả lập độ trễ mạng/DB
func checkSeat(seatID string, results chan<- SeatStatus, wg *sync.WaitGroup) {
    defer wg.Done() // Đảm bảo WaitGroup được giảm kể cả khi hàm panic

    // Mô phỏng độ trễ DB: 50-150ms ngẫu nhiên
    delay := time.Duration(50+rand.Intn(100)) * time.Millisecond
    time.Sleep(delay)

    // Gửi kết quả qua channel — goroutine không trả về giá trị trực tiếp
    // mà "giao tiếp" thông qua channel theo triết lý CSP
    results <- SeatStatus{
        SeatID:    seatID,
        Available: rand.Float32() > 0.3, // 70% khả năng ghế còn trống
    }
}
```

```go
func main() {
    seatIDs := []string{"A1", "A2", "A3", "B1", "B2", "B3", "C1", "C2"}

    // Buffered channel: chứa được len(seatIDs) kết quả
    // Tránh goroutine bị block khi ghi vào channel
    results := make(chan SeatStatus, len(seatIDs))

    var wg sync.WaitGroup // WaitGroup để biết khi nào tất cả goroutine xong

    start := time.Now()

    // Khởi tạo một goroutine riêng cho từng ghế — chi phí cực thấp
    for _, id := range seatIDs {
        wg.Add(1)
        go checkSeat(id, results, &wg) // Từ khóa "go" — đơn giản nhất thế giới
    }

    // Goroutine phụ: đóng channel sau khi tất cả workers hoàn thành
    // Không thể close channel trong main vì có race condition
    go func() {
        wg.Wait()
        close(results) // Báo hiệu không còn giá trị nào được gửi
    }()

    // Range trên channel — tự động dừng khi channel được đóng
    available := []string{}
    for status := range results {
        if status.Available {
            available = append(available, status.SeatID)
        }
    }

    elapsed := time.Since(start)
    fmt.Printf("Kiểm tra %d ghế trong %v (thay vì ~800ms nếu tuần tự)\n",
        len(seatIDs), elapsed)
    fmt.Printf("Ghế còn trống: %v\n", available)
}
```

Phân tích cơ chế ngôn ngữ trong code:

- Từ khóa go: Khởi tạo goroutine mới chỉ bằng một từ khóa - Go runtime xử lý toàn bộ stack allocation và scheduling.
- chan<- SeatStatus: Go hỗ trợ *directional channel* - khai báo channel chỉ-ghi (chan<-) hoặc chỉ-đọc (<-chan) để tăng tính an toàn kiểu.

- sync.WaitGroup: Cơ chế đồng bộ hóa khi không dùng channel để thu kết quả - Add, Done, Wait là ba phương thức duy nhất.
- close(channel) + range: Idiom chuẩn của Go để dừng consumer khi producer hoàn thành - range trên channel tự thoát khi channel được đóng.

1.2.3. Bài toán minh họa nhỏ - Fan-out Pattern với Goroutine

Đề bài: Viết chương trình tính tổng doanh thu từ 5 rạp chiếu phim, mỗi rạp mất 200ms để truy vấn (giả lập bằng time.Sleep). Thực hiện 5 truy vấn *đồng thời* bằng goroutine và thu kết quả qua channel. In ra tổng doanh thu và thời gian thực thi thực tế.

Mục tiêu: Quan sát rằng 5 truy vấn đồng thời mất ~200ms (thay vì ~1000ms nếu tuần tự), thể hiện lợi thế concurrency của Go trong I/O-bound workloads.

Gợi ý hướng giải: Tạo một result chan float64 có buffer bằng 5. Với mỗi rạp, khởi tạo một goroutine tính doanh thu (sleep 200ms rồi gửi kết quả vào channel). Trong main, đọc đúng 5 giá trị từ channel và cộng dồn.

Code minh họa:

```go
package main

import (
    "fmt"
    "time"
)

// fetchRevenue mô phỏng truy vấn DB: lấy doanh thu của một rạp
// Tham số cinemaID và revenue đại diện cho ID và doanh thu giả lập
func fetchRevenue(cinemaID int, revenue float64, ch chan<- float64) {
    time.Sleep(200 * time.Millisecond) // Giả lập độ trễ DB
    ch <- revenue                      // Gửi kết quả vào channel
}

func main() {
    // Dữ liệu giả lập: 5 rạp với doanh thu tương ứng (đơn vị: triệu VNĐ)
    cinemas := map[int]float64{
        1: 150.5, 2: 230.0, 3: 89.75, 4: 310.2, 5: 175.0,
    }

    ch := make(chan float64, len(cinemas)) // Buffered channel
    start := time.Now()

    // Fan-out: mỗi goroutine xử lý một rạp độc lập
    for id, rev := range cinemas {
        go fetchRevenue(id, rev, ch)
    }

    // Fan-in: thu gom kết quả — đọc đúng số lượng đã gửi
    var total float64
    for i := 0; i < len(cinemas); i++ {
        total += <-ch
    }

    fmt.Printf("Tổng doanh thu: %.2f triệu VNĐ\n", total)
    fmt.Printf("Thời gian thực thi: %v (kỳ vọng ~200ms)\n", time.Since(start))
}

// Output mẫu:
// Tổng doanh thu: 955.45 triệu VNĐ
// Thời gian thực thi: 201ms (kỳ vọng ~200ms)
```

Output

```
PS C:\Users\Admin> go run "c:\Users\Admin\turtorial.go"
Tổng doanh thu: 955.45 triệu VNĐ
Thời gian thực thi: 201.1427ms (kỳ vọng ~200ms)
```

1.3. Lý do lựa chọn Go - Phân tích từ góc độ đặc điểm ngôn ngữ

1.3.1. Lý thuyết - Ánh xạ đặc điểm ngôn ngữ vào yêu cầu bài toán

Việc lựa chọn ngôn ngữ lập trình cho một hệ thống phần mềm là quyết định kiến trúc (*architectural decision*), không phải sở thích cá nhân. Mỗi bài toán có một tập hợp yêu cầu phi chức năng (*non-functional requirements*) cụ thể, và ngôn ngữ được chọn cần có các đặc điểm ngôn ngữ học tương ứng để đáp ứng.

Hệ thống đặt vé xem phim (CinemaBooking) là ngữ cảnh minh họa lý tưởng vì nó kết hợp đầy đủ các đặc trưng kỹ thuật khó: lưu lượng biến động đột ngột, tính nhất quán dữ liệu nghiêm ngặt, và yêu cầu độ trễ thấp. Bảng dưới đây ánh xạ từng yêu cầu sang đặc điểm ngôn ngữ của Go:

Yêu cầu hệ thống	**Đặc điểm ngôn ngữ Go đáp ứng**	**Cơ chế cụ thể**
Xử lý nhiều request đồng thời	Goroutine chi phí thấp	go handler() - triệu goroutine/GB RAM
Thời gian phản hồi thấp	GC concurrent, biên dịch native	Stop-the-world < 1ms, không JVM startup
An toàn dữ liệu khi đặt ghế	Race detector built-in	go run -race phát hiện data race
Triển khai đơn giản	Static binary	Một file binary, không cần runtime
Codebase dễ bảo trì	Cú pháp tối giản, gofmt	Một cách viết duy nhất, format tự động
Real-time seat locking	Channel + goroutine	WebSocket events qua buffered channel

1.3.2. So sánh với các lựa chọn thay thế

Để khẳng định tính phù hợp của Go, cần đối chiếu với các ngôn ngữ thay thế khả thi:

Go so với Node.js (JavaScript): Node.js sử dụng mô hình event loop đơn luồng với callback/Promise, phù hợp cho I/O-bound workload đơn giản. Tuy nhiên, khi cần xử lý logic tính toán phức tạp (tính giá vé với nhiều biến động, kiểm tra conflict lịch chiếu), Node.js bị hạn chế bởi single-threaded nature. Go xử lý cả I/O-bound lẫn CPU-bound đồng thời hiệu quả hơn nhờ M:N scheduler.

Go so với Java (Spring Boot): Java có hệ sinh thái mature và mạnh, nhưng chi phí khởi động JVM (~1–3 giây) và bộ nhớ baseline cao (~200–500MB) là bất lợi trong môi trường microservices và containerized deployment. Go khởi động trong mili giây và tiêu thụ RAM thấp hơn một bậc.

Go so với Python (FastAPI): Python với GIL (*Global Interpreter Lock*) không thể thực sự tận dụng multi-core CPU cho CPU-bound tasks. FastAPI dùng asyncio nhưng vẫn bị giới hạn bởi GIL. Go không có GIL - mọi goroutine đều có thể chạy song song thực sự trên nhiều core.

1.3.3. Code minh họa - Race Condition Detection

Một trong những đặc điểm ngôn ngữ học nổi bật của Go là **race detector** được tích hợp sẵn trong toolchain. Đây là công cụ phân tích động (*dynamic analysis*) phát hiện truy cập đồng thời không an toàn vào vùng nhớ chung - một lỗi cực kỳ khó debug trong các ngôn ngữ như C++ hay Java.

```go
package main

import (
    "fmt"
    "sync"
)

// Ví dụ về DATA RACE — lỗi concurrency kinh điển
// Chạy với: go run -race main.go để Go phát hiện lỗi
func demoRaceCondition() {
    var counter int // Biến dùng chung — nguồn gốc của race condition
    var wg sync.WaitGroup

    // 1000 goroutine cùng tăng counter — KHÔNG AN TOÀN
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++ // Đọc-sửa-ghi: không atomic, dễ bị race condition
        }()
    }
    wg.Wait()
    fmt.Println("Kết quả hàm không dùng mutex:", counter) // Có thể < 1000
}


// Cách Go khuyến nghị: dùng atomic hoặc mutex
func demoSafeCounter() {
    var counter int
    var mu sync.Mutex // Mutex — cơ chế đồng bộ hóa tường minh
    var wg sync.WaitGroup

    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock() // Khóa — chỉ một goroutine được vào tại một thời điểm
            counter++
            mu.Unlock() // Mở khóa — goroutine khác có thể vào
        }()
    }
    wg.Wait()
    fmt.Println("Kết quả hàm dùng mutex:", counter) // Luôn là 1000
}

func main() {
    demoRaceCondition()
    demoSafeCounter()
}
```

Output nhận được khi chạy chương trình:

```
PS C:\Users\Admin> go run "c:\Users\Admin\turtorial.go"
Kết quả hàm không dùng mutex: 992
Kết quả hàm dùng mutex: 1000
```

Chạy lệnh go run -race main.go sẽ kích hoạt race detector, và Go sẽ in cảnh báo chi tiết chỉ ra chính xác dòng code nào gây ra data race - một công cụ mà hầu hết ngôn ngữ khác không có sẵn trong toolchain chuẩn.

1.3.4. Bài toán minh họa nhỏ - So sánh Mutex vs Channel

Đề bài: Mô phỏng bài toán bán vé: 10 goroutine đồng thời cố gắng đặt ghế cuối cùng (remainingSeats = 1). Cài đặt hai phiên bản:

1. Dùng sync.Mutex để bảo vệ biến remainingSeats.
2. Dùng channel (một goroutine giữ "quyền sở hữu" biến, các goroutine khác gửi request qua channel).

So sánh kết quả: cả hai phiên bản phải đảm bảo chỉ đúng một goroutine đặt được ghế, phần còn lại nhận thông báo "hết ghế".

```go
package main

import (
    "fmt"
    "sync"
)

// Phiên bản 1: Mutex
func bookingWithMutex() {
    remainingSeats := 1
    var mu sync.Mutex
    var wg sync.WaitGroup

    for i := 1; i <= 10; i++ {
        wg.Add(1)
        go func(userID int) {
            defer wg.Done()
            mu.Lock()
            defer mu.Unlock() // defer đảm bảo Unlock luôn được gọi

            if remainingSeats > 0 {
                remainingSeats--
                fmt.Printf("[Mutex] User %d: Đặt vé thành công!\n", userID)
            } else {
                fmt.Printf("[Mutex] User %d: Hết ghế.\n", userID)
            }
        }(i)
    }
    wg.Wait()
}
```

```go
// Phiên bản 2: Channel — "share by communicating"
func bookingWithChannel() {
    // Channel có buffer = 1 đóng vai trò như "token" của ghế cuối cùng
    seatToken := make(chan struct{}, 1)
    seatToken <- struct{}{} // Đưa token vào channel — ghế còn trống

    var wg sync.WaitGroup
    for i := 1; i <= 10; i++ {
        wg.Add(1)
        go func(userID int) {
            defer wg.Done()
            select {
            case <-seatToken: // Non-blocking: thử lấy token
                fmt.Printf("[Channel] User %d: Đặt vé thành công!\n", userID)
                // Không trả token lại — ghế đã được đặt
            default: // Không có token → hết ghế
                fmt.Printf("[Channel] User %d: Hết ghế.\n", userID)
            }
        }(i)
    }
    wg.Wait()
}

func main() {
    fmt.Println("=== Mutex ===")
    bookingWithMutex()
    fmt.Println("\n=== Channel ===")
    bookingWithChannel()
}
```

Output nhận được:

```
=== Mutex ===
[Mutex] User 1: Đặt vé thành công!
[Mutex] User 4: Hết ghế.
[Mutex] User 2: Hết ghế.
[Mutex] User 6: Hết ghế.
[Mutex] User 5: Hết ghế.
[Mutex] User 7: Hết ghế.
[Mutex] User 8: Hết ghế.
[Mutex] User 9: Hết ghế.
[Mutex] User 10: Hết ghế.
[Mutex] User 10: Hết ghế.
[Mutex] User 3: Hết ghế.

=== Channel ===
[Channel] User 4: Đặt vé thành công!
[Channel] User 10: Hết ghế.
[Channel] User 5: Hết ghế.
[Channel] User 8: Hết ghế.
[Channel] User 9: Hết ghế.
[Channel] User 2: Hết ghế.
[Channel] User 1: Hết ghế.
[Channel] User 3: Hết ghế.
[Channel] User 7: Hết ghế.
[Channel] User 6: Hết ghế.
```

Nhận xét ngôn ngữ học: Cả hai cách tiếp cận đều đúng và an toàn. Go không bắt buộc dùng channel thay cho mutex; triết lý CSP được khuyến khích nhưng không phải quy tắc cứng. Trong thực tế, mutex phù hợp hơn khi bảo vệ truy cập vào một biến đơn giản, trong khi channel phù hợp hơn khi cần điều phối luồng dữ liệu giữa các goroutine.

CHƯƠNG 2: CƠ HỘI NGHỀ NGHIỆP VÀ NHU CẦU THỊ TRƯỜNG

2.1. Nhu cầu tuyển dụng lập trình viên Backend Go:

Trong những năm gần đây, xu hướng phát triển phần mềm toàn cầu đang dịch chuyển mạnh mẽ từ kiến trúc nguyên khối (Monolithic) sang các kiến trúc hiện đại phân tán như Microservices và Cloud-native. Những hệ thống này đòi hỏi tính sẵn sàng cao (High Availability), khả năng co giãn linh hoạt (Scalability) và tối ưu hóa tài nguyên phần cứng. Đây chính là môi trường lý tưởng để ngôn ngữ Go phát huy tối đa thế mạnh.

Được thiết kế nguyên bản để giải quyết các bài toán hạ tầng của Google, Go sở hữu khả năng biên dịch ra mã máy độc lập (statically compiled), thời gian khởi động cực nhanh, Go được thiết kế để xây dựng các hệ thống backend có hiệu năng cao, xử lý nhiều request đồng thời, triển khai dễ dàng trong môi trường cloud và tiêu tốn ít bộ nhớ. Nhờ vậy, nhiều tập đoàn công nghệ lớn (như Uber, Netflix, Twitch, và tại Việt Nam là Shopee, Grab, VNG...) đã và đang chuyển đổi các dịch vụ cốt lõi sang Go. Sự chuyển dịch này kéo theo nhu cầu tuyển dụng kỹ sư Go tăng vọt.

Tại thị trường Việt Nam, các công ty Product (phát triển sản phẩm), Startup công nghệ và Fintech đang áp dụng Go rộng rãi để xây dựng API Server, hệ thống xử lý dữ liệu lớn hoặc nền tảng Microservices. Các vị trí tuyển dụng phổ biến bao gồm:

- Backend Developer (Golang)
- Software Engineer (Go / Distributed Systems)
- Cloud Backend Developer

Những vị trí này thường yêu cầu ứng viên có kiến thức sâu rộng về:

- Phát triển RESTful API / gRPC với hiệu suất cao.
- Làm việc thành thạo với các hệ quản trị cơ sở dữ liệu (PostgreSQL, MySQL, Redis).

- Tư duy thiết kế kiến trúc Microservices.
- Hệ sinh thái Container và Điện toán đám mây (Docker, Kubernetes, AWS/GCP).

Đặc biệt: Đối với các hệ thống có đặc thù lưu lượng truy cập thay đổi đột ngột và thường xuyên đối mặt với lượng tải khổng lồ (High Traffic) như thương mại điện tử, nền tảng streaming, hay hệ thống đặt vé trực tuyến (khi có sự kiện hoặc phim bom tấn ra mắt), Go chứng minh được sự vượt trội. Đây cũng chính là lý do cốt lõi Go được lựa chọn làm công nghệ nền tảng cho đồ án Hệ thống đặt vé xem phim này.

2.2. Mức lương và lộ trình phát triển:

Một trong những yếu tố thiết thực nhất khi đánh giá một công nghệ là giá trị của nó trên thị trường lao động. Do Go là một ngôn ngữ chuyên trị các hệ thống khó, cộng với việc nguồn cung lập trình viên thành thạo Go chưa bão hòa như các ngôn ngữ truyền thống (Java, PHP), nên mức đãi ngộ dành cho Go Developer luôn nằm trong nhóm dẫn đầu.

Tại thị trường Việt Nam, mức lương tham khảo dựa trên kinh nghiệm thực tế thường được phân bổ như sau:

- Junior Golang Developer (0 – 2 năm kinh nghiệm): Dao động từ 12 – 25 triệu VNĐ/tháng. Ở giai đoạn này, ứng viên cần nắm vững core Go, thao tác Database và viết API chuẩn.
- Middle Golang Developer (2 – 4 năm kinh nghiệm): Dao động từ 25 – 45 triệu VNĐ/tháng. Yêu cầu khả năng tối ưu query, xử lý concurrency an toàn và hiểu biết về caching, message broker.
- Senior Golang Developer (trên 4 năm kinh nghiệm): Có thể đạt 45 – 70 triệu VNĐ/tháng hoặc cao hơn. Đòi hỏi tư duy thiết kế hệ thống tốt, khả năng review code và định hướng kiến trúc.

Bên cạnh đó, với xu hướng làm việc từ xa (Remote/Global Working), lập trình viên Go tại Việt Nam hoàn toàn có cơ hội nhận mức lương tính bằng USD từ các công ty tại Mỹ, Châu Âu, Singapore. Về lộ trình phát triển nghề nghiệp (Career Path), một kỹ sư Go có thể tiến xa theo các nhánh:

- Hướng chuyên gia kỹ thuật: Backend Developer → Senior Backend Engineer → System Architect (Kiến trúc sư hệ thống).
- Hướng hạ tầng/DevOps: Backend Developer → DevOps/SRE Engineer (Kỹ sư vận hành và ổn định hệ thống - do Go là ngôn ngữ lõi của các tool DevOps như Docker/K8s).
- Hướng quản lý: Backend Engineer → Tech Lead → Engineering Manager.

Để đạt được các nấc thang này, ngoài Go, lập trình viên bắt buộc phải trau dồi thêm các kỹ năng về Thiết kế hệ thống (System Design), Kiến trúc phần mềm và Tư duy vận hành Đám mây.

2.3. Lợi thế cạnh tranh:

Ứng viên nắm vững ngôn ngữ Go cùng với tư duy thiết kế hệ thống và kiến trúc phần mềm sẽ có lợi thế cạnh tranh rõ rệt trên thị trường lao động. Việc thành thạo Go không chỉ giúp sinh viên tiếp cận với một công nghệ hiện đại mà còn giúp nâng cao khả năng xây dựng các hệ thống phần mềm có hiệu năng cao, dễ mở rộng và dễ bảo trì.

Trước hết, Go là ngôn ngữ được thiết kế với cú pháp đơn giản, rõ ràng và dễ đọc. Nhờ đặc điểm này, lập trình viên có thể nhanh chóng hiểu được cấu trúc của các dự án lớn, từ đó giúp việc phát triển và bảo trì hệ thống trở nên thuận lợi hơn, đặc biệt trong môi trường làm việc nhóm.

Bên cạnh đó, Go nổi bật với khả năng xử lý đồng thời (concurrency) mạnh mẽ thông qua cơ chế goroutines và channels. Cơ chế này cho phép hệ thống xử lý nhiều tác vụ song song với hiệu năng cao, phù hợp với các ứng dụng backend cần phục vụ số lượng lớn người dùng cùng lúc.

Ngoài ra, Go hiện được sử dụng rộng rãi trong các hệ thống backend hiện đại, đặc biệt là trong các kiến trúc microservices và môi trường cloud computing. Vì vậy, sinh viên nắm vững Go có thể dễ dàng tiếp cận và làm việc với các công nghệ phổ biến như Docker, Kubernetes và các nền tảng điện toán đám mây.

Cuối cùng, việc hiểu và áp dụng các nguyên tắc thiết kế phần mềm khi phát triển hệ thống bằng Go, chẳng hạn như kiến trúc phân tầng và việc tách các thành phần như repository, service, handler và middleware, sẽ giúp lập trình viên hình thành tư duy thiết kế hệ thống tốt hơn. Đây là một kỹ năng quan trọng đối với các kỹ sư phần mềm khi tham gia phát triển các hệ thống phần mềm quy mô lớn trong thực tế.

CHƯƠNG 3: THIẾT LẬP MÔI TRƯỜNG VÀ CẤU TRÚC DỰ ÁN

Trong chương này, trình bày cách **chuẩn bị môi trường** và **tổ chức cấu trúc code** cho dự án booking_cinema_golang.

3.1. Thiết lập môi trường phát triển

3.1.1. Cài đặt Go SDK

Phần này làm theo tài liệu chính thức của Go tại trang Download and install - The Go Programming Language.

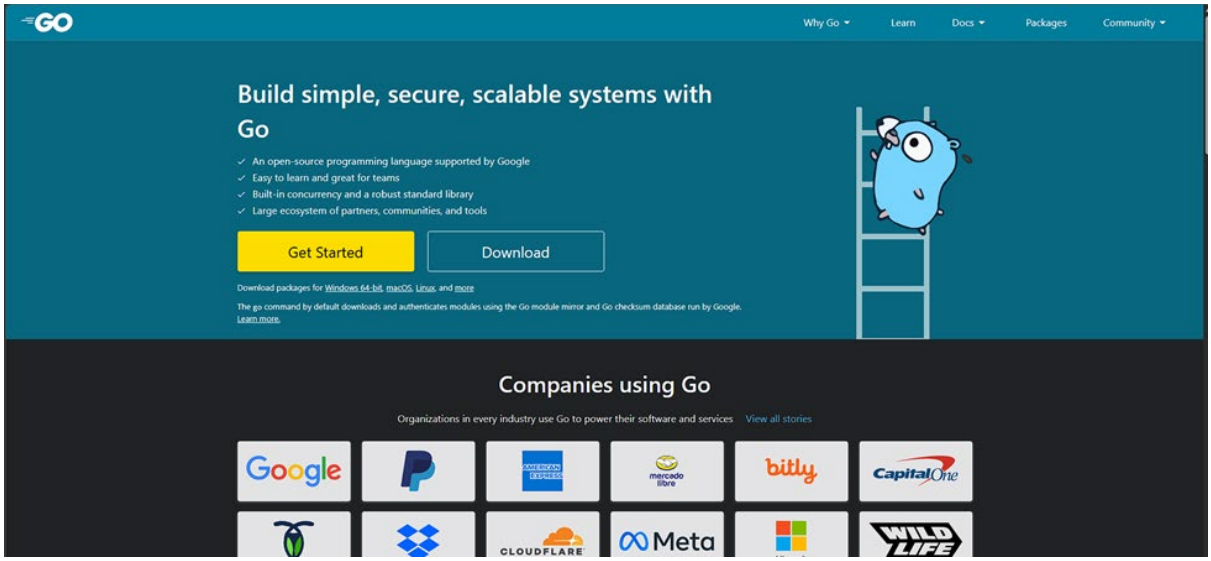
a) Khái niệm đơn giản

- Go SDK: bộ công cụ để lập trình Go.
- Cài xong Go SDK thì trong máy sẽ có lệnh go để:
    - Tạo dự án
    - Tải thư viện
    - Chạy chương trình.

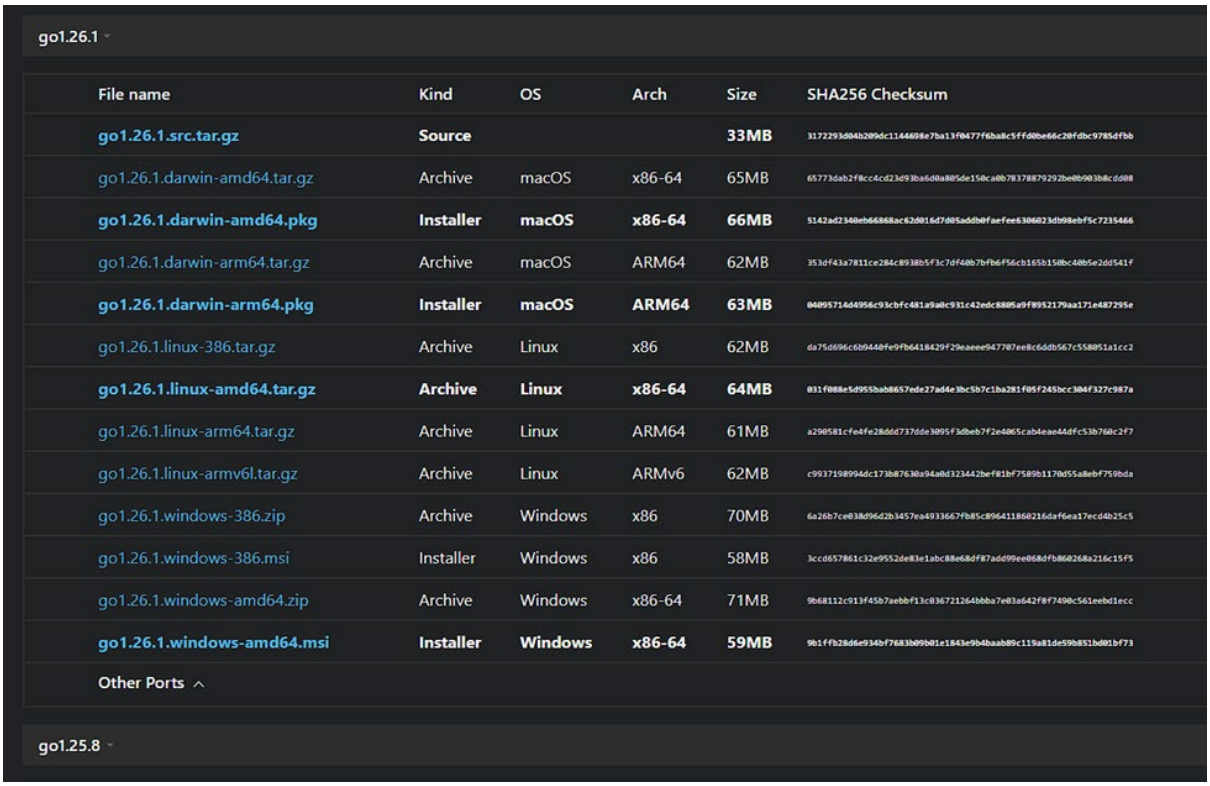
b) Các bước cụ thể trên Windows

Bước 1: Vào trang tải Go

- Mở trình duyệt (Chrome, Edge, …).
- Gõ: https://go.dev/dl rồi Enter.



- Tại đây có danh sách các bản Go. Chọn bản ổn định mới nhất (tài liệu đề xuất Go >= 1.22).



go1.26.1

File name	Kind	OS	Arch	Size	SHA256 Checksum
go1.26.1.src.tar.gz	Source			33MB	3172293d04b209dc1144698e7ba13f0477f6ba8c5ffd0be66c20fdbc9785dfbb
go1.26.1.darwin-amd64.tar.gz	Archive	macOS	x86-64	65MB	65773dab2f8cc4cd23d93ba6d0a805de150ca0b78378879292be0b903b8cdd08
go1.26.1.darwin-amd64.pkg	Installer	macOS	x86-64	66MB	5142ad2340eb66868ac62d016d7d05addb0faefee6306023db98ebf5c7235466
go1.26.1.darwin-arm64.tar.gz	Archive	macOS	ARM64	62MB	353df43a7811ce284c8938b5f3c7df40b7bfb6f56cb165b150bc40b5e2dd541f
go1.26.1.darwin-arm64.pkg	Installer	macOS	ARM64	63MB	040957146495 6c93cbfc481a9a0c931c42edc8805a9f8952179aa171e487295e
go1.26.1.linux-386.tar.gz	Archive	Linux	x86	62MB	da75d696c6b9440fe9fb6418429f29eaeee947707ee8c6ddb567c558051a1cc2
go1.26.1.linux-amd64.tar.gz	Archive	Linux	x86-64	64MB	031f088e5d955bab8657ede27ad4e3bc5b7c1ba281f05f245bcc304f327c987a
go1.26.1.linux-arm64.tar.gz	Archive	Linux	ARM64	61MB	a290581cfe4fe28ddd737dde3095f3dbeb7f2e4065cab4eae44dfc53b760c2f7
go1.26.1.linux-armv6l.tar.gz	Archive	Linux	ARMv6	62MB	c9937198994dc173b87630a94a0d323442bef81bf7589b1170d55a8ebf759bda
go1.26.1.windows-386.zip	Archive	Windows	x86	70MB	6a26b7ce038d96d2b3457ea4933667fb85c896411860216daf6ea17ecd4b25c5
go1.26.1.windows-386.msi	Installer	Windows	x86	58MB	3ccd657861c32e9552de83e1abc88e68df87add99ee068dfb860268a216c15f5
go1.26.1.windows-amd64.zip	Archive	Windows	x86-64	71MB	9b68112c913f45b7aebbf13c036721264bbba7e03a642f8f7498c561eebd1ecc
go1.26.1.windows-amd64.msi	Installer	Windows	x86-64	59MB	9b1ffb28d6e934bf7683b09b01e1843e9b4baab89c119a81de59b851bd01bf73
Other Ports					

go1.25.8

Bước 2: Tải file cài đặt cho Windows (Nếu sử dụng Linux hoặc MacOS thì dựa trên cột "OS" tải bản tương ứng)

- Ở phần Windows, chọn file dạng goX.Y.Z.windows-amd64.msi (bản 64-bit phổ biến).

go1.26.1

File name	Kind	OS	Arch	Size	SHA256 Checksum
go1.26.1.src.tar.gz	**Source**			**33MB**	3172293d04b209dc1144698e7ba13f0477f6ba8c5ffd0be66c20fdbc97854fbb
go1.26.1.darwin-amd64.tar.gz	Archive	macOS	x86-64	65MB	65773dab2f8cc4cd23d93ba6d0a805de150ca0b78378879292be0b903b8cdd08
go1.26.1.darwin-amd64.pkg	**Installer**	**macOS**	**x86-64**	**66MB**	5142ad2340eb66868ac62d016d7d05addb0faefee6306023db98ebf5c7235466
go1.26.1.darwin-arm64.tar.gz	Archive	macOS	ARM64	62MB	353df43a7811ce284c8938b5f3c7df40b7bfb6f56cb165b150bc40b5e2dd541f
go1.26.1.darwin-arm64.pkg	**Installer**	**macOS**	**ARM64**	**63MB**	040957144d4956c93cbfc481a9a0c931c42edc8805a9f8952179aa171e487295e
go1.26.1.linux-386.tar.gz	Archive	Linux	x86	62MB	da75d696c6b9440fe9fb6418429f29eaeee947707ee8c6ddb567c558051a1cc2
go1.26.1.linux-amd64.tar.gz	**Archive**	**Linux**	**x86-64**	**64MB**	031f088e5d955bab8657ede27ad4e3bc5b7c1ba281f05f245bcc304f327c987a
go1.26.1.linux-arm64.tar.gz	Archive	Linux	ARM64	61MB	a290581cfe4fe28ddd737dde3095f3dbeb7f2e4065cab4eae44dfc53b760c2f7
go1.26.1.linux-armv6l.tar.gz	Archive	Linux	ARMv6	62MB	c9937198994dc173b87630a94a0d323442bef81bf7589b1170d55a8ebf759bda
go1.26.1.windows-386.zip	Archive	Windows	x86	70MB	6a26b7ce038d96d2b3457ea4933667fb85c8964118601216daf6ea17ecd4b25c5
go1.26.1.windows-386.msi	Installer	Windows	x86	58MB	3ccd657861c32e9552de83e1abc88e68df87add99ee068dfb860268a216c15f5
go1.26.1.windows-amd64.zip	Archive	Windows	x86-64	71MB	9b68112c913f45b7aebbf13c03672126 4bbba7e03a642f8f7490c561eebd1ecc
go1.26.1.windows-amd64.msi	**Installer**	**Windows**	**x86-64**	**59MB**	9b1ffb28d6e934bf7683b09b01e1843e9b4baab89c119a81de59b851bd01bf73
Other Ports ^					

go1.25.8

- Nhấn tải về và chờ hoàn tất.

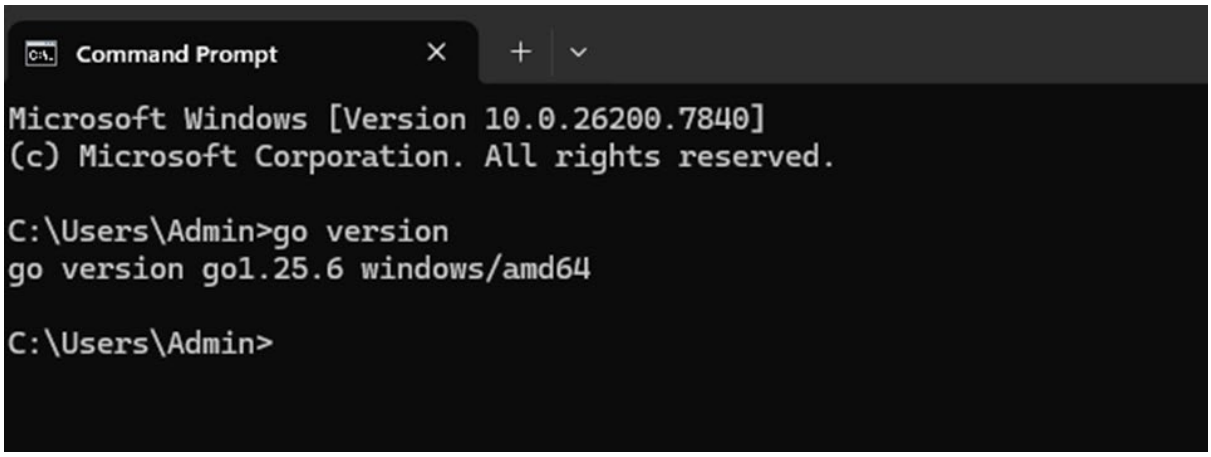
Bước 3: Chạy file cài đặt

- Mở thư mục tải xuống (Downloads).
- Nháy đúp vào file .msi vừa tải.
- Cứ Next → Next theo hướng dẫn:
  - Giữ nguyên đường dẫn mặc định (thường là C:\Program Files\Go).
  - Nhấn **Install**.
  - Đợi cài xong, nhấn **Finish**.

Bước 4: Kiểm tra Go đã cài thành công

- Mở **Command Prompt** hoặc **PowerShell**.
- Gõ: **go version**

- Nếu hiện ra dạng: **go version go1.22.x windows/amd64 là cài đặt thành công.**



3.1.2. Hiểu đơn giản về GOROOT và GOPATH

Theo tài liệu chính thức của Go (Managing Go installations và Setting GOPATH), hai khái niệm hay gặp là:

- GOROOT:
  - Là thư mục cài Go.
  - Ví dụ: C:\Program Files\Go.
  - Bên trong có sẵn các file chuẩn của ngôn ngữ Go.
  - Thông thường không cần tự đặt, Go tự biết.
- GOPATH:
  - Là thư mục làm việc mặc định của Go: nơi chứa các dự án, thư viện tải về.
  - Mặc định trên Windows là:
    - %USERPROFILE%\go, ví dụ: C:\Users\Admin\go.
  - Go dùng GOPATH để lưu:
    - source code bên thứ ba,
    - các binary cài bằng go install, …

Lưu ý:

- Thông thường không cần tự đặt lại GOROOT và GOPATH.
- Không được để GOROOT và GOPATH trùng nhau.
- Trong máy, dự án booking_cinema_golang được để ở thư mục riêng:

C:\Users\Admin\booking_cinema_golang

Nên tách biệt hoàn toàn với C:\Program Files\Go (GOROOT) và C:\Users\Admin\go (GOPATH).

Nếu muốn xem giá trị hiện tại, có thể gõ trong terminal:

go env GOROOT

go env GOPATH

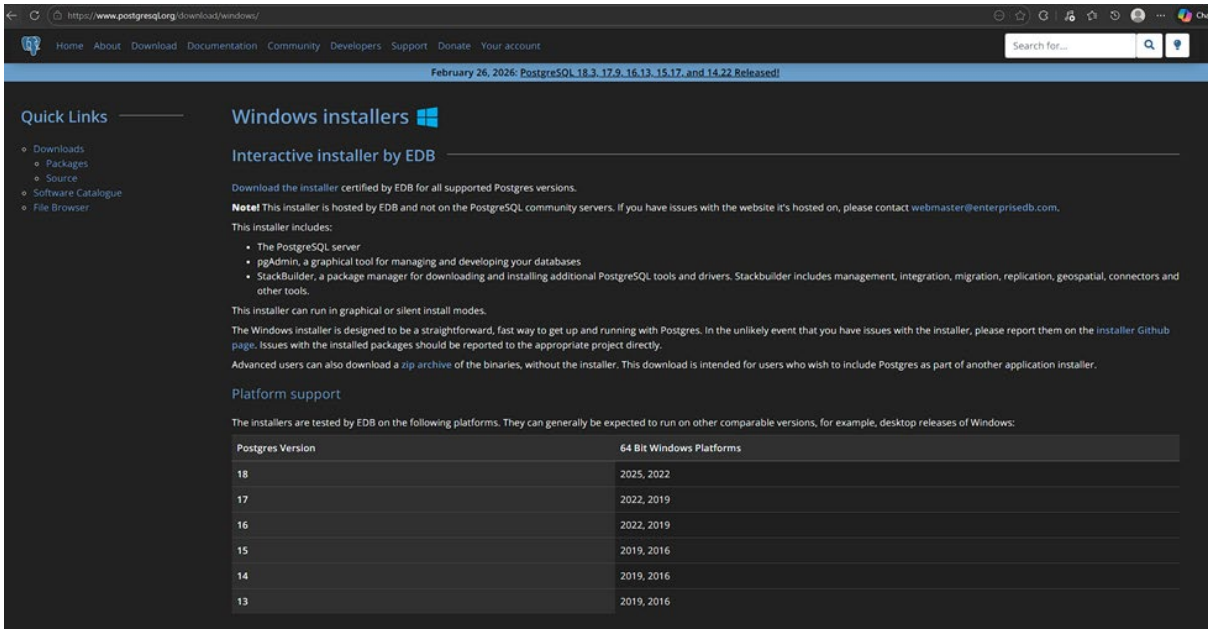
3.1.3. Thiết lập PostgreSQL

Để lưu thông tin người dùng, rạp chiếu, vé,… dự án dùng **PostgreSQL** làm hệ quản trị cơ sở dữ liệu.

a) Cài đặt PostgreSQL (dành cho người không rành kỹ thuật)

Bước 1: Tải bộ cài

- Mở trình duyệt, tìm: postgresql download windows.
- Chọn trang chính thức



- Tải bản mới ổn định (ví dụ PostgreSQL 18, hỗ trợ nền tảng 64 bit 2022-2025).

Bước 2: Chạy file cài đặt

- Nháy đúp file .exe tải về.
- Cứ Next theo các bước:
  - Chọn đường dẫn cài (mặc định thường ổn).
  - Đặt mật khẩu cho user postgres (nhớ lưu lại để sau kết nối từ Go).
  - Để port mặc định là 5432.

Bước 3: Hoàn tất

- Sau khi cài xong, PostgreSQL sẽ chạy như một dịch vụ trên Windows.
- Có thể có thêm công cụ pgAdmin để quản lý bằng giao diện.

b) Dùng DBeaver hoặc pgAdmin để quản lý CSDL

Để người không rành lệnh cũng thao tác được, em dùng **công cụ có giao diện**:

- pgAdmin: thường cài kèm với PostgreSQL.
- Hoặc DBeaver (tải từ trang chính thức https://dbeaver.io/).

Ví dụ: dùng pgAdmin

1. Mở ứng dụng pgAdmin.
2. Khi mở lần đầu, nhập mật khẩu đã đặt cho postgres.
3. Thêm kết nối:
    - Host: localhost
    - Port: 5432
    - User: postgres
    - Password: mật khẩu đã tạo khi cài.
4. Sau khi kết nối được, có thể:
    - Tạo database mới dành cho dự án, ví dụ đặt tên booking_cinema.
    - Chọn "Create Database" → nhập tên → Save.

Sau này, file schema SQL của dự án (internal/database/migrations/schema.sql hoặc schema.sql ở root) sẽ được chạy trên database này để tạo các bảng: users, cinemas, showtimes, bookings, payments, …

3.2. Quản lý dependency với Go Modules

Dự án booking_cinema_golang dùng **Go Modules** để quản lý thư viện bên thứ ba. Đây là cách chính thức mà Go khuyến nghị

3.2.1. Khái niệm đơn giản

- Go module: có thể hiểu là một "gói dự án" Go, được định nghĩa bởi file go.mod nằm ở thư mục gốc.
- Trong go.mod sẽ có:

    - Tên module (thường là tên repo),
    - Phiên bản Go,
    - Danh sách các thư viện bên ngoài mà dự án cần.

Khi dùng Go Modules:

- Không cần để dự án trong GOPATH vẫn chạy bình thường.
- Dự án của em đang đặt tại:

C:\Users\Admin\booking_cinema_golang

là hoàn toàn hợp lệ.

3.2.2. Khởi tạo Go module

Hiện tại trong dự án đã có file go.mod, nhưng nếu mô tả quy trình từ con số 0, các bước sẽ như sau.

Bước 1: Mở terminal tại thư mục dự án

- Mở PowerShell.
- Gõ: cd C:\Users\Admin\booking_cinema_golang

Bước 2: Khởi tạo module

- Gõ lệnh: go mod init github.com/ten-github/booking_cinema_golang
- Trong thực tế, nếu repo chưa public, có thể dùng tên đơn giản hơn, nhưng dạng github.com/... là phổ biến hơn
- Lệnh này sẽ tạo file go.mod ở thư mục gốc với nội dung :

  module github.com/ten-cua-em/booking_cinema_golang go 1.22

3.2.3. Tự động tải và dọn dẹp dependency với go mod tidy

Khi import một thư viện mới trong code (ví dụ github.com/go-chi/chi/v5), Go sẽ cần tải về phiên bản tương ứng.

Bước 1: Gõ lệnh go mod tidy

- Vẫn ở thư mục gốc dự án:

cd C:\Users\Admin\booking_cinema_golang

go mod tidy

Ý nghĩa lệnh này:

- Kiểm tra tất cả các file .go trong dự án,
- Tìm xem có import thư viện nào không nằm trong go.mod không,
    - Nếu có, tự động thêm vào go.mod và tải về (vào cache của Go, thường nằm trong GOPATH/pkg/mod),
- Đồng thời xoá bớt các thư viện không còn dùng nữa khỏi go.mod và go.sum.

Nhờ đó:

- Dự án luôn có danh sách dependency rõ ràng,
- Người khác clone repo về, chỉ cần chạy "go mod tidy" Go sẽ tải đúng các thư viện cần dùng.

3.3. Xây dựng cấu trúc thư mục theo Clean Architecture

Dự án booking_cinema_golang được tổ chức theo nguyên tắc của Clean Architecture, trong đó hệ thống được chia thành nhiều lớp với trách nhiệm rõ ràng. Các lớp bên ngoài phụ thuộc vào lớp bên trong, còn lõi nghiệp vụ (business logic) được tách biệt khỏi các chi tiết triển khai như HTTP hay cơ sở dữ liệu.

Cấu trúc thư mục chính của dự án gồm các thành phần sau:

1. Lớp Entry Point (Application Entry)

Thư mục cmd/ chứa các chương trình khởi động hệ thống.

```
cmd/
├── api/main.go
└── frontend/main.go
```

- cmd/api/main.go: khởi tạo HTTP server cho backend API, cấu hình router, middleware và các dependency cần thiết.
- cmd/frontend/main.go: chạy server tĩnh để phục vụ giao diện người dùng từ thư mục frontend.

Đây là nơi kết nối các thành phần của hệ thống lại với nhau và khởi động ứng dụng.

2. Lớp Domain (Entities)

Thư mục:

internal/domain/

Chứa các entity và model cốt lõi của hệ thống, ví dụ:

models.go

Các struct như:

- User
- Cinema
- Booking
- Payment

Lớp Domain đại diện cho dữ liệu và quy tắc nghiệp vụ cơ bản, và không phụ thuộc vào bất kỳ framework hay cơ sở dữ liệu nào.

3. Lớp Use Case / Business Logic

Thư mục:

internal/service/

Chứa các service xử lý nghiệp vụ của hệ thống, ví dụ:

- auth_service.go
- booking_service.go
- cinema_service.go
- payment_service.go
- email_service.go

Các service này thực hiện các logic như:

- xác thực người dùng,
- xử lý đặt vé,
- kiểm tra ghế trống,
- xử lý thanh toán,
- gửi email xác nhận.

Đây là lớp trung tâm của ứng dụng, nơi thực thi các quy tắc nghiệp vụ.

4. Lớp Data Access (Repository)

Thư mục:

internal/repository/

Chứa các thành phần chịu trách nhiệm truy cập và thao tác với cơ sở dữ liệu.

Ví dụ:

- user_repository.go
- cinema_repository.go
- booking_repository.go
- payment_repository.go

Các repository thực hiện các thao tác như:

- truy vấn dữ liệu
- thêm, sửa, xóa dữ liệu
- tìm kiếm thông tin trong database

Lớp này giúp tách biệt logic nghiệp vụ khỏi cách lưu trữ dữ liệu.

5. Lớp Interface / Delivery

Thư mục:

internal/handler/

Chứa các **HTTP handler** chịu trách nhiệm giao tiếp với client.

Ví dụ:

- auth_handler.go
- booking_handler.go
- cinema_handler.go
- payment_handler.go

Các handler có nhiệm vụ:

- nhận HTTP request
- xử lý input
- gọi các service tương ứng
- trả về HTTP response cho client

Đây là lớp kết nối giữa hệ thống backend và người dùng hoặc frontend.

6. Lớp Middleware

Thư mục:

internal/middleware/

Chứa các middleware xử lý chung cho request, ví dụ:

- auth_middleware.go: xử lý xác thực người dùng bằng JWT
- logger_middleware.go: ghi log request

Middleware được sử dụng để thực hiện các chức năng chung trước hoặc sau khi request được xử lý.

7. Lớp Infrastructure

Một số thành phần hạ tầng được đặt trong các thư mục:

internal/config/
internal/database/

Bao gồm:

- config.go: quản lý cấu hình hệ thống (biến môi trường, file cấu hình)
- postgres.go: thiết lập kết nối với cơ sở dữ liệu PostgreSQL
- migrations/schema.sql: định nghĩa cấu trúc bảng dữ liệu

Các thành phần này cung cấp hạ tầng kỹ thuật cho ứng dụng.

8. Lớp Utilities

Thư mục:

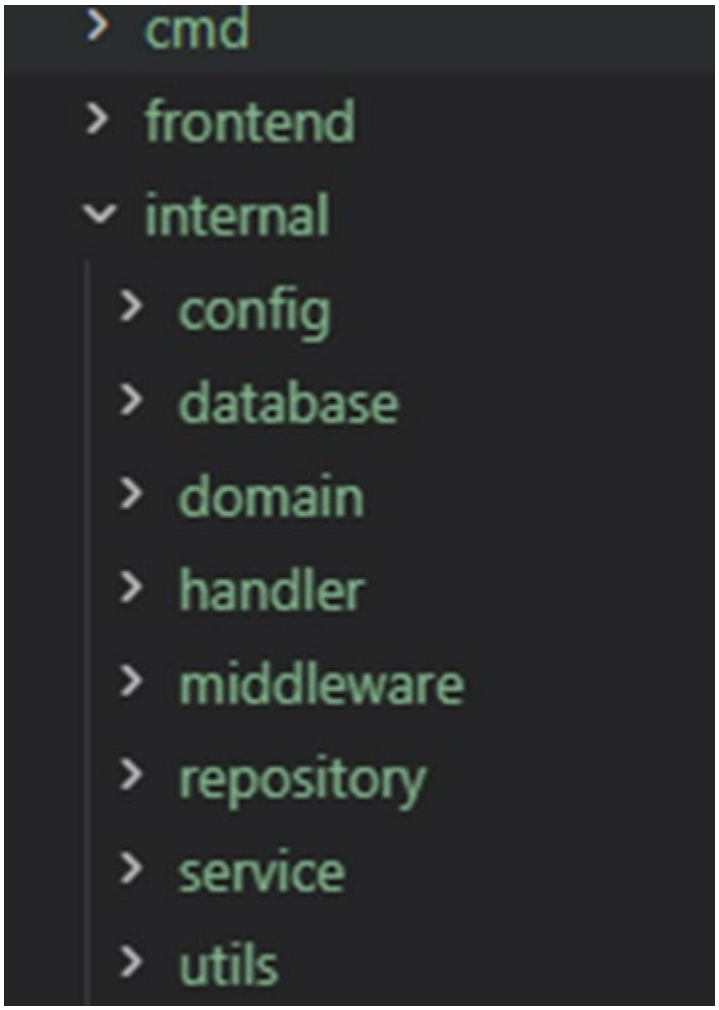
internal/utils/

Chứa các hàm tiện ích dùng chung trong toàn bộ hệ thống, ví dụ:

- jwt.go: tạo và xác thực JWT token
- otp.go: xử lý mã OTP
- response.go: chuẩn hóa format trả về JSON
- validator.go: kiểm tra dữ liệu đầu vào

3.3.1. Kiến trúc thư mục tổng thể

Dựa trên cấu trúc code hiện tại, các thư mục chính như sau:



- cmd/api/
  - main.go: chương trình chính cho HTTP API server (backend).
- cmd/frontend/
  - main.go: chương trình chính cho server tĩnh phục vụ front end từ thư mục frontend.
- internal/config/
  - config.go: xử lý cấu hình ứng dụng (biến môi trường, file cấu hình, …).
- internal/database/
  - postgres.go: kết nối tới PostgreSQL.

    - migrations/schema.sql: định nghĩa các bảng trong CSDL.
- internal/domain/
    - models.go: khai báo các struct như User, Cinema, Booking, Payment, …
- internal/repository/
    - user_repository.go, cinema_repository.go, booking_repository.go, payment_repository.go:
        - Định nghĩa interface và/hoặc impl để truy vấn DB.
- internal/service/
    - auth_service.go, booking_service.go, cinema_service.go, payment_service.go, email_service.go:
        - Chứa logic nghiệp vụ chính (business rules).
- internal/handler/
    - auth_handler.go, booking_handler.go, cinema_handler.go, payment_handler.go:
        - "Tiếp xúc" trực tiếp với HTTP request.
- internal/middleware/
    - auth_middleware.go: xử lý xác thực (JWT, …).
    - logger_middleware.go: log request.
    -
- internal/utils/
    - jwt.go, otp.go, response.go, validator.go:
        - Các hàm tiện ích (tạo token, gửi JSON chuẩn, validate input, …).
- frontend/
    - index.html, styles.css, app.js:
        - Giao diện React đơn giản, flow đặt vé từ phía client.

3.3.2. Vai trò của từng tầng trong mô hình

a) internal/domain/ + schema.sql

- Model là cách mô tả dữ liệu trong hệ thống.
- Ví dụ:
  - Một User có: ID, Email, PasswordHash, FullName, …
  - Một Cinema có: ID, Name, Address, …
  - Một Booking có: ID, UserID, ShowtimeID, Seats, …

Trong Go, các model này được định nghĩa trong file internal/domain/models.go dưới dạng các struct.

Trong PostgreSQL, cấu trúc tương ứng được định nghĩa trong file SQL:

- internal/database/migrations/schema.sql
- (hoặc bản copy schema.sql ở root repo).

Ý nghĩa:

- File .go mô tả dữ liệu trong code.
- File .sql mô tả dữ liệu trong database.
- Hai bên phải khớp với nhau (kiểu dữ liệu, tên trường, …).

b) Tầng Repository – internal/repository/

- Repository là nơi viết code để đọc / ghi dữ liệu từ CSDL PostgreSQL.
- Ví dụ:
  - UserRepository có hàm:
    - CreateUser(...),
    - FindByEmail(email string),
    - GetByID(id int), …
  - BookingRepository có hàm:
    - CreateBooking(...),
    - GetBookingsByUser(userID int), …

Lý do tách ra so với tầng model :

- Nếu sau này đổi từ PostgreSQL sang MySQL, chỉ cần đổi code ở tầng repository.
- Các tầng khác (service, handler) ít bị ảnh hưởng.

c) Tầng Service – internal/service/

- Service là nơi viết logic nghiệp vụ.
- Ví dụ:
    - Trong BookingService:
        - Kiểm tra ghế còn trống không.
        - Tính tổng tiền.
        - Gọi repository để lưu Booking.
    - Trong AuthService:
        - Kiểm tra email đã tồn tại chưa.
        - Hash mật khẩu.
        - Tạo token đăng nhập (JWT).
    - Trong PaymentService:
        - Kiểm tra booking hợp lệ.
        - Lưu thông tin thanh toán, sinh transaction_id, …

Service thường:

- Không biết chi tiết HTTP request,
- Không render HTML,
- Chỉ nhận dữ liệu "thô" (struct, string, số, …) và trả kết quả hoặc lỗi.

d) Tầng Handler – internal/handler/

Đây là phần gần giống Controller trong MVC.

- Các file như:
  - auth_handler.go,
  - booking_handler.go,
  - cinema_handler.go,
  - payment_handler.go.

Nhiệm vụ của handler:

1. **Nhận request HTTP** từ router (ví dụ POST /api/auth/login).
2. **Đọc dữ liệu từ body** (JSON).
3. **Validate** dữ liệu (gọi validator nếu cần).
4. **Gọi service phù hợp**, truyền dữ liệu vào.
5. **Nhận kết quả** từ service.
6. **Trả JSON response** về cho front end (dùng helper trong internal/utils/response.go).

Ví dụ luồng Đăng nhập:

- Front end gửi POST /api/auth/login với body: { "email": "...", "password": "..." }.
- AuthHandler:
  - Đọc email, password.
  - Gọi AuthService.Login(email, password).
  - Nếu thành công, trả về JSON chứa thông tin user + token (tuỳ thiết kế).
  - Nếu thất bại, trả về mã lỗi và thông báo tương ứng.

e) Tầng Router và Middleware – cmd/api/ + internal/middleware/

- File cmd/api/main.go là nơi:
  - Khởi tạo router (dùng net/http hoặc thư viện như chi, gorilla/mux, …).
  - Khai báo các đường dẫn (route) cho API:
    - POST /api/auth/register → AuthHandler.Register
    - POST /api/auth/login → AuthHandler.Login
    - GET /api/cinemas → CinemaHandler.ListCinemas
    - GET /api/showtimes → CinemaHandler.ListShowtimes
    - GET /api/showtimes/{id}/seats → BookingHandler.ListTakenSeats
    - POST /api/bookings → BookingHandler.CreateBooking
    - POST /api/payments → PaymentHandler.CreatePayment
- **Middleware** trong internal/middleware/:
  - auth_middleware.go:
    - Kiểm tra trong request có token không.
    - Giải mã token (dùng internal/utils/jwt.go).
    - Nếu hợp lệ, cho qua, gắn thông tin user vào context.
    - Nếu không hợp lệ, trả lỗi 401 (Unauthorized).
  - logger_middleware.go:
    - Ghi log mỗi request: phương thức, đường dẫn, thời gian xử lý, …

Các middleware này được "gắn" vào router trong cmd/api/main.go, để mọi request đi qua đều được xử lý chung (giống chốt bảo vệ trước cổng).

3.3.3. Luồng xử lý request end-to-end trong dự án

Để kết nối lý thuyết với thực tế dự án, dưới đây là ví dụ **luồng "đặt vé"** từ khi người dùng bấm nút trên giao diện đến khi dữ liệu vào DB.

Bước 1: Người dùng thao tác trên giao diện

- Mở http://localhost:5173/ (được phục vụ bởi cmd/frontend).
- Chọn rạp, ngày, suất chiếu, ghế, và bấm "Đặt vé".

Bước 2: Front end gọi API

- app.js gọi POST /api/bookings
- Gửi JSON như:

```
{
  "cinema_id": 1,
  "showtime_id": 10,
  "seats": ["A1", "A2"]
}
```

Bước 3: Router nhận request

- Trong cmd/api/main.go, router đã gắn route:

POST /api/bookings → BookingHandler.CreateBooking

- Nên request đi vào hàm CreateBooking trong internal/handler/booking_handler.go.

Bước 4: Handler xử lý lớp ngoài cùng

- Handler:
    - Đọc body JSON,
    - Kiểm tra format,

    - Gọi bookingService.CreateBooking(...).

Bước 5: Service xử lý logic nghiệp vụ

- BookingService:
    - Gọi BookingRepository để kiểm tra ghế đã được đặt chưa.
    - Nếu ghế còn trống:
        - Tính toán số tiền.
        - Gọi repository để **insert** bản ghi booking mới.
    - Trả về kết quả (thông tin booking) cho handler.

Bước 6: Repository làm việc với DB

- BookingRepository:
    - Dùng kết nối PostgreSQL trong internal/database/postgres.go.
    - Thực thi câu INSERT vào bảng bookings được định nghĩa trong schema.sql.

Bước 7: Handler trả JSON response

- Nhận kết quả từ service,
- Gọi helper trong internal/utils/response.go để gửi JSON chuẩn lại cho front end,
- Front end cập nhật giao diện (ví dụ hiển thị "Đặt vé thành công").

3.3.4. Lý do chọn cấu trúc Clean Architecture

Cấu trúc của dự án tuân theo phong cách được cộng đồng Go khuyến nghị:

- **cmd/**:
    - Mỗi thư mục con trong cmd/ tương ứng với **một chương trình chính** (một binary).
    - Ở đây:
        - cmd/api → chương trình API server.

        - cmd/frontend → chương trình server tĩnh cho front end.
- **internal/**:
    - Là thư mục chứa **logic "private"** của ứng dụng.
    - Theo quy tắc của Go, các package trong internal/ **chỉ được import từ trong cùng module**, không bị project khác import trực tiếp.
    - Điều này giúp:
        - Giữ cho logic nghiệp vụ không bị "lộ" ra bên ngoài.
        - Ép dự án phải dùng qua các entrypoint cmd/... chứ không "ăn xổi" import lung tung.

3.3.5. Tóm tắt chương

- Tóm lại trong chương đã triển khai các chủ đề chính sau:
    - **Cài đặt Go SDK** theo tài liệu go.dev, hiểu khái niệm GOROOT và GOPATH ở mức đơn giản.
    - **Cài và cấu hình PostgreSQL**, sử dụng công cụ giao diện (pgAdmin hoặc DBeaver) để dễ thao tác với người không rành kỹ thuật.
    - Dùng **Go Modules** với file go.mod và lệnh go mod tidy để quản lý thư viện bên thứ ba một cách tự động, rõ ràng.
    - Thiết kế **cấu trúc thư mục** dự án booking_cinema_golang theo kiểu:
        - cmd/ (entrypoint chương trình),
        - internal/ (logic lõi: config, database, domain, repository, service, handler, middleware, utils),
        - frontend/ (view, giao diện React).
    - Giải thích rõ vai trò của từng tầng trong mô hình **MVC mở rộng**, và mô tả chi tiết **luồng xử lý một request đặt vé** từ giao diện tới cơ sở dữ liệu.

Nhờ những bước này, dự án có **nền tảng môi trường và cấu trúc code rõ ràng**, thuận tiện để có thể tiếp tục phát triển các chức năng chi tiết ở các chương sau.

CHƯƠNG 4: CÁC KHÁI NIỆM NỀN TẢNG - MIDDLEWARE TRONG GO

4.1. Middleware là gì? Phân tích từ góc độ ngôn ngữ học

4.1.1. Định nghĩa và cơ chế hoạt động

Trong Go, middleware không phải là một từ khóa hay cấu trúc ngôn ngữ riêng biệt - nó là một design pattern được xây dựng hoàn toàn từ hệ thống kiểu (*type system*) của ngôn ngữ. Cụ thể, middleware trong Go là một hàm có chữ ký (*signature*) dạng:

func(next http.Handler) http.Handler

Để hiểu tại sao chữ ký này lại là nền tảng, cần phân tích từng thành phần:

- http.Handler là một interface được định nghĩa trong thư viện chuẩn của Go: bất kỳ kiểu nào có phương thức ServeHTTP(ResponseWriter, *Request) đều thỏa mãn interface này - đây là cơ chế structural typing đặc trưng của Go.
- http.HandlerFunc là một function type được khai báo là type HandlerFunc func(ResponseWriter, *Request), và nó cài đặt ServeHTTP bằng cách tự gọi chính nó - cho phép chuyển đổi linh hoạt giữa hàm thông thường và interface.
- Tham số next http.Handler biểu diễn handler tiếp theo trong chuỗi xử lý - đây là cách Go hiện thực hóa mô hình xử lý tuần tự mà không cần framework.

Middleware hoạt động theo nguyên lý bọc ngoài (wrapping): mỗi middleware nhận một handler, tạo ra một handler mới bao quanh handler gốc, và trả về handler mới đó. Request đi vào từ middleware ngoài cùng, xuyên qua từng lớp, đến handler cuối, rồi response đi ngược ra. Đây là cấu trúc Decorator Pattern được thể hiện thuần túy qua hệ thống kiểu của Go mà không cần class hay kế thừa.

So sánh với các ngôn ngữ khác:

Tiêu chí	Go	Java (Spring)	Python (Django)	Node.js (Express)
Cơ chế middleware	Higher-order function + interface	Annotation (@Aspect)	Callable class / decorator	Function (req, res, next)
Kiểm tra kiểu	Compile time	Compile time	Runtime	Runtime
Phụ thuộc framework	Không (stdlib)	Có (Spring AOP)	Có (Django)	Có (Express)
Cú pháp khởi tạo	func(next Handler) Handler	@Before, @Around	def __call__	(req, res, next) =>
Chainability	Manual wrapping hoặc thư viện	Framework tự quản lý	MIDDLEWARE list	app.use()

Điểm đáng chú ý: Go không có "magic" - middleware chỉ là hàm thông thường được compose lại với nhau. Điều này khác hoàn toàn với Spring AOP (dùng bytecode manipulation ở runtime) hay Django middleware (framework tự gọi theo thứ tự list). Go đặt toàn quyền kiểm soát vào tay lập trình viên thông qua việc lập trình hàm bậc cao (*higher-order functions*).

4.1.2. Cấu trúc cơ bản của một Middleware trong Go

Bước 1 - Giải thích chi tiết cấu trúc:

```go
package main

import "net/http"

// Chữ ký chuẩn của mọi middleware trong Go
func MiddlewareName(next http.Handler) http.Handler {
    // http.HandlerFunc chuyển một closure thành http.Handler
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // === TIỀN XỬ LÝ (Pre-processing) ===
        // Code ở đây chạy TRƯỚC khi handler gốc được gọi
        // Ví dụ: kiểm tra token, ghi log request, thêm header...

        next.ServeHTTP(w, r) // Gọi handler tiếp theo trong chuỗi

        // === HẬU XỬ LÝ (Post-processing) ===
        // Code ở đây chạy SAU khi handler gốc đã xử lý xong
        // Ví dụ: tính thời gian xử lý, ghi log response...
    })
}
```

Phân tích cơ chế ngôn ngữ trong cấu trúc này:

- Closure: Hàm ẩn danh bên trong http.HandlerFunc(...) là một closure - nó "bắt" (*capture*) biến next từ scope bên ngoài. Go hỗ trợ closure hoàn toàn, biến next sẽ tồn tại trong bộ nhớ heap chừng nào closure còn được tham chiếu.
- Type conversion: http.HandlerFunc(func(...){...}) là một type conversion, không phải function call - Go chuyển đổi một function literal thành kiểu HandlerFunc vốn cài đặt interface http.Handler.
- Interface satisfaction ngầm: Vì HandlerFunc có phương thức ServeHTTP, nó tự động thỏa mãn http.Handler mà không cần khai báo implements - đây là
- structural typing của Go.

4.1.3. Chuỗi Middleware (Middleware Chain) - Composition hàm

Sức mạnh thực sự của middleware trong Go nằm ở khả năng **compose** nhiều middleware lại với nhau. Đây là một ứng dụng của tư tưởng *composition over inheritance* - triết lý cốt lõi trong thiết kế Go.

```go
package main

import "net/http"

// Giả sử có 3 middleware: Logger, Auth, RateLimit
// và một handler cuối: bookingHandler

// Cách compose thủ công (không dùng thư viện):
// Đọc từ trong ra ngoài:
// 1. bookingHandler được bọc bởi Auth
// 2. Kết quả đó được bọc bởi Logger
// 3. Kết quả đó được bọc bởi RateLimit
finalHandler := RateLimit(Logger(Auth(bookingHandler)))

// Khi request đến, thứ tự thực thi là:
// RateLimit (pre) → Logger (pre) → Auth (pre)
//   → bookingHandler xử lý
// Auth (post) → Logger (post) → RateLimit (post) → response
```

Để tránh việc lồng nhau gây khó đọc, các dự án thực tế thường viết hàm Chain tổng quát:

```go
// Chain nhận một handler gốc và danh sách middleware,
// trả về handler đã được bọc bởi tất cả middleware theo đúng thứ tự
func Chain(h http.Handler, middlewares ...func(http.Handler) http.Handler) http.Handler {
    // Duyệt ngược để middleware đầu tiên trong slice
    // thực sự là lớp ngoài cùng (chạy đầu tiên)
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

// Sử dụng:
finalHandler := Chain(
    bookingHandler,
    Recovery,   // Chạy ngoài cùng — bắt panic trước tiên
    Logger,     // Ghi log mọi request
    Auth,       // Kiểm tra token
    RateLimit,  // Giới hạn tần suất
)
```

4.1.4. Bài toán minh họa nhỏ - Tự viết Chain function

Đề bài: Viết hàm Chain(h http.Handler, middlewares ...func(http.Handler) http.Handler) http.Handler. Định nghĩa hai middleware đơn giản: WithRequestID (gán một UUID giả vào header X-Request-ID) và WithTimer (in ra thời gian xử lý request). Tạo một handler in ra "Hello, Cinema!" rồi bọc nó bằng hai middleware trên theo thứ tự: WithRequestID chạy trước, WithTimer chạy sau.

Mục tiêu: Quan sát rằng Go không cần framework để compose middleware - toàn bộ là hàm bậc cao và closure.

```go
// --- Hàm Chain tổng quát ---
func Chain(h http.Handler, mws ...func(http.Handler) http.Handler) http.Handler {
    for i := len(mws) - 1; i >= 0; i-- {
        h = mws[i](h)
    }
    return h
}

// --- Middleware 1: Gán Request ID ---
func WithRequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Giả lập UUID đơn giản
        requestID := fmt.Sprintf("req-%d", time.Now().UnixNano())
        w.Header().Set("X-Request-ID", requestID)
        fmt.Printf("[RequestID] %s\n", requestID)
        next.ServeHTTP(w, r) // Gọi handler tiếp theo
    })
}

// --- Middleware 2: Đo thời gian xử lý ---
func WithTimer(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now() // Tiền xử lý: ghi lại thời điểm bắt đầu
        next.ServeHTTP(w, r)
        // Hậu xử lý: tính thời gian sau khi handler xong
        fmt.Printf("[Timer] Xử lý xong trong: %v\n", time.Since(start))
    })
}

// --- Handler cuối ---
var helloHandler = http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Hello, Cinema!")
})

func main() {
    // Compose: WithRequestID chạy ngoài cùng → WithTimer → helloHandler
    final := Chain(helloHandler, WithRequestID, WithTimer)

    // Dùng httptest để giả lập request mà không cần server thật
    req := httptest.NewRequest("GET", "/", nil)
    w := httptest.NewRecorder()
    final.ServeHTTP(w, req)

    fmt.Println("Response body:", w.Body.String())
    fmt.Println("X-Request-ID:", w.Header().Get("X-Request-ID"))
}
```

Output nhận được:

```
PS C:\Users\Admin> go run "c:\Users\Admin\turtorial.go"
[RequestID] req-1775229925765073800
[Timer] Xử lý xong trong: 0s
Response body: Hello, Cinema!
```

Nhận xét ngôn ngữ học: Toàn bộ cơ chế middleware được xây dựng từ ba đặc điểm ngôn ngữ cơ bản của Go: higher-order functions (hàm nhận và trả về hàm), closure (bắt biến start, requestID từ scope ngoài), và interface structural typing (http.HandlerFunc thỏa mãn http.Handler tự động).

4.2. Phân tích chuyên sâu ba Middleware cốt lõi

4.2.1. Logger Middleware - Context truyền dữ liệu giữa các lớp

Một thách thức trong thiết kế middleware là làm thế nào để middleware *trước* truyền dữ liệu cho middleware *sau* và cho handler cuối. Go giải quyết bài toán này bằng context.Context - một interface được đặt trong *http.Request.

context.Context là một cơ chế *immutable key-value store* được thiết kế cho việc truyền dữ liệu theo phạm vi request (*request-scoped data*): mỗi lần thêm giá trị, một context *mới* được tạo ra bọc ngoài context cũ, context cũ không bị thay đổi. Đây là cách Go đảm bảo an toàn đồng thời (*concurrency safety*) khi nhiều goroutine cùng đọc context của một request.

Để tránh key collision giữa các package khác nhau, Go spec khuyến nghị dùng unexported custom type làm context key - thay vì dùng string có thể bị trùng:

```go
// Dùng unexported type làm key — tránh xung đột với package khác
type contextKey string
const (
    keyRequestID contextKey = "requestID"
    keyUserID    contextKey = "userID"
)
```

Đặc điểm của Logger Middleware trong hệ thống CinemaBooking là cần ghi lại: method, path, status code, thời gian xử lý, và request ID để trace log. Tuy nhiên, http.ResponseWriter không có phương thức nào để đọc lại status code sau khi đã ghi - đây là một hạn chế của stdlib Go. Giải pháp là **bọc ResponseWriter** trong một struct tùy chỉnh:

```go
// --- Định nghĩa context key theo cách an toàn ---
type contextKey string
const keyRequestID contextKey = "requestID"

// responseWriter là wrapper bổ sung khả năng ghi nhớ status code,
// vì http.ResponseWriter gốc không cho đọc lại status sau khi WriteHeader
type responseWriter struct {
    http.ResponseWriter        // Nhúng (embed) interface gốc
    statusCode         int     // Lưu lại status code đã ghi
    bytesWritten       int     // Số byte đã ghi vào body
}

// Override WriteHeader để bắt status code trước khi ghi xuống
func (rw *responseWriter) WriteHeader(code int) {
    rw.statusCode = code
    rw.ResponseWriter.WriteHeader(code) // Gọi lại phương thức gốc
}

// Override Write để đếm số byte
func (rw *responseWriter) Write(b []byte) (int, error) {
    n, err := rw.ResponseWriter.Write(b)
    rw.bytesWritten += n
    return n, err
}

// newResponseWriter tạo wrapper với status code mặc định 200
func newResponseWriter(w http.ResponseWriter) *responseWriter {
    return &responseWriter{ResponseWriter: w, statusCode: http.StatusOK}
}
```

```go
// --- Logger Middleware ---
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Tiền xử lý: tạo request ID, gắn vào context
        requestID := fmt.Sprintf("%d", time.Now().UnixNano())
        ctx := context.WithValue(r.Context(), keyRequestID, requestID)
        r = r.WithContext(ctx) // r.WithContext trả về *Request mới, không thay đổi r cũ

        // Bọc ResponseWriter để có thể đọc status code sau
        wrapped := newResponseWriter(w)
        start := time.Now()

        // Gọi handler tiếp theo với request đã có context mới
        next.ServeHTTP(wrapped, r)

        // Hậu xử lý: ghi log với đầy đủ thông tin
        duration := time.Since(start)
        log.Printf(
            "[%s] %s %s | Status: %d | Thời gian: %v | Bytes: %d",
            requestID,
            r.Method,
            r.URL.Path,
            wrapped.statusCode,
            duration,
            wrapped.bytesWritten,
        )
    })
}


// GetRequestID lấy request ID từ context — dùng ở bất kỳ handler nào phía sau
func GetRequestID(ctx context.Context) string {
    if id, ok := ctx.Value(keyRequestID).(string); ok {
        return id
    }
    return ""
}
```

Phân tích cơ chế ngôn ngữ:

- Embedding interface (http.ResponseWriter được nhúng vào struct responseWriter): Đây là cơ chế *promotion* của Go - responseWriter tự động kế thừa tất cả phương thức của http.ResponseWriter, ta chỉ cần override những phương thức cần can thiệp.
- r.WithContext(ctx): Trả về một *Request mới với context mới - Go không cho phép mutate *Request trực tiếp vì request được chia sẻ có thể được đọc bởi nhiều goroutine.

- Type assertion ctx.Value(keyRequestID).(string): Context trả về interface{} nên cần type assertion; dạng value, ok := x.(T) là idiom an toàn (comma-ok pattern) của Go tránh panic khi kiểu không khớp.

Bước 3 - Bài toán minh họa nhỏ: ResponseWriter Wrapper

Đề bài: Viết một middleware WithStatusTracker bọc http.ResponseWriter để đếm số lần các nhóm status code được trả về (2xx, 4xx, 5xx). Sau mỗi 10 request, in ra thống kê: "2xx: N, 4xx: N, 5xx: N". Dùng sync.Mutex để bảo vệ bộ đếm vì nhiều goroutine có thể cùng cập nhật.

```go
// 1. StatusCounter – bộ đếm dùng chung, thread-safe
// ────────────────────────────────────────────

type StatusCounter struct {
    mu      sync.Mutex
    counts  map[string]int // key: "2xx" | "4xx" | "5xx" | "other"
    total   int
}

func NewStatusCounter() *StatusCounter {
    return &StatusCounter{
        counts: map[string]int{
            "2xx":   0,
            "4xx":   0,
            "5xx":   0,
            "other": 0,
        },
    }
}
// Record ghi nhận một status code và in thống kê sau mỗi 10 request.
func (sc *StatusCounter) Record(statusCode int) {
    sc.mu.Lock()
    defer sc.mu.Unlock()

    switch {
    case statusCode >= 200 && statusCode < 300:
        sc.counts["2xx"]++
    case statusCode >= 400 && statusCode < 500:
        sc.counts["4xx"]++
    case statusCode >= 500 && statusCode < 600:
        sc.counts["5xx"]++
    default:
        sc.counts["other"]++
    }

    sc.total++

    if sc.total%10 == 0 {
        fmt.Printf(
            "[StatusTracker] Sau %d request → 2xx: %d, 4xx: %d, 5xx: %d\n",
            sc.total,
            sc.counts["2xx"],
            sc.counts["4xx"],
            sc.counts["5xx"],
        )
    }
}
```

```go
// ────────────────────────────────────────────
// 2. statusTrackingWriter – bọc ResponseWriter
// ────────────────────────────────────────────
type statusTrackingWriter struct {
    http.ResponseWriter          // nhúng (embedding) để kế thừa toàn bộ interface
    counter         *StatusCounter
    statusCode      int
    wroteHeader     bool
}
// WriteHeader override để bắt status code trước khi ghi xuống client.
func (w *statusTrackingWriter) WriteHeader(code int) {
    if !w.wroteHeader {
        w.statusCode = code
        w.wroteHeader = true
        w.counter.Record(code)
    }
    w.ResponseWriter.WriteHeader(code)
}
// Write đảm bảo nếu handler gọi Write() mà không gọi WriteHeader() trước,
// ta vẫn ghi nhận 200 OK (hành vi mặc định của net/http).
func (w *statusTrackingWriter) Write(b []byte) (int, error) {
    if !w.wroteHeader {
        w.WriteHeader(http.StatusOK)
    }
    return w.ResponseWriter.Write(b)
}
// ────────────────────────────────────────────
// 3. WithStatusTracker – middleware factory
// ────────────────────────────────────────────

// WithStatusTracker nhận một *StatusCounter dùng chung và trả về middleware.
// Counter được khởi tạo MỘT LẦN bên ngoài, truyền vào để dùng lại qua
// nhiều request (nhiều goroutine đồng thời vẫn an toàn nhờ sync.Mutex).
func WithStatusTracker(counter *StatusCounter, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tracked := &statusTrackingWriter{
            ResponseWriter: w,
            counter:        counter,
            statusCode:     http.StatusOK, // default
        }
        next.ServeHTTP(tracked, r)

        // Nếu handler không gọi Write/WriteHeader nào cả
        if !tracked.wroteHeader {
            tracked.WriteHeader(http.StatusOK)
        }
    })
}
```

```go
// ────────────────────────────────────────────
// 4. Demo handlers
// ────────────────────────────────────────────

func okHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "OK 200")
}

func notFoundHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusNotFound)
    fmt.Fprintln(w, "Not Found 404")
}

func serverErrorHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusInternalServerError)
    fmt.Fprintln(w, "Internal Server Error 500")
}

// ────────────────────────────────────────────
// 5. main – wire up server
// ────────────────────────────────────────────

func main() {
    // Khởi tạo counter MỘT LẦN, dùng chung cho toàn bộ server.
    counter := NewStatusCounter()

    mux := http.NewServeMux()
    mux.HandleFunc("/ok", okHandler)
    mux.HandleFunc("/notfound", notFoundHandler)
    mux.HandleFunc("/error", serverErrorHandler)

    // Bọc toàn bộ mux bằng middleware
    handler := WithStatusTracker(counter, mux)
```

```go
func main() {
    // Khởi tạo counter MỘT LẦN, dùng chung cho toàn bộ server.
    counter := NewStatusCounter()

    mux := http.NewServeMux()
    mux.HandleFunc("/ok", okHandler)
    mux.HandleFunc("/notfound", notFoundHandler)
    mux.HandleFunc("/error", serverErrorHandler)

    // Bọc toàn bộ mux bằng middleware
    handler := WithStatusTracker(counter, mux)

    // — Tự động gửi 30 request để minh họa (chạy sau 100ms để server kịp start) —
    go func() {
        time.Sleep(100 * time.Millisecond)
        endpoints := []string{
            "http://localhost:8080/ok",        // 2xx
            "http://localhost:8080/ok",
            "http://localhost:8080/ok",
            "http://localhost:8080/ok",
            "http://localhost:8080/ok",
            "http://localhost:8080/ok",
            "http://localhost:8080/notfound",  // 4xx
            "http://localhost:8080/notfound",
            "http://localhost:8080/error",     // 5xx
            "http://localhost:8080/error",
        }

        fmt.Println("— Gửi 30 request demo (3 vòng × 10) —")
        for round := 0; round < 3; round++ {
            for _, url := range endpoints {
                resp, err := http.Get(url) //nolint:gosec
                if err != nil {
                    log.Printf("request error: %v", err)
                    continue
                }
                resp.Body.Close()
            }
        }
        fmt.Println("— Xong, nhấn Ctrl+C để dừng server —")
    }()

    fmt.Println("Server chạy tại :8080")
    log.Fatal(http.ListenAndServe(":8080", handler))
}
```

4.2.2. Authentication Middleware - JWT và Context truyền thông tin người dùng

Bước 1 - Lý thuyết: JWT và vấn đề stateless authentication

JWT (JSON Web Token) là chuẩn RFC 7519 cho phép server xác thực người dùng mà *không cần lưu trạng thái phiên* (stateless). Token có dạng Header.Payload.Signature:

- Header: Khai báo thuật toán ký (ví dụ HS256 - HMAC SHA-256).
- Payload: Chứa các *claims* - thông tin về người dùng (userID, role, thời gian hết hạn exp). Lưu ý: Payload chỉ được Base64URL encode, không mã hóa - ai cũng đọc được, do đó tuyệt đối không chứa thông tin nhạy cảm.
- Signature: HMAC(Base64(header) + "." + Base64(payload), secret_key) - đảm bảo token không bị giả mạo.

Trong hệ thống phân tán, mọi instance backend đều xác thực được token nếu có cùng secret key - đây là ưu điểm lớn so với session-based auth yêu cầu shared session store. Nhược điểm cố hữu: JWT không thể thu hồi trước khi hết hạn vì bản chất stateless; giải pháp thực tiễn là duy trì blacklist trong Redis với TTL bằng thời gian token còn lại.

Bước 2 - Code minh họa: JWT Auth Middleware trong CinemaBooking

Đặc điểm ngôn ngữ Go được thể hiện rõ trong middleware này: custom error type, context key pattern, và idiomatic error handling.

```go
// --- Custom Claims: nhúng jwt.RegisteredClaims để kế thừa các trường chuẩn ---
// (exp, iat, iss, ...) — Go dùng embedding thay cho kế thừa class
type Claims struct {
    UserID               int    `json:"user_id"`
    Role                 string `json:"role"`
    jwt.RegisteredClaims        // Nhúng struct chuẩn của thư viện
}

// contextKey để tránh collision khi lưu vào context
type ctxKey string

const claimsKey ctxKey = "claims"

// --- Custom error types: Go không có exception, dùng error values ---
var (
    ErrMissingToken = errors.New("không có token xác thực")
    ErrInvalidToken = errors.New("token không hợp lệ hoặc đã hết hạn")
)

const jwtSecret = "cinema-secret-key" // Trong thực tế đọc từ os.Getenv


// --- Hàm tạo token (dùng trong AuthHandler khi login) ---
func GenerateToken(userID int, role string) (string, error) {
    claims := Claims{
        UserID: userID,
        Role:   role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
            Issuer:    "cinema-booking",
        },
    }
    // jwt.NewWithClaims tạo token chưa ký, SignedString ký và trả chuỗi hoàn chỉnh
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(jwtSecret))
}
```

```go
// --- Auth Middleware ---
func Auth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Bước 1: Lấy token từ header "Authorization: Bearer <token>"
        authHeader := r.Header.Get("Authorization")
        if authHeader == "" || !strings.HasPrefix(authHeader, "Bearer ") {
            http.Error(w, ErrMissingToken.Error(), http.StatusUnauthorized)
            return // Đặc trưng Go: return sớm thay vì else lồng nhau
        }
        tokenStr := strings.TrimPrefix(authHeader, "Bearer ")

        // Bước 2: Parse và xác thực token
        // Hàm callback trả về secret key và kiểm tra thuật toán ký
        // — ngăn tấn công alg:none (kẻ tấn công đặt alg="none" để bỏ qua xác thực
        token, err := jwt.ParseWithClaims(tokenStr, &Claims{},
            func(t *jwt.Token) (interface{}, error) {
                // Kiểm tra thuật toán phải là HMAC — không chấp nhận alg khác
                if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                    return nil, ErrInvalidToken
                }
                return []byte(jwtSecret), nil
            },
        )
        if err != nil || !token.Valid {
            http.Error(w, ErrInvalidToken.Error(), http.StatusUnauthorized)
            return
        }
```

```go
        // Bước 3: Lưu claims vào context để các handler phía sau dùng
        // — không cần parse lại token ở mỗi handler
        claims := token.Claims.(*Claims) // Type assertion an toàn vì đã kiểm
        ctx := context.WithValue(r.Context(), claimsKey, claims)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// --- RequireRole: middleware bổ sung kiểm tra phân quyền ---
// Đây là ví dụ middleware có tham số — trả về một middleware function
func RequireRole(allowedRoles ...string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            claims, ok := r.Context().Value(claimsKey).(*Claims)
            if !ok {
                http.Error(w, "unauthorized", http.StatusUnauthorized)
                return
            }
            for _, role := range allowedRoles {
                if claims.Role == role {
                    next.ServeHTTP(w, r)
                    return
                }
            }
            http.Error(w, "forbidden: không đủ quyền", http.StatusForbidden)
        })
    }
}


// --- Helper: lấy claims từ context trong handler ---
func GetClaims(ctx context.Context) (*Claims, bool) {
    claims, ok := ctx.Value(claimsKey).(*Claims)
    return claims, ok
}
```

Phân tích cơ chế ngôn ngữ:

- Middleware factory pattern (RequireRole(roles...) func(Handler) Handler): Đây là hàm trả về middleware - lợi dụng closure để "bọc" tham số allowedRoles vào bên trong middleware. Go hỗ trợ tự nhiên nhờ first-class functions.
- Variadic parameters (allowedRoles ...string): Go cho phép truyền số lượng argument linh hoạt, giúp RequireRole("admin") và RequireRole("admin", "staff") đều hợp lệ.

- Early return pattern: Thay vì if valid { ... } else { return error }, Go idiom là if !valid { return error } rồi tiếp tục - giữ code phẳng, tránh pyramid of doom.

Cách sử dụng trong router của CinemaBooking:

```go
// Route công khai – không cần auth
mux.Handle("/api/movies", Chain(movieListHandler, Logger))

// Route cần đăng nhập
mux.Handle("/api/bookings", Chain(
    bookingHandler,
    Logger,
    Auth,                // Kiểm tra JWT trước
))

// Route chỉ dành cho admin
mux.Handle("/api/admin/movies", Chain(
    adminMovieHandler,
    Logger,
    Auth,                // Xác thực JWT
    RequireRole("admin"), // Sau đó kiểm tra role
))
```

Bước 3 - Bài toán minh họa nhỏ: Middleware Factory với tham số

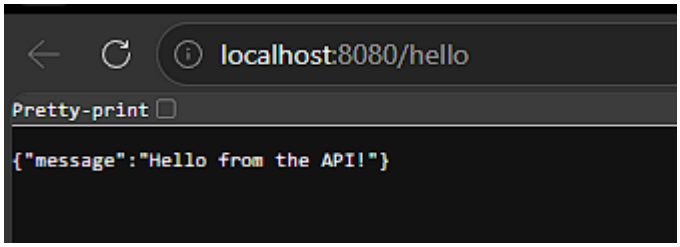
Đề bài: Viết middleware WithAPIVersion(version string) nhận tham số version (ví dụ "v1", "v2") và gắn giá trị đó vào header response X-API-Version. Đồng thời, nếu client gửi header Accept-Version khác với version của server, trả về lỗi 426 Upgrade Required với body JSON {"error": "Please use API version v1"}.

Mục tiêu: Quan sát cách Go dùng closure để tạo middleware có trạng thái cố định (version string), và cách viết middleware factory tổng quát.

```go
// ────────────────────────────────────────────
// 1. Middleware Factory – WithAPIVersion
// ────────────────────────────────────────────

// WithAPIVersion là một middleware factory: nhận tham số version,
// trả về một http.Handler bọc ngoài handler gốc.
//
// Closure giữ biến `version` sống mãi theo vòng đời của middleware –
// đây chính là "trạng thái cố định" mà đề bài nhắc đến.
func WithAPIVersion(version string, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Luôn gắn version của server vào response header
        w.Header().Set("X-API-Version", version)

        // Kiểm tra header client gửi lên (nếu có)
        if clientVer := r.Header.Get("Accept-Version"); clientVer != "" && clientVer != version {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusUpgradeRequired) // 426

            if err := json.NewEncoder(w).Encode(map[string]string{
                "error": fmt.Sprintf("Please use API version %s", version),
            }); err != nil {
                log.Printf("Error encoding response: %v", err)
            }
            return // dừng – không gọi handler tiếp theo
        }

        next.ServeHTTP(w, r)
    })
}
```

```go
// ────────────────────────────────────────────
// 2. Handler demo
// ────────────────────────────────────────────

func helloHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    if err := json.NewEncoder(w).Encode(map[string]string{
        "message": "Hello from the API!",
    }); err != nil {
        log.Printf("Error encoding response: %v", err)
    }
}

// ────────────────────────────────────────────
// 3. main
// ────────────────────────────────────────────

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/hello", helloHandler)

    // Closure "ghi nhớ" chuỗi "v1" - version cố định cho toàn bộ server này
    handler := WithAPIVersion("v1", mux)

    fmt.Println("Server chạy tại :8080")
    fmt.Println()
    fmt.Println("Thử các lệnh curl:")
    fmt.Println(`  1. Không gửi Accept-Version  → 200 OK`)
    fmt.Println(`     curl -i http://localhost:8080/hello`)
    fmt.Println()
    fmt.Println(`  2. Gửi đúng version          → 200 OK`)
    fmt.Println(`     curl -i -H "Accept-Version: v1" http://localhost:8080/hello`)
    fmt.Println()
    fmt.Println(`  3. Gửi sai version           → 426 Upgrade Required`)
    fmt.Println(`     curl -i -H "Accept-Version: v2" http://localhost:8080/hello`)

    log.Fatal(http.ListenAndServe(":8080", handler))
}
```

Khi mở localhost8080 nhận được message



4.2.3. Recovery Middleware - defer, recover() và xử lý Panic

Bước 1 - Lý thuyết: Mô hình xử lý lỗi nghiêm trọng trong Go

Go phân biệt hai cấp độ lỗi:

- error (lỗi có thể xử lý): Giá trị kiểu error được trả về từ hàm, caller phải kiểm tra và xử lý. Đây là cách Go xử lý phần lớn lỗi runtime.
- panic (lỗi nghiêm trọng không lường trước): Xảy ra khi dereferencing nil pointer, index out of range, type assertion thất bại không dùng comma-ok... Khi panic xảy ra, Go stack unwind từ goroutine đang chạy, gọi tất cả defer đã đăng ký, rồi crash chương trình.

recover() là hàm built-in *chỉ có tác dụng khi được gọi bên trong hàm defer*. Nó bắt panic và trả về giá trị được truyền vào panic(value), cho phép chương trình tiếp tục thay vì crash.

So sánh với các ngôn ngữ khác:

	Go	**Java**	**Python**	**C++**
Lỗi thông thường	error return value	Exception (checked/unchecked)	Exception	Return code / std::error_code
Lỗi nghiêm trọng	panic + recover	RuntimeException / Error	BaseException	std::terminate / std::exception
Cleanup	defer	finally	finally	RAII (destructor)
Hiệu suất	Defer có overhead nhỏ	Try/catch cheap khi không throw	Overhead khi raise	Zero-cost khi không throw

Cơ chế defer của Go đáng phân tích kỹ: defer f() đăng ký hàm f vào một **defer stack** - một danh sách LIFO gắn với goroutine hiện tại. Khi hàm chứa defer kết thúc (dù kết

thúc thông thường hay do panic), tất cả defer được gọi theo thứ tự ngược lại (LIFO). Đây là cơ chế Go đảm bảo cleanup resources - tương tự finally nhưng gắn với hàm thay vì block.

```go
// panicResponse định nghĩa format JSON trả về khi có panic
type panicResponse struct {
    Error     string `json:"error"`
    RequestID string `json:"request_id,omitempty"`
}
// GetRequestID extracts the request ID from the context (placeholder implementation)
func GetRequestID(ctx context.Context) string {
    // Implement based on your request ID middleware (e.g., from context.Value)
    if id, ok := ctx.Value("requestID").(string); ok {
        return id
    }
    return "unknown"
}
// Recovery là middleware "lá chắn" ngoài cùng — bắt mọi panic
// trong toàn bộ chuỗi middleware và handler phía trong
func Recovery(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // defer + recover: pattern chuẩn để bắt panic trong Go
        // defer này sẽ chạy ngay cả khi next.ServeHTTP gây ra panic
        defer func() {
            if rec := recover(); rec != nil {
                // recover() trả về nil nếu không có panic,
                // trả về giá trị của panic(value) nếu có panic

                // Lấy stack trace đầy đủ để debug
                stackTrace := debug.Stack()

                // Log lỗi chi tiết ở server (không expose ra client)
                requestID := GetRequestID(r.Context())
                log.Printf(
                    "[PANIC RECOVERED] RequestID: %s\nLỗi: %v\nStack:\n%s",
                    requestID, rec, stackTrace,
                )

                // Ghi header trước khi WriteHeader — sau WriteHeader không thể ghi header
                w.Header().Set("Content-Type", "application/json")
                w.WriteHeader(http.StatusInternalServerError)

                // Trả về message thân thiện cho client (không lộ chi tiết lỗi)
                resp := panicResponse{
                    Error:     "Đã xảy ra lỗi nội bộ. Vui lòng thử lại sau.",
                    RequestID: requestID,
                }
                json.NewEncoder(w).Encode(resp)
            }
        }()

        // Gọi handler tiếp theo — nếu handler gây panic,
        // defer bên trên sẽ bắt và xử lý
        next.ServeHTTP(w, r)
    })
}
```

Phân tích chi tiết cơ chế defer + recover:

```go
// Minh họa thứ tự thực thi defer khi có panic:
func example() {
    defer fmt.Println("defer 1 — chạy sau cùng (LIFO)")
    defer fmt.Println("defer 2 — chạy thứ hai")
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("defer 3 (recover) — bắt panic:", r)
        }
    }()

    panic("có lỗi nghiêm trọng") // Gây panic

    fmt.Println("Dòng này không bao giờ chạy")
}
// Output:
// defer 3 (recover) — bắt panic: có lỗi nghiêm trọng
// defer 2 — chạy thứ hai
// defer 1 — chạy sau cùng (LIFO)
```

Output nhận được:

```
PS C:\Users\Admin> go run "c:\Users\Admin\main.go"
defer 3 (recover) — bắt panic: có lỗi nghiêm trọng
defer 2 — chạy thứ hai
defer 1 — chạy sau cùng (LIFO)
PS C:\Users\Admin>
```

Điều kiện bắt buộc để recover() hoạt động:

1. recover() phải được gọi *trực tiếp* bên trong hàm được defer đăng ký - không phải trong hàm được gọi bởi defer.
2. recover() chỉ có hiệu lực với panic *trong cùng goroutine* - panic từ goroutine khác (ví dụ goroutine con được khởi tạo bởi go func(){}()) sẽ không bị bắt.

Điểm thứ hai đặc biệt quan trọng trong Go khi dùng goroutine: nếu một goroutine con panic mà không có recovery riêng, toàn bộ chương trình sẽ crash dù goroutine cha có recovery.

Bước 3 - Bài toán minh họa nhỏ: Goroutine panic isolation

Đề bài: Viết một handler RiskyHandler thực hiện hai việc: (1) xử lý request bình thường trong goroutine hiện tại, (2) khởi chạy một goroutine phụ bằng go func() để thực hiện tác vụ nền (giả lập bằng một hàm có thể panic). Đảm bảo panic từ goroutine phụ không làm crash server - goroutine phụ phải có defer + recover riêng. In ra log khi bắt được panic từ goroutine phụ.

Mục tiêu: Hiểu rằng Recovery Middleware ở lớp ngoài *không thể* bắt panic từ goroutine con - mỗi goroutine phải tự bảo vệ mình.

```go
// ── Tác vụ nền – có thể panic ──────────────────────────────────────

func riskyBackground(id int) {
    // Mỗi goroutine phụ PHẢI tự recover – middleware ngoài không với tới được
    defer func() {
        if r := recover(); r != nil {
            log.Printf("[background goroutine #%d] bắt được panic: %v", id, r)
        }
    }()

    // Giả lập: request lẻ thì panic, request chẵn thì OK
    if id%2 != 0 {
        panic(fmt.Sprintf("something exploded in task #%d", id))
    }
    time.Sleep(50 * time.Millisecond)
    log.Printf("[background goroutine #%d] hoàn thành bình thường", id)
}


// ── Recovery Middleware (minh họa giới hạn của nó) ──────────────────

func WithRecovery(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                log.Printf("[middleware] bắt được panic từ goroutine CHÍNH: %v", rec)
                http.Error(w, "Internal Server Error", http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

```go
func RiskyHandler(w http.ResponseWriter, r *http.Request) {
    requestID++
    id := requestID

    // (2) Goroutine phụ – có defer+recover riêng, panic KHÔNG lan ra ngoài
    go riskyBackground(id)

    // (1) Xử lý request bình thường – không bị ảnh hưởng bởi goroutine phụ
    fmt.Fprintf(w, "request #%d đã xử lý xong (goroutine phụ chạy nền)\n", id)
}

// ── main ──────────────────────────────────────────────────────────────

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/risky", RiskyHandler)

    handler := WithRecovery(mux)

    fmt.Println("Server :8080 — thử: curl http://localhost:8080/risky (nhiều lần)")
    log.Fatal(http.ListenAndServe(":8080", handler))
}
```

Output nhận được:

```
PS C:\Users\Admin> go run "c:\Users\Admin\main.go"
Server :8080 — thử: curl http://localhost:8080/risky (nhiều lần)
2026/04/03 23:23:37 [background goroutine #1] bắt được panic: something exploded in task #1
2026/04/03 23:23:42 [background goroutine #2] hoàn thành bình thường
2026/04/03 23:24:47 [background goroutine #3] bắt được panic: something exploded in task #3
```

4.3. Tổ chức Middleware trong kiến trúc Clean Architecture của CinemaBooking

4.3.1. Cấu trúc thư mục và phân tầng middleware

Trong dự án CinemaBooking theo Clean Architecture, middleware được tổ chức tách biệt khỏi handler và usecase:

```
internal/
├── middleware/
│   ├── logger.go       // Logger middleware + ResponseWriter wrapper
│   ├── auth.go         // JWT Auth + RequireRole factory
│   ├── recovery.go     // Panic recovery
│   └── chain.go        // Hàm Chain() tổng quát
├── handler/
│   ├── booking_handler.go
│   └── auth_handler.go
└── ...
```

4.3.2. Compose middleware với Gin Framework

Trong thực tế, dự án CinemaBooking dùng Gin framework thay vì stdlib net/http. Gin cung cấp kiểu gin.HandlerFunc (tương đương http.HandlerFunc) và phương thức Use() để đăng ký middleware global hoặc theo group route. Đặc điểm ngôn ngữ Go - interface và function type - vẫn là nền tảng, Gin chỉ bổ sung tiện ích:

```go
func main() {
    r := gin.New() // gin.New() thay vì gin.Default() để tự kiểm soát middlewa

    // === Middleware global — áp dụng cho MỌI route ===
    // Recovery ngoài cùng — bắt panic trước tất cả
    r.Use(gin.Recovery()) // Gin có sẵn, hoặc dùng middleware.Recovery của ta

    // Logger ghi log tất cả request
    r.Use(middleware.GinLogger())

    // === Route công khai — không cần auth ===
    public := r.Group("/api")
    {
        public.GET("/movies", movieHandler.List)
        public.POST("/auth/login", authHandler.Login)
        public.POST("/auth/register", authHandler.Register)
    }

    // === Route cần đăng nhập ===
    protected := r.Group("/api")
    protected.Use(middleware.GinAuth()) // JWT middleware cho group này
    {
        protected.POST("/bookings", bookingHandler.Create)
        protected.GET("/bookings/me", bookingHandler.MyBookings)
        protected.POST("/payments", paymentHandler.Create)
    }

    // === Route chỉ dành cho admin ===
    admin := r.Group("/api/admin")
    admin.Use(middleware.GinAuth())
    admin.Use(middleware.GinRequireRole("admin"))
    {
        admin.POST("/movies", adminMovieHandler.Create)
        admin.PUT("/movies/:id", adminMovieHandler.Update)
        admin.DELETE("/movies/:id", adminMovieHandler.Delete)
    }

    r.Run(":8080")
}
```

Phân tích đặc điểm ngôn ngữ Go trong đoạn code trên:

- Group route trong Gin là một cách compose middleware không phải cho từng handler riêng lẻ mà cho *tập hợp route* - giảm lặp code, đảm bảo nhất quán về chính sách bảo mật.

- r.Use() nhận gin.HandlerFunc - tương tự http.HandlerFunc nhưng nhận thêm *gin.Context là một struct chứa cả request, response, và các tiện ích của Gin.
- Thứ tự Use() xác định thứ tự thực thi - Recovery phải đứng trước Logger và Auth để bắt được panic từ bất kỳ lớp nào.

4.4. Rate Limiting Middleware - Kết hợp Goroutine và Channel

4.4.1. Lý thuyết: Bài toán giới hạn tần suất trong hệ thống đặt vé

Rate Limiting là bài toán đặc biệt phù hợp để minh họa đặc điểm concurrency của Go: khi nhiều goroutine (mỗi request là một goroutine trong Gin) cùng cố đặt vé hoặc đăng nhập, cần một cơ chế giới hạn theo từng user mà không dùng global lock làm tắc nghẽn toàn bộ hệ thống.

Go cung cấp gói golang.org/x/time/rate với thuật toán Token Bucket: mỗi "bucket" có dung lượng burst token, được nạp lại r token/giây. Mỗi request tiêu thụ một token; nếu bucket cạn, request bị từ chối. Điều này phù hợp với bài toán đặt vé: cho phép burst ngắn (nhiều request liên tiếp) nhưng giới hạn tốc độ trung bình.

4.4.2. Code minh họa: Per-user Rate Limiter dùng sync.Map

```go
// userLimiter lưu limiter và thời điểm truy cập gần nhất
type userLimiter struct {
    limiter  *rate.Limiter
    lastSeen time.Time
}

// rateLimiterStore quản lý limiter theo từng IP hoặc userID
type rateLimiterStore struct {
    mu       sync.Mutex              // Bảo vệ map khi nhiều goroutine cùng truy cập
    limiters map[string]*userLimiter
    r        rate.Limit              // Token nạp lại: token/giây
    b        int                     // Burst: số token tối đa trong bucket
}

func newRateLimiterStore(r rate.Limit, b int) *rateLimiterStore {
    store := &rateLimiterStore{
        limiters: make(map[string]*userLimiter),
        r:        r,
        b:        b,
    }
    // Goroutine dọn dẹp limiter không dùng trong 3 phút — chạy mãi ở nền
    go store.cleanupLoop()
    return store
}
```

```go
// getLimiter trả về limiter cho một key, tạo mới nếu chưa có
func (s *rateLimiterStore) getLimiter(key string) *rate.Limiter {
    s.mu.Lock()
    defer s.mu.Unlock() // defer đảm bảo unlock dù hàm kết thúc bình thường hay panic

    ul, exists := s.limiters[key]
    if !exists {
        ul = &userLimiter{limiter: rate.NewLimiter(s.r, s.b)}
        s.limiters[key] = ul
    }
    ul.lastSeen = time.Now()
    return ul.limiter
}

// cleanupLoop chạy trong goroutine riêng, định kỳ xóa limiter cũ
func (s *rateLimiterStore) cleanupLoop() {
    for {
        time.Sleep(time.Minute)
        s.mu.Lock()
        for key, ul := range s.limiters {
            if time.Since(ul.lastSeen) > 3*time.Minute {
                delete(s.limiters, key)
            }
        }
        s.mu.Unlock()
    }
}
// RateLimitByIP tạo middleware giới hạn theo IP — dùng closure để giữ store
func RateLimitByIP(requestsPerSecond float64, burst int) func(http.Handler) http.Handler {
    store := newRateLimiterStore(rate.Limit(requestsPerSecond), burst)

    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ip := r.RemoteAddr
            limiter := store.getLimiter(ip)

            // Allow() kiểm tra và tiêu thụ 1 token — thread-safe trong rate.Limiter
            if !limiter.Allow() {
                w.Header().Set("Retry-After", "1")
                http.Error(w,
                    `{"error":"Quá nhiều request. Vui lòng thử lại sau."}`,
                    http.StatusTooManyRequests,
                )
                return
            }
            next.ServeHTTP(w, r)
```

Phân tích đặc điểm ngôn ngữ:

- defer s.mu.Unlock(): Đây là pattern chuẩn trong Go để unlock mutex đảm bảo unlock xảy ra ngay cả khi hàm return sớm hoặc panic. Quan trọng hơn, defer trên lock phải được viết *ngay sau* Lock() để tránh bỏ sót.

- go store.cleanupLoop(): Goroutine nền chạy vô tận - đây là idiom phổ biến trong Go cho background tasks. Khác với Thread Java/C++, goroutine này chỉ tốn ~2KB stack ban đầu.
- Closure capture store: Biến store được capture bởi closure của middleware đảm bảo tất cả request đến cùng middleware instance dùng chung một store.

CHƯƠNG 5: CÁC KỸ THUẬT NÂNG CAO VÀ TỐI ƯU HÓA

5.1 Xử Lý Đồng Thời - Bài Toán Đặt Ghế Trùng Nhau (Double Booking)

5.1.1 Bản Chất Của Vấn Đề

Khi nhiều người dùng cùng thao tác trên một dữ liệu tại cùng một thời điểm, hệ thống dễ rơi vào tình trạng race condition - tức là kết quả cuối cùng phụ thuộc vào thứ tự thực thi ngẫu nhiên của các luồng xử lý, chứ không phải theo logic nghiệp vụ mà ta mong muốn.

Trong bài toán đặt ghế xem phim, tình huống điển hình diễn ra như sau: Goroutine A (User 1) và Goroutine B (User 2) đều đọc trạng thái ghế G5 và thấy nó đang còn trống. Cả hai đều vượt qua bước kiểm tra và tiến hành ghi "đã đặt" vào cơ sở dữ liệu. Kết quả là ghế G5 bị bán cho cả hai người - đây chính là Double Booking.

Điểm mấu chốt cần hiểu là: kiểm tra ở tầng ứng dụng (application layer) là không đủ. Dù code Go có kiểm tra if seat.Status == "available" trước khi ghi, nhưng nếu hai goroutine cùng đọc dữ liệu trước khi một trong hai ghi xuống, điều kiện kiểm tra đó trở nên vô nghĩa. Vấn đề này càng trầm trọng hơn khi hệ thống chạy nhiều instance song song (horizontal scaling), vì lúc đó nhiều process hoàn toàn độc lập cùng truy cập một cơ sở dữ liệu.

Giải pháp đúng đắn là đẩy cơ chế kiểm soát xuống tầng cơ sở dữ liệu, nơi PostgreSQL đảm bảo tính ACID cho mọi transaction:

Tính Atomicity (nguyên tử) đảm bảo toàn bộ transaction hoặc thành công hoàn toàn, hoặc được hoàn tác hoàn toàn - không tồn tại trạng thái nửa vời. Tính Consistency (nhất quán) đảm bảo dữ liệu luôn hợp lệ theo các ràng buộc đã định trước và sau transaction. Tính Isolation (cô lập) đảm bảo các transaction đang chạy song song không nhìn thấy dữ liệu trung gian của nhau. Tính Durability (bền vững) đảm bảo dữ liệu đã được commit sẽ tồn tại vĩnh viễn kể cả khi hệ thống gặp sự cố.

5.1.2 Cơ Chế SELECT ... FOR UPDATE Hoạt Động Như Thế Nào ?

Câu lệnh SELECT ... FOR UPDATE là một dạng **pessimistic locking** (khóa bi quan) - tức là ta giả định rằng xung đột *sẽ xảy ra*, nên chủ động khóa tài nguyên trước khi xử lý. Khi một transaction thực thi câu lệnh này, PostgreSQL đặt một row-level lock lên hàng đó. Mọi transaction khác muốn đọc hàng đó với FOR UPDATE sẽ phải *chờ* cho đến khi transaction đầu tiên kết thúc (commit hoặc rollback). Nhờ vậy, chỉ một transaction tại một thời điểm có thể kiểm tra và sửa trạng thái của một ghế.

5.1.3 Cách Triển Khai Trong Go

```go
func BookSeat(db *sql.DB, seatID, userID int) error {
    tx, err := db.Begin()
    if err != nil {
        return fmt.Errorf("khởi tạo transaction thất bại: %w", err)
    }
    defer tx.Rollback()

    var status string
    err = tx.QueryRow(
        `SELECT status FROM seats WHERE id = $1 FOR UPDATE`,
        seatID,
    ).Scan(&status)
    if err != nil {
        return fmt.Errorf("không tìm thấy ghế: %w", err)
    }

    if status != "available" {
        return errors.New("ghế này đã có người đặt")
    }

    _, err = tx.Exec(
        `UPDATE seats SET status = 'booked' WHERE id = $1`, seatID,
    )
    if err != nil {
        return fmt.Errorf("cập nhật trạng thái ghế thất bại: %w", err)
    }

    _, err = tx.Exec(
        `INSERT INTO bookings (seat_id, user_id, created_at) VALUES ($1, $2, NOW())`,
        seatID, userID,
    )
    if err != nil {
        return fmt.Errorf("tạo booking thất bại: %w", err)
    }

    return tx.Commit()
}
```

defer tx.Rollback() là dòng quan trọng nhất trong hàm này. Khi đặt ngay sau db.Begin(), nó đảm bảo rằng nếu hàm trả về lỗi ở bất kỳ bước nào - dù là lỗi mạng, lỗi logic hay panic - PostgreSQL sẽ tự động hoàn tác toàn bộ thay đổi. Nếu tx.Commit() thành công thì lệnh Rollback tiếp theo sẽ là no-op, tức là không làm gì cả. Đây là pattern phòng thủ chuẩn khi làm việc với transaction trong Go.

Câu lệnh SELECT ... FOR UPDATE trên hàng ghế đang được truy vấn sẽ đặt một row-level lock ngay tại thời điểm đọc. Điều này có nghĩa là bất kỳ transaction nào khác cũng muốn khóa cùng hàng đó sẽ bị chặn lại và phải đợi - đây chính là điểm then chốt loại bỏ race condition. Việc kiểm tra status != "available" diễn ra **sau khi đã giữ được lock**, nên ta hoàn toàn chắc chắn không có transaction nào khác đang thay đổi hàng đó đồng thời.

Trong môi trường thực tế, nên bổ sung thêm timeout để tránh trường hợp một transaction giữ lock quá lâu:

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

err = tx.QueryRowContext(ctx,
    `SELECT status FROM seats WHERE id = $1 FOR UPDATE NOWAIT`,
    seatID,
).Scan(&status)
```

Từ khóa NOWAIT thay đổi hành vi mặc định: thay vì xếp hàng chờ cho đến khi lock được giải phóng, PostgreSQL sẽ trả về lỗi ngay lập tức nếu hàng đang bị khóa bởi transaction khác. Điều này cho phép ứng dụng phản hồi người dùng nhanh hơn với thông báo "ghế đang được người khác xử lý, vui lòng thử lại" thay vì để request treo trong nhiều giây.

5.1.4 Những Điều Cần Lưu Ý

Luôn khóa các hàng theo đúng một thứ tự nhất định trong mọi transaction để tránh deadlock vòng tròn - ví dụ nếu transaction A khóa ghế 1 rồi ghế 2, thì transaction B cũng phải làm theo thứ tự đó, không được khóa ghế 2 trước. Ngoài ra, cần cân nhắc giữa hai chiến lược: Pessimistic Locking với FOR UPDATE phù hợp khi tần suất xung đột cao như bán vé; còn Optimistic Locking dùng cột version phù hợp hơn khi xung đột hiếm gặp, vì nó cho phép đọc tự do và chỉ kiểm tra khi ghi, giúp tăng throughput đáng kể.

5.2. Connection pool Postgres (tối ưu kết nối DB)

5.2.1 Tại Sao Không Nên Mở Kết Nối Mới Cho Mỗi Request

Mỗi lần ứng dụng Go tạo một kết nối TCP mới đến PostgreSQL, tiến trình đó bao gồm bắt tay TCP, xác thực tài khoản, và khởi tạo phiên làm việc - tổng cộng có thể mất từ 10 đến 50ms. Với một API cần phản hồi trong vòng 100ms, chi phí đó chiếm tỉ lệ rất lớn và hoàn toàn không cần thiết.

Connection Pool giải quyết bài toán này theo nguyên tắc tái sử dụng tài nguyên: thay vì tạo và đóng kết nối theo từng request, pool duy trì sẵn một nhóm kết nối đã được thiết lập. Khi có request đến, ứng dụng mượn một kết nối từ pool, dùng xong thì trả lại - không tạo mới, không đóng thật sự.

Một điểm quan trọng mà nhiều lập trình viên mới hay nhầm: trong Go, *sql.DB không phải là một kết nối đơn lẻ mà chính là connection pool. Một instance *sql.DB duy nhất nên được dùng cho toàn bộ vòng đời ứng dụng và chia sẻ giữa tất cả các goroutine, vì nó đã thread-safe.

5.2.2 Cách Cấu Hình Pool Đúng Cách

Nếu không cấu hình gì, MaxOpenConns mặc định là 0, nghĩa là không giới hạn. Khi traffic tăng đột biến, ứng dụng có thể mở hàng trăm kết nối, vượt quá giới hạn mặc định 100 của PostgreSQL và gây sập toàn bộ hệ thống.

```go
func NewDB(dsn string) (*sql.DB, error) {
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        return nil, err
    }

    db.SetMaxOpenConns(25)
    db.SetMaxIdleConns(10)
    db.SetConnMaxLifetime(30 * time.Minute)
    db.SetConnMaxIdleTime(5 * time.Minute)

    if err = db.Ping(); err != nil {
        return nil, fmt.Errorf("không kết nối được đến database: %w", err)
    }
    return db, nil
}
```

SetMaxOpenConns(25) giới hạn số kết nối đồng thời tối đa đến PostgreSQL - con số 25 chỉ mang tính minh hoạ, trong thực tế nên đặt bằng khoảng 70–80% giá trị max_connections được cấu hình trong Postgres. SetMaxIdleConns(10) là số kết nối được giữ sẵn trong pool khi không có request, giúp tái sử dụng ngay mà không cần thiết lập lại. SetConnMaxLifetime(30 * time.Minute) buộc mỗi kết nối phải được tái tạo sau 30 phút - điều này quan trọng để tránh dùng kết nối đã bị server đóng ngầm sau một khoảng thời gian nhàn rỗi dài. SetConnMaxIdleTime(5 * time.Minute) đóng các kết nối nhàn rỗi sau 5 phút để tiết kiệm tài nguyên khi traffic thấp.

Trong môi trường production, nên theo dõi tình trạng pool theo định kỳ để phát hiện sớm vấn đề nghẽn cổ chai:

```go
func monitorDBPool(db *sql.DB) {
    go func() {
        for range time.NewTicker(30 * time.Second).C {
            s := db.Stats()
            log.Printf(
                "Pool | đang dùng: %d | nhàn rỗi: %d | tổng mở: %d | số lần phải chờ: %d",
                s.InUse, s.Idle, s.OpenConnections, s.WaitCount,
            )
        }
    }()
}
```

Chỉ số WaitCount là quan trọng nhất cần theo dõi - nó đếm số lần một goroutine phải chờ vì pool đã hết kết nối khả dụng. Nếu con số này tăng liên tục, đó là dấu hiệu rõ ràng rằng MaxOpenConns đang quá thấp so với nhu cầu thực tế, hoặc có những query đang chạy quá lâu và chiếm giữ kết nối.

5.2.3 Những Điều Cần Lưu Ý

MaxIdleConns không được lớn hơn MaxOpenConns - nếu đặt ngược lại, Go sẽ tự điều chỉnh xuống nhưng cấu hình sẽ gây nhầm lẫn. Luôn truyền context vào các câu query (QueryContext, ExecContext) thay vì dùng phiên bản không có context, để kết nối được trả về pool ngay khi request bị hủy. Với hệ thống lớn chạy nhiều pods, nên cân nhắc đặt thêm **PgBouncer** ở tầng infrastructure để gom kết nối từ nhiều instance, giảm áp lực trực tiếp lên PostgreSQL.

5.3 Tối Ưu API Response - Phân Trang Danh Sách Phim

5.3.1 Vấn Đề Khi Trả Về Toàn Bộ Dữ Liệu

Hãy thử hình dung API GET /movies trả về tất cả 50.000 bộ phim trong database cùng lúc. Server phải tải toàn bộ dữ liệu vào bộ nhớ, tuần tự hóa thành JSON, rồi truyền hàng chục megabyte qua mạng - trong khi người dùng chỉ muốn xem 20 phim đầu tiên. Đây là sự lãng phí tài nguyên ở mọi tầng: CPU, RAM, băng thông mạng, và thời gian render ở phía client.

Phân trang giải quyết bằng cách chỉ trả về một lát cắt nhỏ của dữ liệu theo từng yêu cầu. Có hai kỹ thuật phổ biến với đặc điểm khác nhau rõ rệt.

5.3.2 Offset Pagination - Đơn Giản, Phù Hợp Với Giao Diện Số Trang

```go
func GetMovies(db *sql.DB, p PaginationParams) (*PagedResult[Movie], error) {
    if p.Limit <= 0 || p.Limit > 100 {
        p.Limit = 20
    }
    if p.Page < 1 {
        p.Page = 1
    }
    offset := (p.Page - 1) * p.Limit

    var total int64
    db.QueryRow(`SELECT COUNT(*) FROM movies WHERE status = 'active'`).Scan(&total)

    rows, err := db.Query(
        `SELECT id, title, genre, release_date
         FROM movies WHERE status = 'active'
         ORDER BY release_date DESC
         LIMIT $1 OFFSET $2`,
        p.Limit, offset,
    )
    // ... đọc rows ...

    totalPages := int(math.Ceil(float64(total) / float64(p.Limit)))
    return &PagedResult[Movie]{
        Data: movies, TotalItems: total,
        TotalPages: totalPages, Page: p.Page,
        HasNext: p.Page < totalPages,
    }, nil
}
```

PagedResult được khai báo dưới dạng generic [T any] để có thể tái sử dụng cho bất kỳ kiểu dữ liệu nào - danh sách phim, danh sách suất chiếu, hay danh sách booking đều dùng chung một struct này. Việc ràng buộc Limit trong khoảng 1–100 là bắt buộc vì không thể tin tưởng giá trị do client gửi lên - một request với limit=100000 có thể làm sập server. Hai câu query được thực thi riêng biệt: một để đếm tổng số bản ghi phục vụ tính TotalPages, một để lấy dữ liệu trang hiện tại. Đây là cách tiếp cận chuẩn, dù tốn thêm một lần query nhưng đổi lại client nhận được đầy đủ thông tin để render giao diện phân trang.

Nhược điểm của kỹ thuật này là khi OFFSET lớn, PostgreSQL vẫn phải quét qua toàn bộ hàng trước rồi mới bỏ đi - ví dụ OFFSET 10000 đồng nghĩa với việc đọc và loại bỏ 10.000 hàng mà không dùng đến. Với dataset dưới vài trăm nghìn bản ghi và người dùng hiếm khi đi quá trang 10, đây không phải vấn đề đáng lo ngại.

5.3.3 Cursor Pagination - Hiệu Suất Ổn Định Cho Dataset Lớn

```go
func GetMoviesCursor(db *sql.DB, cursor int64, limit int) ([]Movie, int64, error) {
    rows, err := db.Query(
        `SELECT id, title FROM movies
         WHERE id < $1
         ORDER BY id DESC
         LIMIT $2`,
        cursor, limit,
    )
```

Thay vì nói "bỏ qua 10.000 hàng đầu tiên", cursor pagination nói "cho tôi các hàng có id nhỏ hơn giá trị này", PostgreSQL dùng index trên cột id để nhảy thẳng đến vị trí cần thiết mà không cần quét gì thêm. Hiệu suất nhờ đó ổn định dù đang ở trang 1 hay trang 1.000. Đổi lại, kỹ thuật này không cho biết tổng số trang và không thể nhảy thẳng đến trang bất kỳ, phù hợp với infinite scroll hoặc feed dữ liệu thay đổi liên tục, không phù hợp với giao diện có số trang cụ thể.

5.3.4 Những Điều Cần Lưu Ý

Thiếu ORDER BY khi dùng LIMIT/OFFSET là lỗi phổ biến nhất - không có sắp xếp thì thứ tự kết quả không xác định và có thể trả về các phần tử trùng lặp hoặc bỏ sót giữa các trang. Cần đánh index trên cột dùng để sắp xếp như release_date hay id để tránh full table scan. Ngoài ra, với những endpoint trả về danh sách ít thay đổi như danh sách thể loại phim, nên kết hợp thêm HTTP caching (Cache-Control, ETag) để giảm số lượng query xuống database.

5.4 Bảo Mật Với JWT

5.4.1 JWT Hoạt Động Như Thế Nào

JWT (JSON Web Token) là một chuẩn mở để truyền thông tin xác thực dưới dạng token tự chứa - server không cần lưu trạng thái phiên làm việc, bất kỳ instance nào trong cluster đều có thể xác thực token mà không cần truy vấn một nơi lưu trữ tập trung.

Token có dạng Header.Payload.Signature. Phần Header khai báo thuật toán ký. Phần Payload chứa các thông tin về người dùng như userID, role, thời gian hết hạn. Phần Signature là chữ ký số tạo từ secret key, đây là thứ đảm bảo token không bị giả mạo hay sửa đổi. Một điểm quan trọng cần nắm rõ: JWT không mã hóa dữ liệu, mà chỉ ký số để xác minh tính toàn vẹn. Payload chỉ là Base64URL encode, ai cũng có thể giải mã và đọc được, vì vậy tuyệt đối không đặt thông tin nhạy cảm như mật khẩu hay số thẻ ngân hàng trong đó.

Luồng xác thực thực tế diễn ra như sau: người dùng đăng nhập → server tạo và ký JWT → client lưu token → mỗi request sau client gửi token trong header Authorization: Bearer <token> → server xác minh chữ ký và đọc thông tin người dùng từ payload, không cần truy vấn database lần nào.

5.4.2 Triển Khai Access Token và Refresh Token

Thực hành tốt là dùng hai loại token với vòng đời khác nhau. Access Token tồn tại ngắn (15 phút) dùng để gọi API - nếu bị lộ thì tác hại giới hạn trong thời gian ngắn. Refresh Token tồn tại lâu hơn (7 ngày) chỉ dùng để xin cấp lại access token mới khi token cũ hết hạn.

```go
type Claims struct {
    UserID int    `json:"user_id"`
    Role   string `json:"role"`
    jwt.RegisteredClaims
}

var jwtSecret = []byte(os.Getenv("JWT_SECRET"))

func GenerateTokenPair(userID int, role string) (accessToken, refreshToken string, err error) {
    accessClaims := Claims{
        UserID: userID, Role: role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(15 * time.Minute)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
            Issuer:    "cinema-service",
        },
    }
    accessToken, err = jwt.NewWithClaims(jwt.SigningMethodHS256, accessClaims).
        SignedString(jwtSecret)

    refreshClaims := jwt.RegisteredClaims{
        Subject:   strconv.Itoa(userID),
        ExpiresAt: jwt.NewNumericDate(time.Now().Add(7 * 24 * time.Hour)),
    }
    refreshToken, err = jwt.NewWithClaims(jwt.SigningMethodHS256, refreshClaims).
        SignedString(jwtSecret)
    return
}
```

jwt.RegisteredClaims được nhúng vào struct Claims để kế thừa sẵn các trường chuẩn như ExpiresAt, IssuedAt, Issuer - tránh phải tự khai báo lại. jwtSecret được đọc từ biến môi trường thay vì hardcode trong source code; đây là nguyên tắc bắt buộc vì nếu secret bị lộ qua repository, kẻ tấn công có thể tự ký bất kỳ token nào với quyền hạn tùy ý. Refresh Token chủ ý không chứa Role hay thông tin nhạy cảm - nó chỉ cần đủ để server biết cấp token mới cho ai.

Middleware xác thực cho Gin:

```go
func JWTMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if !strings.HasPrefix(authHeader, "Bearer ") {
            c.AbortWithStatusJSON(401, gin.H{"error": "thiếu token xác thực"})
            return
        }

        claims := &Claims{}
        token, err := jwt.ParseWithClaims(
            strings.TrimPrefix(authHeader, "Bearer "),
            claims,
            func(t *jwt.Token) (interface{}, error) {
                if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                    return nil, fmt.Errorf("thuật toán ký không hợp lệ: %v", t.Header["alg"])
                }
                return jwtSecret, nil
            },
        )
        if err != nil || !token.Valid {
            c.AbortWithStatusJSON(401, gin.H{"error": "token không hợp lệ hoặc đã hết hạn"})
            return
        }
        c.Set("userID", claims.UserID)
        c.Set("role", claims.Role)
        c.Next()
    }
}
```

Hàm callback bên trong jwt.ParseWithClaims có vai trò kép: vừa trả về secret key để thư viện xác minh chữ ký, vừa kiểm tra rằng token đang dùng đúng thuật toán ký mà ta kỳ vọng. Bước kiểm tra t.Method.(*jwt.SigningMethodHMAC) tuy nhỏ nhưng quan trọng , nó ngăn chặn tấn công alg:none, trong đó kẻ tấn công tự tạo token với trường alg là "none" để bỏ qua toàn bộ bước xác minh chữ ký. Sau khi xác thực thành công, userID và role được gắn vào context để các handler phía sau có thể đọc mà không cần parse token lại lần nữa.

5.4.3 Những Điều Cần Lưu Ý

JWT về bản chất không thể thu hồi trước khi hết hạn, đây là nhược điểm cố hữu của kiến trúc stateless. Để xử lý trường hợp người dùng đổi mật khẩu hoặc đăng xuất khẩn cấp, cần duy trì một danh sách token bị vô hiệu hóa trong Redis với TTL bằng thời gian còn lại của token. Luôn lưu Refresh Token trong httpOnly cookie thay vì localStorage để JavaScript không thể đọc được, qua đó ngăn chặn tấn công XSS đánh cắp token.

5.5 Idempotency Cho Booking & Payment

5.5.1 Tại Sao Retry Là Vấn Đề Nghiêm Trọng

Trong hệ thống phân tán, mạng không bao giờ hoàn toàn đáng tin cậy - khoảng 2–5% request có thể bị timeout hoặc mất gói. Khi đó client hoặc API gateway sẽ tự động gửi lại request. Nếu API không được thiết kế để xử lý điều này, người dùng có thể bị trừ tiền hai lần hoặc đặt hai vé cho cùng một giao dịch.

Tình huống kinh điển: client gửi request thanh toán → server xử lý thành công và trừ tiền → mất kết nối trước khi response đến được client → client không nhận được phản hồi → client gửi lại request → server xử lý lại lần nữa → tiền bị trừ hai lần. Lỗi này không xuất phát từ code sai mà từ bản chất không đáng tin cậy của mạng.

Idempotency là tính chất của một thao tác mà dù thực hiện bao nhiêu lần với cùng đầu vào, kết quả vẫn y như thực hiện đúng một lần. Để đạt được điều này, client tạo một Idempotency Key là một UUID duy nhất và gửi kèm trong header của mỗi request. Server lưu kết quả theo key đó - nếu cùng key được gửi lại, server trả về kết quả đã lưu thay vì xử lý lại.

5.5.2 Thiết Kế Và Triển Khai

Trước tiên cần một bảng để lưu trữ trạng thái các request:

```sql
CREATE TABLE idempotency_keys (
    key        UUID PRIMARY KEY,
    user_id    INT NOT NULL,
    endpoint   VARCHAR(100) NOT NULL,
    response   JSONB,
    status     VARCHAR(20) DEFAULT 'processing',
    created_at TIMESTAMPTZ DEFAULT NOW(),
    expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '24 hours'
);
CREATE INDEX ON idempotency_keys (expires_at);
```

Cột status có hai giá trị chính: processing khi request đang được xử lý, và done khi đã hoàn tất và kết quả đã được lưu vào cột response. Cột expires_at cùng với index đi kèm phục vụ cho việc dọn dẹp định kỳ các key cũ, tránh bảng phình to theo thời gian.

```go
func IdempotencyMiddleware(db *sql.DB) gin.HandlerFunc {
    return func(c *gin.Context) {
        idempKey := c.GetHeader("Idempotency-Key")
        if idempKey == "" {
            c.AbortWithStatusJSON(400, gin.H{"error": "thiếu Idempotency-Key"})
            return
        }

        var cachedResp []byte
        var status string
        err := db.QueryRow(
            `SELECT response, status FROM idempotency_keys WHERE key = $1 AND user_id = $2`,
            idempKey, c.GetInt("userID"),
        ).Scan(&cachedResp, &status)

        if err == nil {
            switch status {
            case "done":
                c.Header("X-Idempotent-Replayed", "true")
                c.Data(200, "application/json", cachedResp)
                c.Abort()
                return
            case "processing":
                c.AbortWithStatusJSON(409, gin.H{"error": "request đang được xử lý, vui lòng thử lại sau"})
                return
            }
        }

        db.Exec(
            `INSERT INTO idempotency_keys (key, user_id, endpoint) VALUES ($1, $2, $3)`,
            idempKey, c.GetInt("userID"), c.FullPath(),
        )
        c.Set("idempotencyKey", idempKey)
        c.Next()
    }
}
```

Middleware này hoạt động như một lớp kiểm tra trước handler thật sự. Khi nhận được request, nó tra cứu key trong database theo cặp (key, user_id) - sở dĩ phải kết hợp cả user_id là để đảm bảo cùng một key từ hai người dùng khác nhau không bị nhầm thành cùng một giao dịch. Nếu key đã tồn tại với trạng thái done, middleware lập tức trả về kết quả đã lưu kèm header X-Idempotent-Replayed: true để client biết đây là response được phục vụ từ cache, không phải kết quả của một lần xử lý mới. Nếu trạng thái là processing, tức là có luồng khác đang xử lý cùng request, trả về 409 để client biết cần đợi thay vì tiếp tục retry.

```go
func CreateBooking(c *gin.Context) {

    idempKey, _ := c.Get("idempotencyKey")
    responseBody, _ := json.Marshal(bookingResult)
    db.Exec(
        `UPDATE idempotency_keys SET response = $1, status = 'done' WHERE key = $2`,
        responseBody, idempKey,
    )
    c.JSON(201, bookingResult)
}
```

Bước lưu kết quả này phải được thực hiện **ngay trước khi trả response**, sau khi toàn bộ logic nghiệp vụ đã hoàn tất thành công. Thứ tự này quan trọng: nếu lưu trước khi xử lý xong và có lỗi xảy ra giữa chừng, các lần retry sau sẽ nhận được response done dù giao dịch thực ra chưa hoàn thành.

```go
go func() {
    for range time.NewTicker(1 * time.Hour).C {
        db.Exec(`DELETE FROM idempotency_keys WHERE expires_at < NOW()`)
    }
}()
```

Goroutine này chạy ngầm định kỳ mỗi giờ để xóa các key đã hết hạn 24 giờ. Đây là bước vệ sinh cần thiết - nếu bỏ qua, bảng idempotency_keys sẽ tích lũy hàng triệu bản ghi theo thời gian và làm chậm các câu query tra cứu.

5.5.3 Những Điều Cần Lưu Ý

Client phải tạo UUID mới cho mỗi giao dịch mới - khi retry cùng một giao dịch bị timeout thì dùng lại đúng key cũ. Đây là trách nhiệm của phía client và cần được ghi rõ trong tài liệu API. Với hệ thống có lưu lượng cao, có thể dùng Redis thay vì PostgreSQL để lưu idempotency key nhờ tốc độ đọc ghi nhanh hơn và tính năng tự động xóa theo TTL, tuy nhiên cần bật persistence để tránh mất dữ liệu khi Redis khởi động lại.

CHƯƠNG 6. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

6.1. Thu thập và đặc tả yêu cầu

6.1.1. Mô tả tổng quan hệ thống

Đề tài tập trung xây dựng một hệ thống đặt vé xem phim trực tuyến toàn diện. Hệ thống cho phép người dùng cuối thực hiện các thao tác từ đăng ký, đăng nhập, tra cứu lịch chiếu, lựa chọn ghế ngồi, đến thanh toán và quản lý vé đã đặt. Song song đó, hệ thống cung cấp một phân hệ quản trị (Admin Dashboard) hỗ trợ ban quản lý vận hành dữ liệu phim, rạp chiếu, lịch chiếu và đối soát doanh thu.

Hệ thống được thiết kế để phục vụ hai nhóm tác nhân (actors) chính:

- Người dùng cuối (Customer/User): Khách hàng có nhu cầu tra cứu thông tin điện ảnh và thực hiện giao dịch mua vé.
- Quản trị viên (Admin): Nhân sự vận hành hệ thống, chịu trách nhiệm quản lý nội dung và giám sát hoạt động kinh doanh.

6.1.2. Đặc tả chức năng người dùng (Customer)

a) Đăng ký tài khoản

- Mục tiêu: Cho phép người dùng tạo lập định danh cá nhân trên hệ thống.
- Dữ liệu đầu vào: Tên đăng nhập (duy nhất), Email (duy nhất), Mật khẩu, Họ và tên, Số điện thoại.
- Quy trình nghiệp vụ:
    1. Người dùng cung cấp thông tin qua biểu mẫu (form) đăng ký.
    2. Hệ thống thực hiện xác thực tính hợp lệ của dữ liệu: kiểm tra trùng lặp (username, email), kiểm tra định dạng email và các tiêu chuẩn bảo mật của mật khẩu.
    3. Nếu dữ liệu hợp lệ, hệ thống khởi tạo tài khoản ở trạng thái *chưa xác thực (unverified)*.

4. Hệ thống tự động khởi tạo và gửi mã OTP hoặc liên kết kích hoạt đến email đã đăng ký.

- Kết quả đầu ra: Thông báo thành công và yêu cầu xác thực email, hoặc trả về mã lỗi chi tiết nếu dữ liệu không thỏa mãn điều kiện.

b) Xác thực Email qua OTP

- Mục tiêu: Định danh chủ sở hữu email, ngăn chặn việc tạo tài khoản ảo và thư rác (spam).
- Dữ liệu đầu vào: Email, Mã OTP (6 ký tự).
- Quy trình nghiệp vụ:
    1. Người dùng nhập mã OTP nhận được vào màn hình xác thực.
    2. Hệ thống đối chiếu mã OTP dựa trên tính chính xác và hiệu lực thời gian (Time-to-live).
    3. Nếu hợp lệ, trạng thái tài khoản được cập nhật thành *đã xác thực (verified)*.
- Kết quả đầu ra: Thông báo kích hoạt thành công, cho phép người dùng đăng nhập hệ thống.

c) Đăng nhập và Đăng xuất

- Đăng nhập:
    - *Đầu vào:* Username/Email và Mật khẩu.
    - *Quy trình:* Hệ thống kiểm tra sự tồn tại của tài khoản, đối chiếu mật khẩu (đã được băm bảo mật) và kiểm tra trạng thái xác thực. Nếu hợp lệ, hệ thống cấp phát một phiên làm việc (Session/JWT Token) trả về cho phía client.
- Đăng xuất:
    - *Đầu vào:* Token hiện tại.
    - *Quy trình:* Hệ thống đưa Token hiện tại vào danh sách đen (Blacklist) hoặc vô hiệu hóa phiên làm việc trên server, đảm bảo Token cũ không thể tiếp tục truy xuất các tài nguyên bảo mật.

d) Tra cứu lịch chiếu phim

- Mục tiêu: Cung cấp thông tin chi tiết về các suất chiếu theo thời gian thực.
- Quy trình nghiệp vụ: Người dùng áp dụng các bộ lọc (theo cụm rạp, khu vực địa lý, ngày chiếu). Hệ thống truy xuất và trả về danh sách các suất chiếu thỏa mãn điều kiện, bao gồm: tên phim, rạp chiếu, khung giờ và giá vé tiêu chuẩn. Chỉ những suất chiếu trong tương lai và còn hiệu lực mới được hiển thị.

e) Truy xuất sơ đồ rạp và Lựa chọn ghế ngồi

- Mục tiêu: Trực quan hóa không gian phòng chiếu để người dùng lựa chọn vị trí phù hợp.
- Đầu vào: Mã rạp chiếu, Mã suất chiếu.
- Quy trình nghiệp vụ:

  1. Hệ thống tải lên sơ đồ ma trận ghế của phòng chiếu tương ứng.

  2. Trạng thái của từng ghế được biểu diễn rõ ràng: *Trống (Available)*, *Đang giữ chỗ (Pending)*, *Đã bán (Confirmed/Sold)*, kèm theo phân loại ghế (Thường, VIP, Ghế đôi).

  3. Người dùng thao tác chọn ghế. Hệ thống áp dụng cơ chế khóa tạm thời (Locking mechanism - ví dụ: 10 phút) đối với các ghế được chọn để tránh tình trạng tranh chấp dữ liệu (Race condition) giữa nhiều người dùng cùng lúc.

f) Khởi tạo Đơn đặt vé (Booking)

- Mục tiêu: Ghi nhận và lưu trữ yêu cầu giao dịch của người dùng.
- Đầu vào: Mã tài khoản, Mã suất chiếu, Danh sách mã ghế đã chọn.
- Quy trình nghiệp vụ:

  1. Hệ thống tái kiểm tra tính khả dụng của danh sách ghế tại thời điểm chốt yêu cầu.

  2. Nếu hợp lệ, khởi tạo một bản ghi Booking với trạng thái *Pending*.

3. Tính toán tổng giá trị đơn hàng dựa trên đơn giá ghế và các chương trình khuyến mãi (nếu có)

4. Chuyển hướng người dùng sang cổng thanh toán với một Mã đơn hàng (Booking ID) duy nhất.

g) Xử lý Thanh toán

- Mục tiêu: Xử lý luồng tiền tệ và xác nhận tính hợp lệ của đơn hàng.
- Đầu vào: Booking ID, Thông tin phương thức thanh toán.
- Quy trình nghiệp vụ:
  - Kiểm tra thời gian hiệu lực của Booking (còn trong thời gian giữ ghế).
  - Giao tiếp với API của bên thứ ba (Cổng thanh toán điện tử/Ngân hàng).
  - Nhận phản hồi (Webhook/Callback):
    - *Thành công:* Cập nhật trạng thái thanh toán (*Success*) và trạng thái vé (*Confirmed*).
    - *Thất bại:* Cập nhật trạng thái (*Failed*), giải phóng ghế và cho phép thực hiện lại giao dịch nếu chưa hết thời gian chờ.
  - *Ràng buộc kỹ thuật:* Đảm bảo tính toàn vẹn của giao dịch (Idempotency), tuyệt đối không xảy ra tình trạng trừ tiền trùng lặp cho một đơn hàng.

h) Quản lý Lịch sử đặt vé

- Mục tiêu: Cung cấp chức năng tra cứu thông tin các giao dịch đã thực hiện.
- Quy trình nghiệp vụ: Dựa vào Token định danh, hệ thống truy xuất toàn bộ lịch sử Bookings của người dùng, hỗ trợ phân trang và hiển thị chi tiết trạng thái của từng mã vé (QR Code nếu có).

6.1.3. Đặc tả chức năng Quản trị (Admin)

Phân hệ quản trị cung cấp các tính năng CRUD (Create, Read, Update, Delete) cho các thực thể cốt lõi của hệ thống:

- Quản lý Phim (Movies): Thêm mới, cập nhật thông tin (tiêu đề, mô tả, thời lượng, phân loại độ tuổi, poster, lịch chiếu dự kiến), và thay đổi trạng thái hiển thị của phim.
- Quản lý Cụm rạp (Theaters): Khởi tạo rạp chiếu mới, quản lý vị trí địa lý, cấu hình số lượng phòng chiếu và thiết lập cấu trúc ma trận sơ đồ ghế cho từng phòng.
- Quản lý Suất chiếu (Shows): Thiết lập lịch chiếu cụ thể cho từng bộ phim tại các phòng chiếu tương ứng. Quản lý linh hoạt khung giờ và chính sách giá vé theo thời điểm.
- Quản lý Danh mục Ghế (Seats): Tùy chỉnh trạng thái hoạt động của ghế (ví dụ: đánh dấu bảo trì) và phân loại định dạng ghế.
- Quản lý Thanh toán: Tích hợp và cấu hình trạng thái hoạt động của các đối tác thanh toán (VNPay, Momo,...).
- Giám sát & Báo cáo: Cung cấp bảng điều khiển (Dashboard) thống kê lưu lượng giao dịch, số lượng vé bán ra và biểu đồ doanh thu theo các mốc thời gian.

6.4. Thiết kế RESTful API (API Contract)

Hệ thống cung cấp hệ sinh thái API giao tiếp giữa Client và Server thông qua chuẩn RESTful, sử dụng định dạng JSON.

6.4.1. Phân hệ Định danh & Người dùng (Authentication & User)

- Đăng ký tài khoản: POST /api/auth/register
    - *Body:* username, email, password, full_name, phone
    - *Response:* 201 Created (Thành công) | 400 Bad Request (Dữ liệu không hợp lệ / Trùng lặp).
- Xác thực Email: POST /api/auth/verify-email
    - *Body:* email, otp_code
    - *Response:* 200 OK | 400 Bad Request (OTP sai/hết hạn).
- Đăng nhập: POST /api/auth/login

    - *Body:* email, password
    - *Response:* 200 OK (Kèm Access Token) | 401 Unauthorized.
- Đăng xuất: POST /api/auth/logout
    - *Header:* Authorization: Bearer <token>
    - *Response:* 200 OK.

6.4.2. Phân hệ Phim & Suất chiếu (Client-side)

- Lấy danh sách phim: GET /api/movies (Hỗ trợ query: page, limit, city, date) -> 200 OK.
- Lấy chi tiết phim: GET /api/movies/{movieId} -> 200 OK | 404 Not Found.
- Tra cứu suất chiếu: GET /api/shows (Query: movie_id, theater_id, date) -> 200 OK.

6.4.3. Phân hệ Rạp chiếu & Sơ đồ ghế

- Lấy danh sách rạp: GET /api/theaters -> 200 OK.
- Truy xuất sơ đồ ghế theo suất chiếu: GET /api/theaters/{theaterId}/seats?show_id={id} -> 200 OK (Trả về ma trận ghế và trạng thái).

6.4.4. Phân hệ Đặt vé (Bookings)

- Tạo mới đơn đặt vé: POST /api/bookings
    - *Header:* Authorization: Bearer <token>
    - *Body:* show_id, seat_ids (array), payment_method
    - *Response:* 201 Created (Trả về Booking ID) | 409 Conflict (Ghế đã bị đặt).
- Truy xuất lịch sử đặt vé: GET /api/bookings/me -> 200 OK.
- Xem chi tiết đơn vé: GET /api/bookings/{bookingId} -> 200 OK.

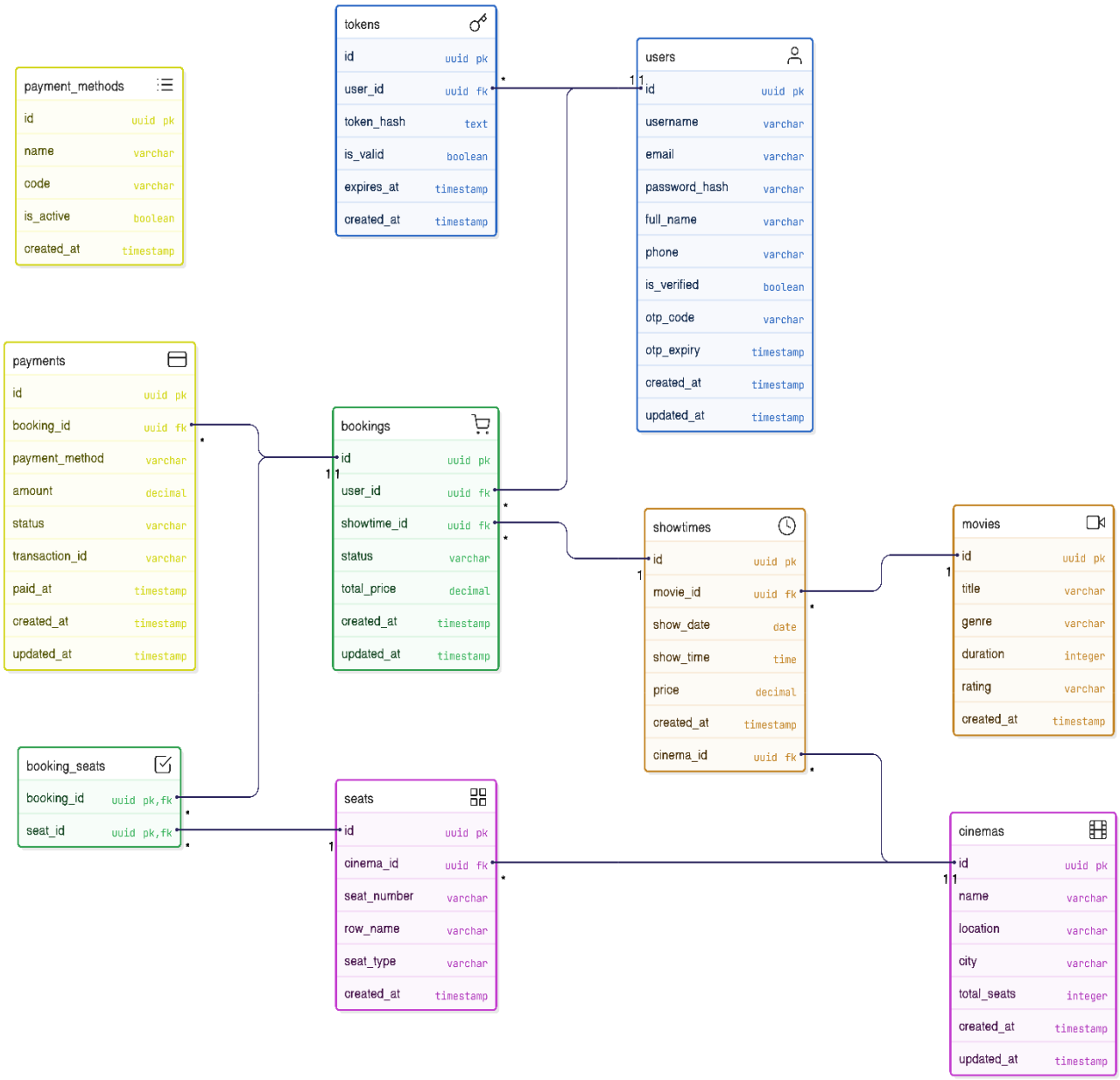
6.4.5. Phân hệ Thanh toán (Payments)

- Lấy danh sách cổng thanh toán: GET /api/payments/methods -> 200 OK.
- Tạo yêu cầu thanh toán: POST /api/payments
    - *Body:* booking_id, payment_method, payment_details
    - *Response:* 201 Created (Trả về Payment URL hoặc trạng thái xử lý).
- Nhận Callback từ Cổng thanh toán (Webhook): POST /api/payments/webhook/{provider} -> 200 OK (Hệ thống nội bộ xử lý logic cập nhật dữ liệu).

6.4.6. Phân hệ Quản trị (Admin APIs) *(Toàn bộ các endpoint nhóm này đều yêu cầu Header chứa Token có phân quyền Admin).*

- Quản lý Phim:
    - Thêm: POST /api/admin/movies
    - Sửa: PUT /api/admin/movies/{movieId}
    - Xóa: DELETE /api/admin/movies/{movieId}
- Quản lý Rạp chiếu:
    - Thêm/Sửa/Xóa qua: /api/admin/theaters/{id}
- Quản lý Suất chiếu:
    - Thêm/Sửa/Xóa qua: /api/admin/shows/{id}

6.5. Sơ đồ ERD


payment_methods
id uuid pk
name varchar
code varchar
is_active boolean
created_at timestamp
tokens
id uuid pk
user_id uuid fk
token_hash text
is_valid boolean
expires_at timestamp
created_at timestamp
users
id uuid pk
username varchar
email varchar
password_hash varchar
full_name varchar
phone varchar
is_verified boolean
otp_code varchar
otp_expiry timestamp
created_at timestamp
updated_at timestamp
payments
id uuid pk
booking_id uuid fk
payment_method varchar
amount decimal
status varchar
transaction_id varchar
paid_at timestamp
created_at timestamp
updated_at timestamp
bookings
id uuid pk
user_id uuid fk
showtime_id uuid fk
status varchar
total_price decimal
created_at timestamp
updated_at timestamp
showtimes
id uuid pk
movie_id uuid fk
show_date date
show_time time
price decimal
created_at timestamp
cinema_id uuid fk
movies
id uuid pk
title varchar
genre varchar
duration integer
rating varchar
created_at timestamp
booking_seats
booking_id uuid pk,fk
seat_id uuid pk,fk
seats
id uuid pk
cinema_id uuid fk
seat_number varchar
row_name varchar
seat_type varchar
created_at timestamp
cinemas
id uuid pk
name varchar
location varchar
city varchar
total_seats integer
created_at timestamp
updated_at timestamp

CHƯƠNG 7: TRIỂN KHAI VÀ KIỂM THỬ API

7.1. Công Cụ Kiểm Thử - Postman

7.1.1. Postman Là Gì ?

Postman là một ứng dụng desktop/web miễn phí được lập trình viên toàn thế giới sử dụng để kiểm thử API mà không cần viết code frontend. Thay vì phải mở trình duyệt hay xây dựng giao diện người dùng, Postman cho phép bạn gửi các HTTP request (GET, POST, PUT, DELETE...) trực tiếp đến server, xem kết quả trả về theo thời gian thực và kiểm tra API có hoạt động đúng như kỳ vọng hay không.

Dành cho người mới học: Hãy tưởng tượng bạn đang xây một nhà hàng (backend server). Postman chính là người khách hàng giả lập - họ vào quán, gọi món (gửi request), và bạn kiểm tra xem bếp có nấu đúng món không (kiểm tra response). Bạn không cần xây cả mặt tiền nhà hàng (frontend) chỉ để test xem bếp có nấu được không.

7.1.2. Cài Đặt Và Cấu Hình Môi Trường (Environment)

Bước 1 - Cài đặt Postman: Truy cập https://www.postman.com/downloads/, tải và cài đặt phiên bản phù hợp với hệ điều hành (Windows/macOS/Linux). Phần mềm hoàn toàn miễn phí cho cá nhân.

Bước 2 - Tạo Workspace: Tạo một Workspace mới đặt tên là *Cinema Booking API* để tập trung toàn bộ request liên quan đến dự án, tránh lẫn lộn với các project khác.

Bước 3 - Tạo Environment (Môi trường biến): Environment là nơi lưu các biến dùng chung để tránh nhập đi nhập lại. Nhóm tạo hai environment: *Local* (chạy máy tính cá nhân) và *Production* (server thật).

Tên Biến	Giá Trị (Local)	Mô Tả
base_url	*http://localhost:8080/api/v1*	Địa chỉ gốc API khi chạy local
auth_token	(để trống, tự điền sau login)	JWT Token sau khi đăng nhập
user_id	*1*	ID người dùng dùng để test
movie_id	*3*	ID bộ phim dùng để test
showtime_id	*5*	ID suất chiếu dùng để test
booking_id	(để trống, tự điền sau đặt vé)	ID đơn đặt vé vừa tạo

Cách dùng biến: Trong Postman, viết *{{ten_bien}}* ở bất kỳ đâu (URL, Header, Body). Ví dụ: *{{base_url}}/movies* sẽ tự động thay thành *http://localhost:8080/api/v1/movies*. Khi chuyển sang môi trường Production, chỉ cần đổi giá trị *base_url* là tất cả request cập nhật theo.

7.1.3. Cấu Trúc Collection Đã Tạo

Collection trong Postman là một "thư mục dự án" chứa tất cả các request được tổ chức theo nhóm chức năng. Nhóm đã xây dựng Collection *Cinema Booking API* với cấu trúc như sau:

Folder (Nhóm chức năng)	Số Request	Các Endpoint Chính
Auth - Xác thực	3	POST /register, POST /login, POST /logout
Movies - Phim	5	GET /movies, GET /movies/:id, POST /movies, PUT /movies/:id, DELETE /movies/:id
Cinemas & Rooms - Rạp & Phòng	6	GET /cinemas, POST /cinemas, GET /rooms/:cinema_id, POST /rooms, GET /seats/:room_id
Showtimes - Suất chiếu	4	GET /showtimes, POST /showtimes, GET /showtimes/:id, DELETE /showtimes/:id
Booking - Đặt vé	5	POST /bookings, GET /bookings/:id, GET /bookings/user/:uid, POST /payment, DELETE /bookings/:id
Users - Người dùng (Admin)	3	GET /users, GET /users/:id, PUT /users/:id/role

7.1.4. Hướng Dẫn Chi Tiết Thiết Lập Một Request

Để minh họa cách nhóm sử dụng Postman, dưới đây là hướng dẫn từng bước thiết lập request POST /bookings (tạo đơn đặt vé):

Bước 1 - Method và URL:

Method: POST
URL: {{base_url}}/bookings

Bước 2 - Headers (tiêu đề request):

Key	Value	Giải thích
Content-Type	*application/json*	Báo server biết body gửi lên là JSON
Authorization	*Bearer {{auth_token}}*	Gửi kèm token xác thực danh tính

Bước 3 - Body (dữ liệu gửi lên, chọn raw → JSON):

```
{
{
        "showtime_id": 5,
         "seat_ids": [12, 13],
         "payment_method": "cash"
}
```

Bước 4 - Pre-request Script (tự động lấy token trước khi chạy):

Script này chạy tự động trước mỗi request, đăng nhập và lưu token mới vào biến môi trường, đảm bảo token không bao giờ hết hạn khi test:

```
pm.sendRequest({
   url: pm.environment.get("base_url") + "/auth/login",
```

```
    method: "POST",
    header: { "Content-Type": "application/json" },
    body: {
        mode: "raw",
        raw: JSON.stringify({ email: "test@gmail.com", password: "123456" })
    }
}, function(err, res) {
    pm.environment.set("auth_token", res.json().data.token);
});
```

Bước 5 - Tests (kiểm tra kết quả tự động):

Postman cho phép viết JavaScript trong tab **Tests** để tự động kiểm tra response sau khi nhận được - không cần nhìn bằng mắt từng lần:

```javascript
// Kiểm tra HTTP status code phải là 201 Created
pm.test("Status 201 Created", () => {
    pm.response.to.have.status(201);
});

// Kiểm tra response có chứa booking_id
pm.test("Response trả về booking_id hợp lệ", () => {
    const json = pm.response.json();
    pm.expect(json.data).to.have.property("booking_id");
    pm.expect(json.data.status).to.equal("confirmed");
});

// Lưu booking_id vào biến để dùng cho request tiếp theo
pm.environment.set("booking_id", pm.response.json().data.booking_id);
```

Tính năng Collection Runner: Postman có công cụ **"Run Collection"** cho phép chạy toàn bộ tất cả request trong Collection theo thứ tự tự động, với báo cáo kết quả pass/fail rõ ràng. Nhóm dùng tính năng này để chạy luồng test end-to-end hoàn chỉnh: **Login → Xem phim → Chọn suất chiếu → Đặt vé → Thanh toán → Kiểm tra trạng thái** chỉ bằng một lần nhấn nút.

7.2. Tích Hợp Swagger - Tài Liệu API Tự Động

7.2.1. Swagger Là Gì ?

Swagger (còn gọi là OpenAPI Specification) là một tiêu chuẩn quốc tế để mô tả và tài liệu hóa API**.** Khi tích hợp vào dự án Go, nó tự động sinh ra một trang web tương tác đẹp mắt - gọi là Swagger UI - cho phép bất kỳ ai cũng có thể xem toàn bộ tài liệu API và thử gọi thẳng từ trình duyệt mà không cần cài đặt thêm bất kỳ công cụ nào.

Lợi ích cụ thể với dự án:

- Frontend developer biết chính xác cần gửi gì, nhận lại gì từ mỗi endpoint.
- Thầy cô khi chấm điểm có thể test API trực tiếp trên trình duyệt, không cần cài Postman hay dùng terminal.
- Toàn nhóm có tài liệu thống nhất, tránh hiểu nhầm về cấu trúc dữ liệu.

7.2.2. Các Bước Tích Hợp Swagger Vào Dự Án Go (Gin Framework)

Bước 1 - Cài đặt thư viện:

bash
go get -u github.com/swaggo/swag/cmd/swag
go get -u github.com/swaggo/gin-swagger
go get -u github.com/swaggo/files

Bước 2 - Viết annotation (chú thích đặc biệt) cho handler:

Swagger đọc các comment viết ngay trên hàm handler để biết cách mô tả endpoint đó. Ví dụ cho handler tạo booking:

```go
// CreateBooking godoc
// @Summary      Tạo đơn đặt vé mới
// @Description  Người dùng đã đăng nhập chọn ghế và tạo đơn đặt vé cho suất chiếu cụ thể
// @Tags         booking
// @Accept       json
// @Produce      json
// @Param        Authorization  header    string        true  "Bearer {token}"
// @Param        payload        body      BookingRequest true  "Thông tin đặt vé"
// @Success      201            {object}  BookingResponse      "Đặt vé thành công"
// @Failure      400            {object}  ErrorResponse        "Ghế đã được đặt hoặc dữ liệu sai"
// @Failure      401            {object}  ErrorResponse        "Chưa đăng nhập hoặc token hết hạn"
// @Failure      500            {object}  ErrorResponse        "Lỗi server"
// @Router       /bookings [post]
func (h *BookingHandler) CreateBooking(c *gin.Context) {
    // ... code xử lý
}
```

Bước 3 - Đăng ký route cho Swagger UI trong router:

```go
import (
    swaggerFiles "github.com/swaggo/files"
    ginSwagger   "github.com/swaggo/gin-swagger"
    _ "your-project/docs" // import docs được sinh tự động
)

// Thêm vào hàm khởi tạo router
r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))
```

Bước 4 - Sinh file tài liệu tự động:

bash
Chạy lệnh này từ thư mục gốc dự án mỗi khi cập nhật annotation
swag init -g cmd/main.go -o docs/
```

Lệnh này sẽ tự động tạo thư mục `*docs/*` chứa các file `*swagger.json*`, `*swagger.yaml*` và `*docs.go*` - đây là nguồn dữ liệu để Swagger UI hiển thị.

**Bước 5 - Truy cập Swagger UI:**
```
http://localhost:8080/swagger/index.html
```

---

## 7.3. Bảng Kiểm Thử Tổng Hợp

| STT | Tên Test Case | Module | Loại | Method | Endpoint | Điều Kiện / Dữ Liệu Đầu Vào | Kết Quả Mong Đợi | HTTP Code | Kết Quả Thực Tế | Trạng Thái |
|---|---|---|---|---|---|---|---|---|---|---|
| TC-01 | Đăng ký tài khoản thành công | Auth | Happy Path | POST | /api/v1/register | Email chưa tồn tại, đủ các trường: name, email, password hợp lệ | Trả về user_id và message "Đăng ký thành công" | 201 | {"data":{"user_id":7,"name":"Nguyen Van A"},"message":"Đăng ký thành công"} | PASS |
```

TC-02	Đăng ký email đã tồn tại	Auth	Error Path	POST	/api/v1/register	Email "vana@test.com" đã có trong hệ thống	Báo lỗi trùng email, không tạo tài khoản	409	{"error":"Email này đã được đăng ký. Vui lòng dùng email khác hoặc đăng nhập."}	PASS
TC-03	Đăng ký thiếu trường bắt buộc	Auth	Error Path	POST	/api/v1/register	Body thiếu trường password: {"name":"A","email":"a@test.com"}	Báo lỗi validation, chỉ rõ trường còn thiếu	400	{"error":"Trường password là bắt buộc và không được để trống"}	PASS
TC-04	Đăng ký email sai định dạng	Auth	Error Path	POST	/api/v1/register	Email không đúng chuẩn: {"email":"khong-phai-email","password":"Pass@123"}	Báo lỗi định dạng email không hợp lệ	400	{"error":"Email không đúng định dạng"}	PASS
TC-05	Đăng ký password quá ngắn	Auth	Error Path	POST	/api/v1/register	Password ít hơn 6 ký tự: {"email":"b@test.com", "password": "123"}	Báo lỗi password không đủ độ dài tối thiểu	400	{"error":"Mật khẩu phải có ít nhất 6 ký tự"}	PASS
TC-06	Đăng nhập thành công	Auth	Happy Path	POST	/api/v1/login	Đúng email và password đã đăng ký: {"email":"vana@test.com","passwo	Trả về JWT token hợp lệ và expires_in	200	{"data":{"token":"eyJhbGci...","expires_in":86400}}	PASS

						rd":"Pass@123"}				
TC-07	Đăng nhập sai mật khẩu	Auth	Error Path	POST	/api/v1/login	Email đúng, password sai: {"email":"vana@test.com","password":"SaiMatKhau"}	Thông báo chung, không tiết lộ trường nào sai	401	{"error":"Email hoặc mật khẩu không đúng"}	PASS
TC-08	Đăng nhập email không tồn tại	Auth	Error Path	POST	/api/v1/login	Email chưa đăng ký: {"email":"khongco@test.com","password":"Pass@123"}	Thông báo chung, không tiết lộ email chưa tồn tại	401	{"error":"Email hoặc mật khẩu không đúng"}	PASS
TC-09	Đăng xuất thành công	Auth	Happy Path	POST	/api/v1/logout	Token hợp lệ trong header Authorization	Token bị vô hiệu hóa, thêm vào blacklist	200	{"message":"Đăng xuất thành công"}	PASS
TC-10	Gọi API bảo vệ không có token	Auth	Error Path	POST	/api/v1/bookings	Không gửi header Authorization	Từ chối, yêu cầu đăng nhập	401	{"error":"Vui lòng đăng nhập để thực hiện chức năng này"}	PASS
TC-11	Gọi API với token giả mạo	Auth	Error Path	POST	/api/v1/bookings	Authorization: Bearer eyJhbGci.INVALID.token	Từ chối token không hợp lệ	401	{"error":"Token không hợp lệ hoặc đã hết hạn. Vui lòng đăng nhập lại."}	PASS

TC-12	Gọi API với token đã hết hạn	Auth	Error Path	GET	/api/v1/bookings/user/1	Token đã quá thời hạn expires_in	Báo token hết hạn, yêu cầu đăng nhập lại	401	{"error":"Token đã hết hạn. Vui lòng đăng nhập lại."}	PASS
TC-13	Lấy danh sách phim đang chiếu	Movie	Happy Path	GET	/api/v1/movies?status=showing	Không cần đăng nhập, public endpoint	Mảng phim đang chiếu với đầy đủ thông tin	200	Array gồm 5 phim, mỗi phim có movie_id, title, duration, genre, poster_url	PASS
TC-14	Lấy danh sách phim sắp chiếu	Movie	Happy Path	GET	/api/v1/movies?status=coming_soon	Không cần đăng nhập, public endpoint	Mảng phim sắp ra mắt	200	Array gồm 3 phim với status "coming_soon"	PASS
TC-15	Lấy chi tiết một bộ phim hợp lệ	Movie	Happy Path	GET	/api/v1/movies/3	movie_id = 3 tồn tại trong DB	Thông tin đầy đủ của phim bao gồm mô tả, trailer	200	{"movie_id":3,"title":"Avengers", "duration":180," description":"...", "trailer_url":"..." }	PASS
TC-16	Lấy chi tiết phim không tồn tại	Movie	Error Path	GET	/api/v1/movies/9999	movie_id = 9999 không có trong DB	Báo không tìm thấy phim	404	{"error":"Không tìm thấy phim với ID 9999"}	PASS
TC-17	Tìm kiếm phim theo tên	Movie	Happy Path	GET	/api/v1/movies?search=Avengers	Từ khóa hợp lệ có phim khớp trong DB	Danh sách phim chứa từ	200	Array các phim có title chứa "Avengers"	PASS

							khóa trong tên			
TC-18	Tìm kiếm phim không có kết quả	Movie	Happy Path	GET	/api/v1/movies?search=xyzabckhongco	Từ khóa không khớp bất kỳ phim nào	Trả về mảng rỗng, không báo lỗi	200	{"data":[],"message":"Không tìm thấy phim phù hợp"}	PASS
TC-19	Admin thêm phim mới	Movie	Happy Path	POST	/api/v1/movies	Token role admin, body đầy đủ: title, duration, genre, status	Phim được tạo, trả về movie_id mới	201	{"data":{"movie_id":8,"title":"Phim Mới"},"message":"Tạo phim thành công"}	PASS
TC-20	User thường cố thêm phim	Movie	Error Path	POST	/api/v1/movies	Token role user (không phải admin)	Từ chối, báo không đủ quyền	403	{"error":"Bạn không có quyền thực hiện thao tác này"}	PASS
TC-21	Admin cập nhật thông tin phim	Movie	Happy Path	PUT	/api/v1/movies/3	Token admin, body cập nhật title và status	Thông tin phim được cập nhật thành công	200	{"message":"Cập nhật phim thành công","data":{"movie_id":3,"title":"Tên Đã Sửa"}}	PASS
TC-22	Admin xóa phim không có suất chiếu	Movie	Happy Path	DELETE	/api/v1/movies/8	Token admin, phim tồn tại và không có suất chiếu tương lai	Phim bị xóa khỏi hệ thống	200	{"message":"Xóa phim thành công"}	PASS
TC-23	Admin xóa phim	Movie	Error Path	DELETE	/api/v1/movies/3	Token admin, phim đang	Từ chối xóa, báo lý do còn	409	{"error":"Không thể xóa phim đang có suất	PASS

	đang có suất chiếu					có suất chiếu trong tương lai	suất chiếu đang hoạt động		chiếu. Vui lòng hủy suất chiếu trước."}	
TC-24	Lấy danh sách rạp chiếu phim	Cinema	Happy Path	GET	/api/v1/cinemas	Không cần đăng nhập	Danh sách tất cả rạp với địa chỉ và số phòng	200	Array 3 rạp với cinema_id, name, address, total_rooms	PASS
TC-25	Lấy danh sách phòng chiếu của rạp	Cinema	Happy Path	GET	/api/v1/rooms?cinema_id=1	cinema_id = 1 tồn tại	Danh sách phòng thuộc rạp, kèm sức chứa	200	Array phòng với room_id, name, capacity, room_type	PASS
TC-26	Lấy sơ đồ ghế của phòng chiếu	Cinema	Happy Path	GET	/api/v1/seats/room/2	room_id = 2 tồn tại	Toàn bộ ghế trong phòng với mã ghế và loại ghế	200	Array 80 ghế với seat_id, seat_code (A1, A2...), seat_type (normal/vip)	PASS
TC-27	Admin thêm rạp chiếu mới	Cinema	Happy Path	POST	/api/v1/cinemas	Token admin, body: name và address đầy đủ	Rạp được tạo, trả về cinema_id mới	201	{"data":{"cinema_id":4,"name":"CGV Quan 1"},"message":"Thêm rạp thành công"}	PASS
TC-28	Admin thêm phòng	Cinema	Happy Path	POST	/api/v1/rooms	Token admin, body: cinema_id,	Phòng được tạo và liên	201	{"data":{"room_id":7,"name":"Phòng	PASS

	chiếu vào rạp					name, capacity, room_type	kết với rạp		3","capacity":100}}	
TC-29	Lấy danh sách suất chiếu theo phim và ngày	Showtime	Happy Path	GET	/api/v1/showtimes?movie_id=3&date=2025-08-15	movie_id và date hợp lệ, có suất chiếu trong ngày	Danh sách suất chiếu kèm thông tin rạp, phòng, giờ	200	Array 3 suất chiếu với showtime_id, room_name, start_time, available_seats	PASS
TC-30	Lấy suất chiếu của ngày không có lịch	Showtime	Happy Path	GET	/api/v1/showtimes?movie_id=3&date=2025-01-01	Ngày hợp lệ nhưng không có lịch chiếu	Trả về mảng rỗng, không báo lỗi	200	{"data":[],"message":"Không có suất chiếu trong ngày này"}	PASS
TC-31	Admin tạo suất chiếu mới	Showtime	Happy Path	POST	/api/v1/showtimes	Token admin, body: movie_id, room_id, start_time hợp lệ	Suất chiếu được tạo, trả về showtime_id	201	{"data":{"showtime_id":10,"start_time":"2025-08-20T19:00:00Z"}}	PASS
TC-32	Admin tạo suất chiếu trùng giờ phòng	Showtime	Error Path	POST	/api/v1/showtimes	Token admin, room_id và start_time trùng với suất đã tồn tại	Báo lỗi xung đột lịch chiếu	409	{"error":"Phòng chiếu này đã có suất chiếu trong khung giờ trên. Vui lòng chọn giờ khác."}	PASS

TC-33	Admin xóa suất chiếu chưa bán vé	Showtime	Happy Path	DELETE	/api/v1/showtimes/10	Token admin, suất chiếu tồn tại và chưa có vé nào được bán	Suất chiếu bị xóa khỏi hệ thống	200	{"message":"Xóa suất chiếu thành công"}	PASS
TC-34	Admin xóa suất chiếu đã có người đặt vé	Showtime	Error Path	DELETE	/api/v1/showtimes/5	Token admin, suất chiếu đã có booking confirmed	Từ chối xóa để bảo vệ vé người dùng đã mua	409	{"error":"Không thể xóa suất chiếu đã có vé được đặt."}	PASS
TC-35	Xem trạng thái ghế theo suất chiếu	Showtime	Happy Path	GET	/api/v1/seats/showtime/5	showtime_id = 5 hợp lệ	Mảng ghế với trạng thái available hoặc booked	200	Array 80 ghế: 12 ghế status "booked", còn lại status "available"	PASS
TC-36	Đặt vé thành công	Booking	Happy Path	POST	/api/v1/bookings	Token hợp lệ, showtime_id và seat_ids hợp lệ, ghế còn trống	booking_id mới, status pending, danh sách ghế giữ	201	{"booking_id":42,"status":"pending","seats":["B3","B4"],"total_price":160000,"expires_at":"..."}	PASS
TC-37	Đặt ghế đã có người đặt	Booking	Error Path	POST	/api/v1/bookings	Token hợp lệ, seat_ids chứa ghế đã ở trạng thái booked	Báo lỗi ghế không còn trống, không tạo booking	409	{"error":"Ghế B3 đã được đặt bởi người khác. Vui lòng chọn ghế khác."}	PASS

TC-38	Đặt vé cho suất chiếu đã kết thúc	Booking	Error Path	POST	/api/v1/bookings	showtime_id ứng với suất chiếu trong quá khứ	Báo lỗi suất chiếu đã kết thúc	400	{"error":"Suất chiếu này đã kết thúc. Không thể đặt vé."}	PASS
TC-39	Đặt vé thiếu trường seat_ids	Booking	Error Path	POST	/api/v1/bookings	Body chỉ có showtime_id, thiếu seat_ids: {"showtime_id":5}	Báo lỗi validation thiếu trường bắt buộc	400	{"error":"Trường seat_ids là bắt buộc và phải là mảng số nguyên không rỗng"}	PASS
TC-40	Thanh toán hoàn tất đơn đặt vé	Booking	Happy Path	POST	/api/v1/payment	booking_id hợp lệ, còn trong thời gian giữ chỗ, method hợp lệ	Trạng thái chuyển confirmed, trả về ticket_code	200	{"status":"confirmed","ticket_code":"TK-20250815-042","message": "Thanh toán thành công"}	PASS
TC-41	Thanh toán booking đã hết thời gian giữ chỗ	Booking	Error Path	POST	/api/v1/payment	booking_id hợp lệ nhưng đã quá thời gian giữ chỗ (>15 phút)	Báo lỗi hết thời gian, ghế đã được giải phóng	400	{"error":"Đơn đặt vé đã hết thời gian giữ chỗ. Vui lòng đặt lại."}	PASS
TC-42	Thanh toán booking không thuộc về mình	Booking	Error Path	POST	/api/v1/payment	Token của user A nhưng booking_id thuộc về user B	Từ chối, báo không có quyền	403	{"error":"Bạn không có quyền thanh toán đơn đặt vé này."}	PASS

TC-43	Xem chi tiết đơn đặt vé của chính mình	Booking	Happy Path	GET	/api/v1/bookings/42	Token user đã tạo booking_id = 42	Thông tin đầy đủ của booking và danh sách ghế	200	{"booking_id":42,"status":"confirmed","movie_title":"...","seats":["B3","B4"],"ticket_code":"..."}	PASS
TC-44	Xem đơn đặt vé của người khác	Booking	Error Path	GET	/api/v1/bookings/42	Token user A nhưng booking_id = 42 thuộc user B	Từ chối, báo không có quyền xem	403	{"error":"Bạn không có quyền xem đơn đặt vé này."}	PASS
TC-45	Xem lịch sử đặt vé của bản thân	Booking	Happy Path	GET	/api/v1/bookings/user/7	Token hợp lệ, user_id = 7 là chính mình	Danh sách toàn bộ booking đã tạo theo thời gian giảm dần	200	Array các booking với đầy đủ thông tin phim, ghế, trạng thái	PASS
TC-46	Hủy đơn đặt vé đang pending	Booking	Happy Path	DELETE	/api/v1/bookings/42	Token hợp lệ, booking_id đang ở trạng thái pending (chưa thanh toán)	Booking bị hủy, ghế được giải phóng trở về available	200	{"message":"Hủy đơn đặt vé thành công. Ghế đã được giải phóng."}	PASS
TC-47	Hủy đơn vé đã thanh toán confirmed	Booking	Error Path	DELETE	/api/v1/bookings/43	Token hợp lệ, booking_id đã ở trạng thái confirmed	Từ chối hủy vé đã thanh toán, hướng	400	{"error":"Không thể hủy vé đã được xác nhận thanh toán. Vui lòng liên hệ hỗ trợ."}	PASS

							dẫn liên hệ hỗ trợ			
TC-48	Admin xem danh sách tất cả người dùng	User	Happy Path	GET	/api/v1/admin/users	Token role admin	Danh sách toàn bộ người dùng với thông tin cơ bản	200	Array users với user_id, name, email, role, created_at	PASS
TC-49	User thường cố xem danh sách user	User	Error Path	GET	/api/v1/admin/users	Token role user (không phải admin)	Từ chối, báo không đủ quyền truy cập	403	{"error":"Bạn không có quyền truy cập chức năng này."}	PASS
TC-50	Admin thay đổi role người dùng	User	Happy Path	PUT	/api/v1/admin/users/7/role	Token admin, body: {"role":"admin"}	Role người dùng được cập nhật thành công	200	{"message":"Cập nhật quyền thà	

7.4. Tổng Kết Kết Quả Kiểm Thử

Nhóm Chức Năng	Tổng Test Case	Happy Path	Error Path	Passed	Failed	Tỉ Lệ Passed
Auth (Xác thực)	12	4	8	12	0	100%
Movie (Quản lý phim)	11	7	4	11	0	100%
Cinema & Room (Rạp & Phòng)	5	5	0	5	0	100%
Showtime (Suất chiếu)	7	4	3	7	0	100%
Booking & Payment (Đặt vé)	12	6	6	12	0	100%
User Admin (Quản trị)	3	2	1	3	0	100%
Tổng cộng	**50**	**28**	**22**	**50**	**0**	**100%**

CHƯƠNG 8: ỨNG DỤNG NGÔN NGỮ GOLANG THỰC HIỆN WEB BOOKING CINEMA

8.1. Xác Thực Và Phân Quyền Người Dùng (Jwt + Middleware Pattern)

8.1.1 Lý thuyết JSON Web Token (JWT)

JWT (JSON Web Token) là một tiêu chuẩn mở (RFC 7519) định nghĩa cách truyền thông tin an toàn giữa các bên dưới dạng đối tượng JSON. Token gồm 3 phần: header.payload.signature

- Header: Khai báo thuật toán mã hóa (HS256 – HMAC với SHA-256).
- Payload (Claims): Dữ liệu cần truyền (user_id, email, thời gian hết hạn).
- Signature: HMAC-SHA256(base64url(header) + "." + base64url(payload), secret). Chỉ server biết secret, đảm bảo token không bị giả mạo.

JWT là Stateless: Server không cần lưu session, chỉ cần verify chữ ký là biết ngay ai đang gọi. Điều này giúp hệ thống scale theo chiều ngang (horizontal scaling) không cần shared session store.

8.1.2 Lý thuyết Middleware Pattern trong Go

Go không có class hay kế thừa. Thay vào đó, Go sử dụng Higher-Order Functions (hàm nhận hàm làm tham số, trả về hàm). Middleware trong Go là một hàm nhận http.Handler và trả về http.Handler khác đã được "bọc" thêm logic.

Đây là ứng dụng của Decorator Pattern – một trong các Design Pattern (GoF) nhưng được diễn đạt cực kỳ ngắn gọn và tự nhiên nhờ tính năng "functions as first-class citizens" của Go.

Request → [AuthMiddleware] → [LoggerMiddleware] → [Handler]

Mỗi Middleware trong chuỗi chỉ làm một việc, tuân theo Single Responsibility Principle (SRP) trong SOLID.

8.1.3 Lý thuyết Context (Truyền dữ liệu xuyên suốt request)

Context.Context trong Go là một công cụ cơ bản để:

1. Truyền metadata qua các ranh giới API (giữa Handler, Service, Repository) mà không cần thay đổi chữ ký hàm.
2. Kiểm soát timeout và cancellation: Khi một request bị hủy (client ngắt kết nối), context tự động cancel, các goroutine bên dưới dừng lại, tránh lãng phí tài nguyên.

context.WithValue tạo ra một bản sao mới của context với key-value được thêm vào. Điều này đảm bảo immutability – không có race condition khi nhiều goroutine đọc cùng một context.

8.1.4 Use Case người dùng

Thành phần	**Cụ thể**
Tên Use Case	Đăng nhập và truy cập tài nguyên được bảo vệ bằng JWT
Tác nhân	Người dùng đã đăng ký (Customer / Admin)
Mô tả ngắn gọn	Người dùng đăng nhập bằng email và mật khẩu. Hệ thống tạo JWT có thời hạn và trả về cho trình duyệt. Mọi request tới tài nguyên bảo vệ đều phải mang token hợp lệ. AuthMiddleware tự động xác thực token và truyền thông tin người dùng xuống Handler qua Go Context mà không cần truy vấn Database lần nào.
Điều kiện tiên quyết	Người dùng đã có tài khoản. Tài khoản đã được xác thực OTP. Cấu hình JWT_SECRET và JWT_EXPIRY_HOURS đã được thiết lập trên server.
Luồng sự kiện chính	1. Người dùng nhập email và mật khẩu vào form đăng nhập, nhấn "Đăng nhập". 2. Handler gọi AuthService.Login(), hash mật khẩu bằng bcrypt và so sánh với dữ liệu trong Database. 3. Nếu khớp, hệ thống gọi GenerateToken() tạo JWT với payload chứa UserID, Email, Username và thời gian hết hạn. Token được ký bằng thuật toán HMAC-SHA256 với secretKey. 4. Frontend lưu token vào localStorage. 5. Người dùng truy cập "Vé của tôi". Trình duyệt gửi request kèm header Authorization: Bearer <token>.

	6. AuthMiddleware trích xuất token, gọi ParseToken() giải mã và xác thực chữ ký. Claims được nhúng vào context.Context của request. 7. Handler gọi GetClaims(ctx) lấy UserID và trả về đúng dữ liệu của người dùng.
Luồng ngoại lệ	Bước 5: Nếu token hết hạn (ExpiresAt đã qua), ParseToken() trả về lỗi, Middleware trả ngay 401 Unauthorized – không gọi Handler. Bước 2: Nếu mật khẩu sai, trả về 401 với message chung (không tiết lộ lý do cụ thể để tránh enumeration attack). Bước 6: Nếu header Authorization thiếu hoặc không đúng format Bearer <token>, trả về 401 Unauthorized.
Hậu điều kiện	Người dùng nhận được danh sách vé cá nhân. UserID được lưu trong context, sẵn sàng cho toàn bộ pipeline xử lý request mà không cần query lại Database.

8.1.5 Ứng dụng thực tế trong dự án

File: internal/utils/jwt.go – Tạo và giải mã token

```go
var (
    ErrInvalidToken = errors.New("invalid token")
)

// Claims for JWT (custom fields as needed).
type Claims struct {
    UserID   string `json:"user_id"`
    Email    string `json:"email"`
    Username string `json:"username"`
    jwt.RegisteredClaims
}

// GenerateToken creates a JWT for the user. Expiry from now + expiryHours.
func GenerateToken(secret string, userID, email, username string, expiryHours int) (string, error) {
    exp := time.Now().Add(time.Duration(expiryHours) * time.Hour)
    claims := &Claims{
        UserID:   userID,
        Email:    email,
        Username: username,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(exp),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(secret))
}

// ParseToken validates and returns claims. Returns ErrInvalidToken on failure.
func ParseToken(secret, tokenString string) (*Claims, error) {
    token, err := jwt.ParseWithClaims(tokenString, &Claims{}, func(t *jwt.Token) (interface{}, error) {
        if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, ErrInvalidToken
        }
        return []byte(secret), nil
    })
    if err != nil {
        return nil, ErrInvalidToken
    }
    claims, ok := token.Claims.(*Claims)
    if !ok || !token.Valid {
        return nil, ErrInvalidToken
    }
    return claims, nil
}
```

File: internal/middleware/auth_middleware.go – Middleware Closure

```go
import (
    "context"
    "net/http"
    "strings"

    "booking_cinema_golang/internal/utils"
)

type contextKey string

const UserClaimsKey contextKey = "user_claims"

// AuthMiddleware validates JWT and sets claims in context. Use JWT_SECRET from config.
func AuthMiddleware(jwtSecret string) func(next http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            auth := r.Header.Get("Authorization")
            if auth == "" {
                utils.JSONUnauthorized(w, "missing authorization header")
                return
            }
            parts := strings.SplitN(auth, " ", 2)
            if len(parts) != 2 || parts[0] != "Bearer" {
                utils.JSONUnauthorized(w, "invalid authorization format")
                return
            }
            claims, err := utils.ParseToken(jwtSecret, parts[1])
            if err != nil {
                utils.JSONUnauthorized(w, "invalid or expired token")
                return
            }
            ctx := context.WithValue(r.Context(), UserClaimsKey, claims)
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// GetClaims returns Claims from request context (nil if not set).
func GetClaims(ctx context.Context) *utils.Claims {
    c, _ := ctx.Value(UserClaimsKey).(*utils.Claims)
    return c
}
```

8.2. Xử Lý Đồng Thời Và Race Condition (Concurrency Control)

8.2.1 Lý thuyết Concurrency trong Go: Goroutines và Scheduler

Go cung cấp Goroutines – lightweight threads được quản lý bởi Go Runtime, không phải OS thread. Mỗi goroutine ban đầu chỉ chiếm ~2KB stack (so với 1-8MB của OS thread).

Go Runtime sử dụng mô hình M:N threading (M goroutines ánh xạ vào N OS threads) thông qua bộ scheduler G-P-M:

- G (Goroutine): Đơn vị thực thi.
- P (Processor): Ngữ cảnh thực thi, mỗi P có local run queue.
- M (Machine/OS thread): Thread thực sự.

Điều này cho phép hàng nghìn goroutines tồn tại song song mà không tốn nhiều bộ nhớ, phù hợp cho hệ thống xử lý đồng thời nhiều yêu cầu đặt vé.

8.2.2 Lý thuyết CSP (Communicating Sequential Processes) và Channels

Go áp dụng triết lý CSP của Tony Hoare: "Do not communicate by sharing memory; instead, share memory by communicating." Thay vì các goroutine cùng truy cập một biến shared và lock nó, chúng giao tiếp qua Channels – các ống dẫn an toàn type-safe. Channel trong Go hoạt động theo cơ chế FIFO (First In, First Out) và là concurrent-safe theo thiết kế của ngôn ngữ (không cần lock thủ công).

- Unbuffered channel: Sender block cho đến khi Receiver sẵn sàng và ngược lại – đảm bảo đồng bộ hóa tuyệt đối giữa hai goroutine.
- Buffered channel: Sender chỉ block khi buffer đầy – cho phép phi đồng bộ có giới hạn.

8.2.3 Lý thuyết sync.Mutex và sync.RWMutex

Khi cần chia sẻ state (ví dụ: map các providers), Go cung cấp sync.Mutex (khóa độc quyền) và sync.RWMutex (cho phép nhiều reader đồng thời, một writer độc quyền).

sync.RWMutex phù hợp khi đọc nhiều hơn ghi – giúp tối ưu throughput so với Mutex thông thường.

8.2.4 Lý thuyết Distributed Lock với Redis (SetNX)

Trong môi trường phân tán (nhiều instance server), sync.Mutex chỉ hoạt động trong cùng một process. Để đồng bộ xuyên các server, cần Distributed Lock.

Redis cung cấp lệnh SETNX (SET if Not eXists): Atomically gán giá trị cho key nếu chưa tồn tại. Tính atomic được đảm bảo bởi Redis single-threaded model – không thể có hai client nào cùng SETNX thành công với cùng key.

Kết hợp SETNX với EXPIRE (hoặc SET key value NX EX seconds), ta tạo được khóa phân tán có TTL (Time-To-Live), tự giải phóng nếu client crash.

8.2.5 Lý thuyết Pessimistic Locking trong Database

SELECT ... FOR UPDATE trong PostgreSQL thực hiện Row-Level Lock – khóa cứng các hàng đã chọn cho đến khi transaction kết thúc. Các transaction khác cố đọc các hàng này với FOR UPDATE sẽ bị block hoặc fail ngay (với NOWAIT).

Đây là kỹ thuật Pessimistic Locking – phù hợp cho dữ liệu tranh chấp cao như ghế xem phim, vì giả định xung đột sẽ xảy ra và ngăn chặn từ đầu.

8.2.6 Use Case

Thành phần	Cụ thể
Tên Use Case	Chọn ghế và bảo vệ tính toàn vẹn trước race condition
Tác nhân	Hai người dùng (User A, User B) cùng xem trang đặt vé của một suất chiếu
Mô tả ngắn gọn	Hệ thống sử dụng hai lớp khóa (Redis Distributed Lock và PostgreSQL Pessimistic Lock) để đảm bảo tuyệt đối rằng mỗi ghế chỉ có thể được đặt bởi một người tại một thời điểm, dù có bao nhiêu người dùng cùng cố gắng.
Điều kiện tiên quyết	Cả hai người dùng đã đăng nhập. Suất chiếu đang mở nhận đặt vé. Ghế A5 hiện đang ở trạng thái trống.
Luồng sự kiện chính	1. User A nhấn chọn ghế A5. Frontend gọi POST /bookings/lock với seat_id = A5. 2. BookingService.LockSeat() gọi Redis.SetNX("seat_lock:showtime:A5", "locked", 10min). Redis atomic operation thành công, trả về true. 3. Hub broadcast WebSocket: Ghế A5 chuyển màu vàng (Holding) trên màn hình tất cả người đang xem cùng suất, kể cả User B. 4. User A tiếp tục chọn thêm ghế khác (nếu muốn), sau đó nhấn "Xác nhận đặt vé". 5. BookingService.CreateBooking() mở PostgreSQL Transaction, thực thi SELECT ... FOR UPDATE NOWAIT để khóa hàng ghế A5 ở tầng Database. 6. Query COUNT(*) kiểm tra lại: Không có booking confirmed hay pending (< 5 phút) nào cho ghế A5. 7. INSERT INTO bookings với status pending. Transaction commit.

	8. Redis lock được giải phóng. Hub broadcast: Ghế A5 chuyển màu đen (Sold). 9. Frontend chuyển hướng User A tới trang thanh toán với đồng hồ đếm ngược 5 phút.
Luồng ngoại lệ	Bước 2 (User B nhấn A5): Redis.SetNX("seat_lock:showtime:A5") trả về false vì key đã tồntại. Hệ thống trả về 409 Conflict với message "Ghế đang bị giữ bởi người khác". Bước 5 (Database conflict): Nếu Redis bị restart và cả hai đều vượt qua Layer 1, SELECT ... FOR UPDATE NOWAIT của transaction thứ 2 fail ngay với lỗi. Hệ thống trả về ErrSeatAlreadyTaken. Bước 6 (Double Check fail): Nếu booking tồn tại, existingSeatCount > 0, transaction rollback tự động qua defer tx.Rollback(ctx).
Hậu điều kiện	Đúng một booking được tạo trong Database với status pending. Ghế A5 hiển thị màu đen cho tất cả người xem real-time. User A đang trong quy trình thanh toán 5 phút. User B thấy ghế A5 không thể chọn.

8.2.7 Ứng dụng thực tế trong dự án

File: internal/infrastructure/redis/redis.go

```go
func (c *Client) LockSeat(ctx context.Context, showtimeID, seatID string, expiration time.Duration) (bool, error) {
    key := fmt.Sprintf("seat_lock:%s:%s", showtimeID, seatID)
    return c.rdb.SetNX(ctx, key, "locked", expiration).Result() // Atomic: true=lock OK
}
```

File: internal/repository/postgres_booking_repository.go

```go
tx, _ := r.db.Pool.Begin(ctx)
defer tx.Rollback(ctx) // Idiomatic Go: Auto rollback nếu không commit

// ORDER BY id: Luôn lock theo thứ tự cố định → Tránh deadlock
tx.Query(ctx, `SELECT id, seat_number FROM seats
    WHERE room_id = $1 AND seat_number = ANY($2::text[])
    ORDER BY id FOR UPDATE NOWAIT`, roomID, seatCodes)

// Double Check trong transaction
var count int
tx.QueryRow(ctx, `SELECT COUNT(*) FROM booking_seats bs
    JOIN bookings b ON b.id = bs.booking_id
    JOIN seats s ON s.id = bs.seat_id
    WHERE b.showtime_id = $1 AND s.seat_number = ANY($2::text[])
      AND (b.status = 'confirmed'
        OR (b.status = 'pending' AND b.created_at > NOW() - INTERVAL '5 minutes'))
`, booking.ShowtimeID, seatCodes).Scan(&count)
if count > 0 { return ErrSeatAlreadyTaken }
return tx.Commit(ctx)
```

8.3. Hệ Thống Real-Time Và Broadcast Trạng Thái Ghế (Websocket)

8.3.1 Lý thuyết WebSocket Protocol

HTTP là giao thức request-response: Client hỏi → Server trả lời. Với yêu cầu "cập nhật trạng thái ghế theo thời gian thực cho tất cả người đang xem cùng suất chiếu", HTTP không đủ – Server không thể chủ động push dữ liệu.

WebSocket (RFC 6455) giải quyết vấn đề này: Bắt đầu bằng HTTP Upgrade handshake, sau đó duy trì kết nối full-duplex liên tục. Cả Client và Server đều có thể gửi bất kỳ lúc nào.

8.3.2 Lý thuyết Fan-Out Pattern

Khi một sự kiện xảy ra (ghế A5 bị lock), cần thông báo tới N clients đang xem cùng showtime. Đây là Fan-Out pattern: Một producer, nhiều consumer.

Go's Hub pattern triển khai Fan-Out qua:

- Single producer: Hàm BroadcastSeatUpdate gửi một message vào hub.broadcast.
- Single consumer goroutine (Hub.Run): Nhận từ channel và gửi đến tất cả client.Send channels.
- N goroutines (WritePumps): Mỗi goroutine đọc từ client.Send và gửi qua WebSocket connection.

Kiến trúc này tránh được việc phải lock khi iterate qua map clients, vì chỉ có một goroutine duy nhất làm việc đó.

8.3.3 Lý thuyết Goroutine per Connection

Go Runtime cực kỳ hiệu quả với goroutine: Một server có thể xử lý hàng chục nghìn WebSocket connections đồng thời, mỗi connection có 2 goroutines (ReadPump + WritePump), tương đương ~80K goroutines với tổng bộ nhớ khoảng 160MB, điều không thể với OS threads.

8.3.4 Use Case người dùng

Thành phần	Cụ thể
Tên Use Case	Cập nhật trạng thái ghế theo thời gian thực cho tất cả người xem cùng suất chiếu
Tác nhân	Người dùng đang xem bản đồ ghế (nhiều người cùng lúc)
Mô tả ngắn gọn	Khi bất kỳ người dùng nào chọn hoặc hoàn tất đặt một ghế, trạng thái ghế đó được cập nhật tức thì trên màn hình của tất cả người dùng khác đang xem cùng suất chiếu, mà không cần họ reload trang.

Điều kiện tiên quyết	Người dùng đang ở trang bản đồ ghế. Kết nối WebSocket đã được thiết lập khi trang tải. Hub đang chạy trên một goroutine nền.
Luồng sự kiện chính	1. Người dùng A và B cùng mở trang đặt vé. Trình duyệt mỗi người tự động kết nối GET /ws. Server nâng cấp HTTP lên WebSocket, đăng ký client vào Hub, khởi động goroutine WritePump và ReadPump. 2. Người dùng A nhấn ghế B3. Backend xử lý LockSeat thành công. 3. bookingService gọi hub.BroadcastSeatUpdate(showtimeID, "B3", "locked"). Message JSON được đẩy vào hub.broadcast channel. 4. Goroutine Hub.Run() nhận message từ channel, iterate qua map clients và gửi vào client.Send channel của từng client (non-blocking). 5. WritePump goroutine của Client B nhận từ client.Send channel và gửi qua WebSocket connection. 6. Trình duyệt của User B nhận JSON, JavaScript cập nhật màu ghế B3 sang vàng ngay lập tức.
Luồng ngoại lệ	Bước 4 (Client lag): Nếu client.Send channel đã đầy (client xử lý quá chậm), Hub thực hiện default case: Đóng channel và xóa client khỏi map – tự động ngắt kết nối client chậm để không block các client khác. Bước 1 (Client đóng tab): ReadPump phát hiện lỗi đọc WebSocket, gửi signal vào hub.unregister, Hub xóa client và đóng client.Send channel. WritePump phát hiện channel đóng và thoát. Không có resource leak.
Hậu điều kiện	Tất cả người dùng đang xem cùng suất chiếu thấy trạng thái ghế chính xác theo thời gian thực mà không cần refresh trang.

8.3.5 Ứng dụng thực tế trong dự án

File: internal/handler/ws_handler.go

```go
type Hub struct {
    clients   map[*Client]bool
    broadcast chan []byte    // Fan-out channel
    register  chan *Client
    mu        sync.Mutex
}

func (h *Hub) Run() {
    for {
        select {
        case client := <-h.register:
            h.mu.Lock(); h.clients[client] = true; h.mu.Unlock()
        case msg := <-h.broadcast:
            h.mu.Lock()
            for client := range h.clients {
                select {
                case client.Send <- msg: // Non-blocking send
                default:                 // Buffer đầy → Disconnect client chậm
                    close(client.Send); delete(h.clients, client)
                }
            }
            h.mu.Unlock()
        }
    }
}

func ServeWs(hub *Hub, w http.ResponseWriter, r *http.Request) {
    conn, _ := upgrader.Upgrade(w, r, nil)
    client := &Client{Hub: hub, Conn: conn, Send: make(chan []byte, 256)}
    hub.register <- client
    go client.WritePump() // 1 goroutine per connection – hiệu quả với Go Runtime
    go client.ReadPump()
}
```

8.4. Kiến Trúc Phần Mềm (Clean Architecture & Depend

ency Injection)

8.4.1 Lý thuyết Clean Architecture

Clean Architecture (Robert C. Martin) định nghĩa các vòng tròn phụ thuộc: Domain → Repository (Interface) → Service → Handler. Mỗi lớp chỉ phụ thuộc vào lớp bên trong, không bao giờ phụ thuộc vào lớp bên ngoài.

Điều quan trọng nhất: Business logic (Service layer) hoàn toàn không biết về Framework, Database, hay HTTP. Điều này cho phép thay thế PostgreSQL bằng MongoDB, thay thế chi-router bằng gin, mà không cần sửa code logic.

8.4.2 Lý thuyết Interface trong Go (Implicit Implementation)

Go không có từ khóa implements. Một type tự động thỏa mãn một interface nếu nó cài đặt tất cả các method của interface đó. Đây là Duck Typing có kiểm tra tại compile time – linh hoạt hơn Java/C# nhưng vẫn type-safe.

Interfaces trong Go thường nhỏ và tập trung (Interface Segregation Principle). HubInterface trong booking service chỉ có một method duy nhất, giúp tránh circular import giữa các packages.

8.4.3 Lý thuyết Dependency Injection (DI) thủ công

DI là kỹ thuật cung cấp phụ thuộc từ bên ngoài thay vì để component tự tạo. Go thường dùng DI thủ công qua constructor functions thay vì DI frameworks.

Lợi ích:

1. Testability: Mocking một interface để unit test không cần DB thực.
2. Flexibility: Thay thế implementation (ví dụ: swap từ Redis sang Memcached) chỉ cần thay đổi ở main.go.
3. Explicit dependencies: Rõ ràng module nào cần gì, không có magic.

8.4.4 Use Case người dùng

Thành phần	**Cụ thể**
Tên Use Case	Request xem lịch sử vé đi qua các layer độc lập
Tác nhân	Người dùng đã đăng nhập
Mô tả ngắn gọn	Khi người dùng xem danh sách vé đã đặt, request đi qua Handler → Service → Repository theo chuỗi phụ thuộc một chiều. Mỗi layer không biết chi tiết triển khai của layer khác – chỉ giao tiếp qua Interface. Điều này cho phép mỗi layer được test độc lập và thay thế mà không ảnh hưởng đến các layer khác.
Điều kiện tiên quyết	Người dùng đã đăng nhập và có JWT hợp lệ. Hệ thống đã khởi động và tất cả dependencies đã được inject trong main.go.
Luồng sự kiện chính	1. Người dùng truy cập trang "Vé của tôi". Trình duyệt gửi GET /api/v1/bookings/my. 2. AuthMiddleware xác thực JWT, inject UserID vào context. 3. BookingHandler.ListMyBookings() đọc UserID từ context, gọi bookingSvc.ListByUserID(ctx, userID, page). Handler không biết gì về Database. 4. BookingService nhận lệnh, áp dụng logic pagination (validate giới hạn, tính offset), gọi repo.ListByUserID(ctx, userID, page). Service không biết gì về SQL. 5. PostgresBookingRepository thực thi SQL query với filter loại bỏ booking pending quá 5 phút. Kết quả ánh xạ vào []domain.Booking. 6. Dữ liệu được trả ngược qua Repository → Service → Handler. Handler serialize thành JSON và trả về response.
Luồng ngoại lệ	**Bước 5 (DB lỗi):** Repository wraps lỗi với context: fmt.Errorf("booking: list by user: %w", err). Handler nhận lỗi, trả về 500 Internal Server Error. **Môi trường Test:** Thay PostgresBookingRepository bằng MockBookingRepository (triển khai cùng interface), test Service Layer không cần kết nối Database thực.
Hậu điều kiện	Người dùng nhận danh sách vé đã lọc (chỉ hiển thị vé còn hạn hoặc đã xác nhận). Mỗi layer hoàn toàn độc lập và có thể được unit test riêng biệt.

8.4.5 Ứng dụng thực tế trong dự án

File: cmd/api/main.go – Dependency Graph và Wiring

```go
// Thứ tự khởi tạo: Infrastructure → Repository → Service → Handler
db, _ := database.Connect(ctx, cfg.Database.DSN())
rdb, _ := redis.NewClient(cfg.Redis)
hub := handler.NewHub(); go hub.Run()

bookingRepo := repository.NewBookingRepository(db) // Trả về interface
bookingSvc  := service.NewBookingService(bookingRepo, rdb, pricingSvc, hub)
bookingHandler := handler.NewBookingHandler(bookingSvc)
```

File: internal/service/booking_service.go – Interface thay vì Concrete Type

```go
type bookingService struct {
    repo  repository.BookingRepository // Interface - không biết là Postgres hay Mock
    wsHub interface{}                  // interface{} để tránh circular import
}

type HubInterface interface {
    BroadcastSeatUpdate(showtimeID, seatID, status string) // Chỉ 1 method cần thiết
}

if hub, ok := s.wsHub.(HubInterface); ok { // Type assertion tại runtime
    hub.BroadcastSeatUpdate(showtimeID, seatID, "locked")
}
```

8.5. Tích Hợp Cổng Thanh Toán & Idempotency (Payment Safety)

8.5.1 Lý thuyết Strategy Pattern cho Payment Providers

Hệ thống cần hỗ trợ nhiều cổng thanh toán (VNPay, MoMo, Tiền mặt) với giao thức khác nhau. Sử dụng Strategy Pattern: Định nghĩa một Interface chung, mỗi provider là một Implementation.

Điều này tuân theo Open/Closed Principle: Open for extension (thêm provider mới), Closed for modification (không sửa code hiện có).

8.5.2 Lý thuyết Idempotency trong Distributed Systems

Idempotency (Tính lũy đẳng): Việc thực thi cùng một operation nhiều lần cho ra kết quả giống như thực thi một lần.

Trong thanh toán, đây là tính chất bắt buộc: Nếu mạng lag, client click 3 lần, chỉ một giao dịch được tạo. Cách triển khai:

1. Client tạo một unique Idempotency-Key cho mỗi yêu cầu.
2. Server kiểm tra trong Redis xem key này đã được xử lý chưa.
3. Nếu đã có, trả về kết quả cũ ngay lập tức.
4. Nếu chưa, xử lý và lưu kết quả vào Redis với TTL.

8.5.3 Lý thuyết SingleFlight Pattern

SingleFlight (golang.org/x/sync/singleflight) giải quyết bài toán cache stampede: Khi cache miss, hàng trăm request đồng loạt cùng query database.

SingleFlight đảm bảo chỉ một request thực sự gọi hàm đắt tiền, các request còn lại chờ và nhận cùng kết quả. Khác với Mutex (chặn tuần tự), SingleFlight cho phép deduplication.

8.5.4 Lý thuyết HMAC-SHA512 cho chữ ký số VNPay

HMAC (Hash-based Message Authentication Code) kết hợp một hash function (SHA-512) với một secret key để tạo chữ ký xác thực. VNPay yêu cầu ký toàn bộ tham số query string theo thứ tự bảng chữ cái, đảm bảo tính toàn vẹn dữ liệu và xác thực nguồn gốc.

8.5.5 Use Case người dùng

Thành phần	Cụ thể
Tên Use Case	Thanh toán đặt vé qua cổng VNPay với bảo vệ chống trùng lặp giao dịch
Tác nhân	Người dùng đã có booking ở trạng thái pending
Mô tả ngắn gọn	Người dùng chọn phương thức VNPay và xác nhận thanh toán. Hệ thống sử dụng Idempotency Key để đảm bảo chỉ một giao dịch duy nhất được tạo tại VNPay, dù người dùng nhấn thanh toán nhiều lần do mạng không ổn định. Chữ ký HMAC-SHA512 đảm bảo tính toàn vẹn dữ liệu trao đổi với VNPay.
Điều kiện tiên quyết	Booking đang ở trạng thái pending và chưa quá 5 phút. Cấu hình VNPay (TmnCode, HashSecret, PayURL, ReturnURL) đã được thiết lập. Redis đang hoạt động (bắt buộc cho Idempotency).
Luồng sự kiện chính	1. Người dùng chọn "Thanh toán qua VNPay" và nhấn xác nhận. Frontend tạo Idempotency-Key = "pay-{bookingId}-{timestamp}" và gửi POST /payment kèm key trong Header. 2. Handler đọc Idempotency-Key, kiểm tra trong Redis – chưa có (cache miss). 3. paymentService acquire Distributed Lock trên key đó (Redis.SetNX) để tránh 2 request cùng key xử lý đồng thời. 4. Service đọc VNPayProvider từ map providers["VNPAY"] theo Strategy Pattern. 5. VNPayProvider.CreatePayment() sort tham số theo alphabet, tính HMAC-SHA512 với HashSecret, tạo URL redirect tới cổng VNPay. 6. Kết quả lưu vào Database và cache vào Redis với TTL 24 giờ. Lock được giải phóng qua defer. 7. Frontend nhận redirect URL và chuyển người dùng sang trang VNPay để nhập OTP.

Luồng ngoại lệ	Bước 1 (Mạng lag, nhấn 3 lần): Request 2 và 3 mang cùng Idempotency-Key. Bước 2 tìm thấy key trong Redis (cache hit), trả về kết quả đã lưu ngay lập tức – không gọi VNPay thêm lần nào. Bước 3 (2 request đến đồng thời): Request 2 không acquire được lock (SetNX trả về false), sleep 100ms và thử checkIdempotency lần nữa – lúc này request 1 đã xử lý xong và lưu cache. Bước 1 (Booking hết hạn): time.Now().After(booking.CreatedAt.Add(5 * time.Minute)) – trả về lỗi "Booking đã hết hạn".
Hậu điều kiện	Đúng một bản ghi Payment được tạo trong Database với status = "pending". Người dùng được redirect sang VNPay. Nếu paymentMethod là Cash/MoMo, MockProvider tự set status = "paid" ngay.

8.5.6 Ứng dụng thực tế trong dự án

File: internal/service/payment_service_imp.go

```go
type PaymentProvider interface { // Strategy Interface
    GetName() string
    IsActive() bool
    CreatePayment(ctx context.Context, booking *domain.Booking, ...) (*domain.Payment, string, error)
    HandleWebhook(ctx context.Context, ...) error
}

func (s *paymentService) CreatePayment(ctx context.Context, ..., idempotencyKey string, ...) {
    if payment, url, _ := s.checkIdempotency(ctx, idempotencyKey); payment != nil {
        return payment, url, nil // Cache hit - trả ngay
    }
    locked, _ := s.redis.SetNX(ctx, "lock:"+idempotencyKey, "1", 30*time.Second).Result()
    if !locked { time.Sleep(100*time.Millisecond); return s.checkIdempotency(ctx, idempotencyKey) }
    defer s.redis.Del(ctx, "lock:"+idempotencyKey) // Auto release

    s.mu.RLock() // RWMutex: Đọc đồng thời an toàn
    provider := s.providers[strings.ToUpper(paymentMethod)]
    s.mu.RUnlock()
    payment, redirectURL, _ := provider.CreatePayment(ctx, booking, ...)
    s.paymentRepo.Create(ctx, payment)
}

// SingleFlight: 100 goroutines gọi → chỉ 1 goroutine query DB
func (s *paymentService) GetPaymentMethods(ctx context.Context) ([]domain.PaymentMethod, error) {
    v, _, _ := s.requestGroup.Do("payment_methods", func() (interface{}, error) {
        // Hàm này chỉ chạy 1 lần dù có 100 goroutine gọi đồng thời
        if cached, _ := s.redis.Get(ctx, "payment_methods").Bytes(); len(cached) > 0 {
            var methods []domain.PaymentMethod
            json.Unmarshal(cached, &methods); return methods, nil
        }
        // Query DB, cache kết quả...
    })
    return v.([]domain.PaymentMethod), nil
```

8.6. Xử Lý Webhook Bất Đồng Bộ (Async Processing)

8.6.1 Lý thuyết Fire-and-Forget Pattern với Goroutine

Webhook từ VNPay bắt buộc phải nhận được response 200 OK ngay lập tức (trong vài giây). Nếu server xử lý chậm (query DB, update trạng thái), VNPay sẽ retry và tạo ra duplicate xử lý.
Giải pháp: Fire-and-Forget – tách logic xử lý ra một goroutine riêng, trả về 200 OK ngay. Goroutine sẽ hoàn tất xử lý trong nền.
Rủi ro: Nếu server crash sau khi trả 200 nhưng trước khi goroutine hoàn tất → Cần Idempotency để VNPay retry an toàn.

8.6.2 Use Case người dùng

Thành phần	**Cụ thể**
Tên Use Case	Nhận và xử lý webhook xác nhận thanh toán từ VNPay
Tác nhân	VNPay Server (hệ thống bên thứ ba)
Mô tả ngắn gọn	Sau khi người dùng hoàn tất thanh toán trên trang VNPay, VNPay gửi POST request (webhook) về hệ thống để xác nhận kết quả giao dịch. Hệ thống phải phản hồi 200 OK ngay lập tức và xử lý logic cập nhật Database trong nền để đáp ứng yêu cầu thời gian phản hồi của VNPay.
Điều kiện tiên quyết	Người dùng đã hoàn tất thanh toán trên trang VNPay. Endpoint webhook đã được đăng ký với VNPay và có thể truy cập công khai.
Luồng sự kiện chính	1. Người dùng hoàn tất thanh toán OTP trên VNPay. VNPay gửi POST /api/v1/payment/webhook/vnpay kèm thông tin giao dịch và chữ ký HMAC. 2. Handler đọc toàn bộ request body vào []byte. 3. Handler spawn một goroutine mới (Fire-and-Forget) để xử lý trong nền và ngay lập tức trả về 200 OK cho VNPay.

	4. VNPay nhận 200 OK, đánh dấu webhook là đã giao thành công – không retry. 5. Goroutine nền: Verify chữ ký HMAC-SHA512 với HashSecret. Kiểm tra vnp_ResponseCode == "00" (thành công). Gọi bookingRepo.UpdateStatus(ctx, bookingID, "confirmed"). Ghi audit log kết quả.
Luồng ngoại lệ	Bước 5 (Chữ ký không hợp lệ): Goroutine log lỗi "Webhook: invalid HMAC signature" và dừng – không cập nhật Database. Bước 5 (Server crash sau bước 3): Goroutine chưa kịp cập nhật DB. VNPay retry sau 5 phút. Lần retry: Idempotency check phát hiện booking đã confirmed hoặc webhook đã được xử lý – skip an toàn. Bước 3 (context.Background() thay r.Context()): Nếu dùng r.Context(), context bị cancel ngay khi response trả về, goroutine bị ngắt giữa chừng trước khi update DB.
Hậu điều kiện	VNPay không retry (đã nhận 200 OK). Booking được cập nhật thành confirmed trong Database. Người dùng nhận thấy vé đã được xác nhận khi truy cập "Vé của tôi".

8.6.3 Ứng dụng thực tế trong dự án

File: internal/handler/payment_handler.go

```go
func (h *PaymentHandler) HandlePaymentWebhook(w http.ResponseWriter, r *http.Request) {
    body, _ := io.ReadAll(r.Body)
    defer r.Body.Close()

    provider := chi.URLParam(r, "provider")

    go func() {
        // PHẢI dùng context.Background() - không dùng r.Context()
        // r.Context() bị cancel ngay khi HTTP response trả về
        ctx := context.Background()
        if err := h.svc.HandlePaymentWebhook(ctx, provider, r.Header, body, r.URL.Query()); err != nil {
            log.Printf("Webhook processing error for %s: %v", provider, err)
        }
    }() // Fire-and-Forget

    // Trả 200 OK ngay - VNPay không retry
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "received"})
}
```

8.7. Domain Modeling Và Struct Tags

8.7.1 Lý thuyết Struct Tags trong Go

Struct Tags là metadata gắn vào từng field của struct, được đọc bởi các thư viện qua reflection tại runtime. Go không có annotation như Java, nhưng struct tags cung cấp chức năng tương tự theo cách explicit hơn.

Các tag phổ biến:

- json:"name": Điều khiển JSON serialization/deserialization.
- db:"col_name": Ánh xạ column trong database (sqlx).
- json:"-": Không serialze field này ra JSON.
- json:"omitempty": Bỏ qua field trong JSON nếu giá trị là zero value.

8.7.2 Lý thuyết Pointer Semantics trong Domain Models

Go phân biệt rõ value types và pointer types. Sử dụng *string hoặc *time.Time cho phép phân biệt "không có giá trị (nil)" với "có giá trị là chuỗi rỗng" – quan trọng cho các field optional trong database (nullable columns).

8.7.3 Use Case người dùng

Thành phần	**Cụ thể**
Tên Use Case	Đăng ký tài khoản với thông tin tùy chọn và bảo mật mật khẩu
Tác nhân	Người dùng mới (chưa có tài khoản)
Mô tả ngắn gọn	Người dùng điền form đăng ký. Số điện thoại là tùy chọn (có thể bỏ trống). Hệ thống đảm bảo mật khẩu hash không bao giờ lộ ra ngoài qua API response nhờ struct tags. Các field tùy chọn được ánh xạ chính xác với NULL trong Database.
Điều kiện tiên quyết	Email người dùng nhập chưa tồn tại trong hệ thống.

Luồng sự kiện chính	1. Người dùng nhập username, email, password. Bỏ trống trường phone. Frontend gửi JSON {"username":"...", "email":"...", "phone": null}. 2. Go json.Unmarshal vào struct User: Phone *string được set nil (phân biệt rõ với "" – người dùng không nhập). OTPCode *string là nil. 3. Service hash password bằng bcrypt, lưu vào PasswordHash string. 4. Repository INSERT vào PostgreSQL – cột phone nhận NULL (nil pointer ánh xạ chính xác). 5. Service gửi OTP qua email để xác thực. 6. Handler serialize User thành JSON để trả response: json:"-" trên PasswordHash → trường này không xuất hiện trong JSON. json:"phone" với giá trị nil → xuất hiện như "phone": null hoặc không có nếu có omitempty.
Luồng ngoại lệ	Bước 2 (Email đã tồn tại): Repository INSERT vi phạm unique constraint, trả về lỗi PostgreSQL. Service wrap lỗi, Handler trả về 409 Conflict: "Email đã được sử dụng".
Hậu điều kiện	Tài khoản được tạo với is_verified = false. Mật khẩu hash không bị lộ trong bất kỳ API response nào. Trường phone = NULL trong Database thay vì chuỗi rỗng.

8.7.4 Ứng dụng thực tế trong dự án

File: internal/domain/models.go – Struct Tags và Pointer Semantics

```go
type User struct {
    ID           string     `json:"id"`
    Username     string     `json:"username"`
    Email        string     `json:"email"`
    PasswordHash string     `json:"-"`            // KHÔNG bao giờ serialize ra JSON
    Phone        *string    `json:"phone"`         // nil = NULL, "" = chuỗi rỗng
    OTPCode      *string    `json:"-"`             // Nullable + ẩn khỏi JSON
    OTPExpiry    *time.Time `json:"-"`             // Nullable time + ẩn
    IsVerified   bool       `json:"is_verified"`
    CreatedAt    time.Time  `json:"created_at"`
}

type Booking struct {
    PromotionID  *string    `json:"promotion_id,omitempty"` // nil → không xuất hiện trong JSON
    Seats        []string   `json:"seats,omitempty"`        // nil/empty → không xuất hiện
    CancelledAt  *time.Time `json:"cancelled_at,omitempty"`
}

// Value receiver: Go convention cho pure computation – không modify receiver
func (p Page) Offset() int {
    if p.Page < 1 { p.Page = 1 }
    return (p.Page - 1) * p.Limit
}
```

8.8. Graceful Shutdown Và Quản Lý Vòng Đời Ứng Dụng

8.8.1 Lý thuyết OS Signals và Context Cancellation

Linux/Windows gửi tín hiệu (signal) cho process khi muốn dừng nó:

- SIGINT (Ctrl+C): Interrupt signal.
- SIGTERM: Graceful termination (Docker stop, Kubernetes pod eviction).

Khi nhận SIGTERM, một server "thô" sẽ dừng ngay, làm đứt các request đang xử lý, dẫn đến mất dữ liệu hoặc incomplete transaction.

Graceful Shutdown: Server ngừng nhận request mới, chờ các request hiện tại hoàn tất trong một khoảng timeout, sau đó đóng kết nối.

context.WithTimeout tạo một Context với deadline, tự động cancel sau timeout. http.Server.Shutdown(ctx) dừng server gracefully, block cho đến khi tất cả connections đóng hoặc ctx bị cancel.

8.8.2 Lý thuyết defer trong Go

Defer đảm bảo một lệnh được thực thi khi hàm chứa nó kết thúc, bất kể kết thúc bình thường hay do panic. Đây là idiom "cleanup" phổ biến trong Go:

```
f := open("file.txt")
defer f.Close()        // Đảm bảo file được đóng dù hàm return ở đâu
```

defer theo LIFO order (Last In, First Out) – hữu ích khi cần cleanup nhiều tài nguyên theo thứ tự ngược lại với khởi tạo.

8.8.3 Use Case người dùng

Thành phần	Cụ thể
Tên Use Case	Triển khai phiên bản mới mà không làm gián đoạn người dùng đang sử dụng
Tác nhân	DevOps / Kubernetes (hệ thống triển khai)
Mô tả ngắn gọn	Trong quy trình rolling update của Kubernetes, container đang chạy nhận tín hiệu SIGTERM để dừng gracefully. Hệ thống Go đảm bảo tất cả request đang xử lý được hoàn tất, các kết nối Database và Redis được đóng đúng thứ tự, trước khi process thoát. Người dùng đang sử dụng ứng dụng không cảm nhận bất kỳ gián đoạn nào.
Điều kiện tiên quyết	Phiên bản mới đã được build và push lên registry. Kubernetes đang thực hiện rolling update – khởi động pod mới trước khi dừng pod cũ.
Luồng sự kiện chính	1. Kubernetes gửi SIGTERM cho pod đang chạy. 2. Go Runtime giao signal tới quit channel. Main goroutine (đang block tại <-quit) nhận signal và tiếp tục chạy. 3. srv.Shutdown(shutdownCtx) được gọi với context timeout 15 giây: Server từ chối tất cả connection mới, không ngắt các connection đang active.

	4. Trong 15 giây: Tất cả goroutines đang xử lý HTTP request hoàn tất việc của mình và trả response về client. Booking transactions commit thành công. Payment requests hoàn tất. 5. srv.Shutdown() trả về khi tất cả active connections đã đóng. 6. defer trong main() thực thi theo LIFO: logger.Sync() flush buffer log → rdb.Close() đóng Redis pool → db.Close() đóng DB connection pool. 7. Process thoát với exit code 0. Kubernetes ghi nhận pod đã dừng gracefully và cân nhắc tiếp tục rolling update.
Luồng ngoại lệ	Bước 4 (Hết 15 giây timeout): shutdownCtx bị cancel, srv.Shutdown() trả về dù còn request chưa xong. Các request đó bị ngắt (connection reset). Process vẫn thoát để không block deployment. Bước 2 (SIGINT từ Ctrl+C trong development): Xử lý giống hệt – developer có thể dừng server an toàn trong khi đang test mà không mất dữ liệu.
Hậu điều kiện	Process đã thoát sạch. Không có transaction nào bị incomplete. Tất cả connections DB/Redis đã được đóng đúng cách. Người dùng đang dùng ứng dụng không gặp lỗi nào trong suốt quá trình này.

8.8.4 Ứng dụng thực tế trong dự án

```go
go func() { // Server chạy goroutine riêng để không block main
    if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        logger.Fatal("server error", zap.Error(err))
    }
}()

quit := make(chan os.Signal, 1) // Buffered 1: Không miss signal
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit // Main goroutine block ở đây

shutdownCtx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
defer cancel()
srv.Shutdown(shutdownCtx)
// Sau Shutdown, defer trong main chạy LIFO:
// defer logger.Sync() → defer rdb.Close() → defer db.Close()
```

8.9. CẤU TRÚC LỖI VÀ STATE MACHINE TRẠNG THÁI ĐẶT VÉ

8.9.1 Lý thuyết Error Handling trong Go

Go không có exceptions. Lỗi là giá trị trả về thông thường (value type). Convention: Hàm trả về (result, error) – caller bắt buộc xử lý lỗi明確.

errors.New() tạo error với message cố định. Để so sánh lỗi chính xác (không phải so sánh string), dùng sentinel errors (biến global). Tại Handler, dùng errors.Is() để phân loại lỗi và quyết định HTTP status code phù hợp.

Error Wrapping (fmt.Errorf("context: %w", err)): Bọc lỗi gốc với thêm context. %w cho phép errors.Is() và errors.As() unwrap qua nhiều layer.

8.9.2 Lý thuyết State Machine cho Booking Status

Trạng thái đặt vé là một Finite State Machine (FSM):

[pending] ──thanh toán OK──→ [confirmed]

[pending] ──5 phút qua──────→ [expired/deleted]

[pending] ──hủy bởi user────→ [cancelled]

[confirmed] ──không đổi─────→ [confirmed]

State transitions phải được validate nghiêm ngặt để tránh inconsistent state (ví dụ: không thể từ confirmed quay về pending).

8.9.3 Use Case người dùng

Thành phần	Cụ thể
Tên Use Case	Đặt vé cho suất chiếu đã bắt đầu và xử lý vé hết hạn giữ chỗ
Tác nhân	Người dùng đã đăng nhập
Mô tả ngắn gọn	Hệ thống sử dụng Sentinel Errors và kiểm tra thời gian rõ ràng để từ chối đặt vé cho các suất chiếu đã bắt đầu. Đồng thời, State Machine đảm bảo booking pending quá 5 phút tự động không hiển thị trong danh sách vé, giải phóng ghế cho người khác.
Điều kiện tiên quyết	Người dùng đã đăng nhập. Suất chiếu tồn tại trong Database.
Luồng sự kiện chính (UC-A: Vé hết hạn)	1. Người dùng đặt vé lúc 14:00, tạo booking pending. Đồng hồ đếm ngược 5 phút bắt đầu trên frontend. 2. Người dùng bị phân tâm, không thanh toán. Đồng hồ về 0:00 lúc 14:05, frontend hiển thị thông báo "Hết thời gian giữ chỗ" và redirect về trang chủ. 3. Lúc 14:10, người dùng vào "Vé của tôi". 4. Repository thực thi query với filter: WHERE (status != 'pending' OR created_at > NOW() - INTERVAL '5 minutes'). Booking 14:00 với status pending không thỏa điều kiện (đã qua 10 phút). 5. Danh sách trả về không có booking đó. Người dùng thấy thông báo "Bạn chưa có vé nào". Ghế A5 đã trống cho người khác đặt.
Luồng sự kiện chính (UC-B: Đặt vé suất đã bắt đầu)	1. Người dùng mở trang đặt vé lúc 09:55 (suất 10:00 chưa bắt đầu). 2. Suy nghĩ 10 phút, nhấn "Xác nhận đặt vé" lúc 10:05. 3. Repository truy vấn start_time từ DB. Go kiểm tra: startTime.Before(time.Now()) → 10:00.Before(10:05) → true. Trả về ErrShowtimeExpired. 4. Handler kiểm tra errors.Is(err, ErrShowtimeExpired), trả về 400 Bad Request với message rõ ràng.
Luồng ngoại lệ	UC-A, Bước 2 (User vẫn ở trang thanh toán): Đồng hồ đếm ngược được hệ thống tính từ booking.created_at (lưu trong DB) – không bị

	reset khi refresh trang. Khi về 0, frontend tự redirect mà không cần gọi thêm API. UC-B (Frontend cache suất chiếu cũ): Frontend hiển thị "chưa bắt đầu" nhưng thực tế đã bắt đầu. Backend là source of truth – từ chối tại tầng Repository với ErrShowtimeExpired.
Hậu điều kiện	UC-A: Booking pending quá hạn không xuất hiện trong danh sách. Ghế được giải phóng tự động (không cần cronjob). UC-B: Không có booking nào được tạo. Người dùng thấy thông báo lỗi rõ ràng và có thể chọn suất chiếu khác.

8.9.4 Ứng dụng thực tế trong dự án

File: internal/repository/errors.go

```go
var (
    ErrSeatAlreadyTaken = errors.New("ghế đã được bán hoặc đang chờ thanh toán")
    ErrShowtimeExpired  = errors.New("suất chiếu đã bắt đầu hoặc đã qua thời gian")
    ErrBookingNotFound  = errors.New("booking không tìm thấy")
    ErrSeatLockConflict = errors.New("ghế đang được giữ bởi người khác")
)
```

File: internal/repository/postgres_booking_repository.go

```go
// time.Before(): Rõ ràng, không magic - đọc như ngôn ngữ tự nhiên
if startTime.Before(time.Now()) {
    return ErrShowtimeExpired // Sentinel error: Handler compare bằng errors.Is()
}

// Error wrapping: Thêm context, vẫn unwrap được lỗi gốc
return nil, fmt.Errorf("booking: list by user: %w", err)

// State filter tại DB: Tận dụng index, tránh load dữ liệu thừa về Go
WHERE (status != 'pending' OR created_at > NOW() - INTERVAL '5 minutes')
```

CHƯƠNG 9: KẾT QUẢ ĐẠT ĐƯỢC

9.1 Mức Độ Hoàn Thành Các Chức Năng

Nhìn lại toàn bộ quá trình phát triển, nhóm đã hoàn thành dự án ở mức độ đáp ứng đầy đủ các yêu cầu chức năng cốt lõi ban đầu. Dưới đây là bảng đánh giá chi tiết từng module:

Module	Chức Năng Đã Hoàn Thành	Mức Độ Hoàn Thành	Ghi Chú
Quản lý người dùng	Đăng ký, đăng nhập, phân quyền (user/admin), JWT authentication	100%	Middleware xác thực hoạt động ổn định
Quản lý phim	CRUD đầy đủ, tìm kiếm theo tên/thể loại, lọc theo trạng thái	100%	Admin quản lý, user chỉ xem
Quản lý rạp & phòng	Thêm/sửa rạp, tạo phòng chiếu, quản lý sơ đồ ghế	100%	Hỗ trợ nhiều loại ghế (thường, VIP)
Quản lý suất chiếu	Tạo/xóa lịch chiếu, kiểm tra trùng lịch phòng	100%	Validation tránh trùng suất
Đặt vé & Thanh toán	Chọn ghế, tạo booking, thanh toán, xem lịch sử vé	95%	Chỉ hỗ trợ thanh toán nội bộ, chưa tích hợp cổng thanh toán thực tế
Xử lý đồng thời	Ngăn 2 người cùng đặt 1 ghế (Race Condition)	100%	Sử dụng DB Transaction kết hợp SELECT FOR UPDATE
Quản lý trạng thái ghế	Cập nhật trạng thái ghế trống/đã đặt sau mỗi giao dịch	100%	Đồng bộ ngay khi booking chuyển sang trạng thái confirmed
Tổng thể	Toàn bộ chức năng cốt lõi theo thiết kế ban đầu	~98%	Phần còn lại là tích hợp cổng thanh toán thực tế, xác định là hướng phát triển tương lai

9.2. Hệ Thống Đáp Ứng Kiến Trúc Clean MVC

Dự án được xây dựng theo kiến trúc MVC mở rộng (Clean MVC), trong đó tầng Controller được tách thêm thành hai lớp riêng biệt là Service và Repository, nhằm đảm bảo mỗi tầng chỉ chịu trách nhiệm duy nhất một nhiệm vụ. Kiến trúc này tuân thủ nguyên tắc Separation of Concerns - tách biệt rõ ràng giữa logic dữ liệu, logic nghiệp vụ và logic trình bày.

Toàn bộ luồng xử lý một request đi qua các tầng theo thứ tự sau:

```
HTTP Request
  |
  v
Router (main.go / router.go)
  |       Phân loại request, gắn middleware xác thực JWT
  v
Handler (internal/handlers/)        <-- Tầng Controller
  |       Nhận request, validate đầu vào, gọi Service
  v
Service (internal/services/)        <-- Tầng nghiệp vụ
  |       Xử lý logic, điều phối giữa các Repository
  v
Repository (internal/repositories/)  <-- Tầng truy xuất dữ liệu
  |       Viết câu truy vấn SQL, tương tác với DB
  v
Model (internal/models/)            <-- Tầng cấu trúc dữ liệu
  |       Định nghĩa struct ánh xạ với bảng PostgreSQL
  v
PostgreSQL Database
  |
  v
JSON Response trả về client         <-- Tầng View
```

Tầng Model - Cấu trúc dữ liệu:

Toàn bộ struct định nghĩa thực thể (User, Movie, Showtime, Booking, Seat...) nằm trong thư mục internal/models/. Mỗi model ánh xạ trực tiếp với một bảng trong PostgreSQL thông qua GORM tag, đồng thời khai báo các ràng buộc validation. Tầng này không chứa bất kỳ logic nghiệp vụ nào - chỉ mô tả thuần túy cấu trúc và quan hệ giữa các thực thể dữ liệu.

Tầng Repository - Truy xuất dữ liệu:

Nằm trong internal/repositories/, đây là tầng duy nhất được phép viết câu truy vấn SQL hoặc gọi GORM để tương tác với PostgreSQL. Mỗi entity có một Repository riêng (MovieRepository, BookingRepository, SeatRepository...). Tầng này không biết gì về logic nghiệp vụ - chỉ thực hiện các thao tác CRUD và trả dữ liệu thô về cho Service.

Tầng Service - Xử lý nghiệp vụ:

Nằm trong internal/services/, đây là nơi tập trung toàn bộ logic nghiệp vụ phức tạp của hệ thống. Ví dụ, luồng đặt vé bao gồm: kiểm tra ghế còn trống, khóa ghế bằng transaction, tính tổng tiền, tạo bản ghi booking và cập nhật trạng thái - toàn bộ các bước này chỉ được viết tại tầng Service. Tầng này không biết gì về HTTP request hay JSON response, chỉ nhận dữ liệu thuần từ Handler và trả kết quả về.

Tầng Handler (Controller) - Điều phối request:

Nằm trong internal/handlers/, Handler đóng vai trò cầu nối giữa HTTP và logic ứng dụng. Nhiệm vụ của Handler bao gồm: đọc dữ liệu từ request (path param, query param, body), gọi Service tương ứng, rồi serialize kết quả trả về dạng JSON. Handler không chứa logic nghiệp vụ - nếu Handler cần tự xử lý logic thay vì gọi Service, đó là dấu hiệu kiến trúc đang bị vi phạm.

Tầng View - JSON Response:

Vì đây là REST API thuần túy, không có giao diện HTML, tầng View tương đương với cấu trúc JSON trả về cho client. Nhóm định nghĩa hai struct chuẩn hóa dùng thống nhất toàn dự án:

```
go
// Phản hồi thành công
type SuccessResponse struct {
```

```
    Data    interface{} `json:"data"`
    Message string      `json:"message"`
}

// Phản hồi lỗi
type ErrorResponse struct {
    Error string `json:"error"`
    Code  int    `json:"code"`
}
```

Việc chuẩn hóa response giúp Frontend luôn biết chính xác cấu trúc dữ liệu nhận về, bất kể đang gọi endpoint nào.

Bảng dưới đây tóm tắt trách nhiệm của từng tầng và quy tắc phân tầng nhóm tuân thủ:

Tầng	**Thư Mục**	**Trách Nhiệm**	**Được Phép**	**Không Được Phép**
Model	internal/models/	Định nghĩa cấu trúc dữ liệu, ánh xạ DB	Khai báo struct, validation tag, quan hệ bảng	Viết logic, gọi DB trực tiếp
Repository	internal/repositories/	Truy xuất và lưu trữ dữ liệu	Viết SQL, dùng GORM, truy vấn DB	Xử lý logic nghiệp vụ, biết về HTTP
Service	internal/services/	Xử lý toàn bộ logic nghiệp vụ	Gọi Repository, xử lý transaction, tính toán	Biết về HTTP request, viết SQL trực tiếp
Handler	internal/handlers/	Điều phối request và response	Đọc request, gọi Service, trả JSON	Viết logic nghiệp vụ, gọi Repository trực tiếp

Tầng	Thư Mục	Trách Nhiệm	Được Phép	Không Được Phép
View	(JSON Response)	Định nghĩa cấu trúc phản hồi	Chuẩn hóa SuccessResponse, ErrorResponse	Chứa logic xử lý dữ liệu

Kết quả vận hành thực tế:

Nhờ kiến trúc phân tầng rõ ràng, khi một lỗi phát sinh trong quá trình phát triển, nhóm xác định ngay được tầng cần kiểm tra mà không phải dò toàn bộ codebase. Lỗi dữ liệu sai cấu trúc tìm trong Model, lỗi câu truy vấn SQL tìm trong Repository, lỗi tính toán nghiệp vụ tìm trong Service, lỗi đọc sai tham số request tìm trong Handler. Server Go khởi động trong dưới 1 giây, xử lý đồng thời nhiều request nhờ cơ chế goroutine tự nhiên của ngôn ngữ, kết nối PostgreSQL qua connection pool với tối đa 25 connection đồng thời, không ghi nhận hiện tượng crash hay memory leak trong suốt quá trình kiểm thử.

9.3. Đánh Giá Quá Trình Làm Việc Nhóm

9.3.1. Quản Lý Source Code Với Git

Nhóm sử dụng GitHub để quản lý toàn bộ source code với quy trình làm việc có tổ chức. Dưới đây là tóm tắt từ Git History và Insights của repository:

Mẫu commit message nhóm sử dụng (theo chuẩn Conventional Commits):

feat(booking): thêm logic kiểm tra ghế trống trước khi đặt
fix(auth): sửa lỗi token không được refresh đúng cách
refactor(movie): tách repository layer khỏi handler

docs(api): cập nhật swagger annotation cho /showtimes
test(booking): thêm test case cho trường hợp ghế đã bị đặt
chore: cập nhật .env.example và README hướng dẫn cài đặt

Quy ước commit message nhất quán giúp nhìn vào lịch sử Git là hiểu ngay ai làm gì, phần nào của hệ thống thay đổi.

9.3.2. Mô Hình Phân Nhánh (Git Branching Model)

*Nhóm áp dụng mô hình **GitHub Flow** - đơn giản hơn Git Flow nhưng phù hợp với nhóm nhỏ và deadline học thuật:*

main (*nhánh chính - luôn ở trạng thái có thể chạy được*)
├── *develop* (*nhánh phát triển tích hợp*)
│ ├── *feature/auth-jwt* ← *Thành viên A làm module xác thực*
│ ├── *feature/movie-crud* ← *Thành viên B làm quản lý phim*
│ ├── *feature/booking-logic* ← *Thành viên C làm đặt vé*
│ ├── *feature/seat-management* ← *Thành viên A làm quản lý ghế*
│ └── *fix/race-condition-booking* ← *Sửa lỗi cụ thể*
└── (*merge vào main sau khi* test *xong trên develop*)

Quy tắc làm việc với nhánh:

- Mỗi tính năng mới hoặc bug fix tạo một nhánh riêng từ *develop*, đặt tên theo format *feature/ten-tinh-nang* hoặc *fix/mo-ta-loi*.
- Khi hoàn thành, mở **Pull Request** (PR) vào *develop*, ít nhất 1 thành viên khác review code trước khi merge.
- Nhánh *main* chỉ được cập nhật khi *develop* đã qua kiểm thử đầy đủ - đảm bảo *main* luôn là phiên bản ổn định có thể demo được bất cứ lúc nào.
- Xóa nhánh feature sau khi merge để repository sạch sẽ.

9.3.3. Cách Giải Quyết Conflict Khi Merge Code

Conflict (xung đột) xảy ra khi 2 người cùng sửa một file ở cùng vị trí. Nhóm đã gặp và xử lý theo quy trình sau:

Conflict thường gặp nhất**:** File *internal/router/router.go* - nơi đăng ký tất cả routes. Khi nhiều người cùng thêm route mới, Git không biết giữ dòng nào.

Quy trình xử lý conflict:

Bước 1 - Phòng tránh bằng giao tiếp: Trước khi bắt đầu làm việc mỗi ngày, nhóm thông báo trong nhóm chat xem ai đang làm file nào để tránh làm trùng.

Bước 2 - Cập nhật code mới nhất trước khi làm: Trước khi code, luôn chạy *git pull origin develop* để lấy code mới nhất, giảm thiểu độ lệch với nhánh chính.

Bước 3 - Khi conflict xảy ra, giải quyết thủ công có chủ đích:

```
# Khi merge PR và có conflict
git checkout feature/my-branch
git rebase origin/develop   # Rebase thay vì merge để lịch sử gọn hơn

# Git báo conflict ở router.go
# Mở file, tìm đoạn <<< HEAD ... === ... >>> origin/develop
# Giữ LẠI cả hai phần (cả route của mình và route của đồng đội)
# Xóa các marker <<<, ===, >>>
# Test lại, rồi commit
git add .
git rebase --continue
```

Bước 4 - Họp nhanh khi conflict phức tạp: Nếu conflict ở logic nghiệp vụ (không chỉ là thêm dòng), hai người liên quan sẽ ngồi lại cùng xem và quyết định giữ code nào, đảm bảo không mất logic quan trọng.

9.4. Khó Khăn Và Cách Khắc Phục

9.4.1. Khó Khăn 1 - Kết Nối Go Với PostgreSQL

Vấn đề gặp phải: Ban đầu nhóm gặp lỗi *pq: SSL is not enabled on the server* khi kết nối từ Go đến PostgreSQL chạy local. Ngoài ra, khi load test với nhiều request đồng thời, ứng dụng bị lỗi *too many connections* vì mỗi request mở một connection mới mà không đóng lại.

Nguyên nhân:

- PostgreSQL local mặc định không bật SSL, nhưng driver Go lại yêu cầu SSL theo mặc định.
- Không cấu hình connection pool, dẫn đến số lượng connection tăng không kiểm soát.

Cách khắc phục:

```
// Thêm ?sslmode=disable vào connection string
dsn := "host=localhost user=postgres password=123456
        dbname=cinema_db port=5432 sslmode=disable"

db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})

// Cấu hình connection pool
sqlDB, _ := db.DB()
sqlDB.SetMaxOpenConns(25)       // Tối đa 25 connection mở đồng thời
sqlDB.SetMaxIdleConns(10)       // Giữ sẵn 10 connection ở trạng thái idle
sqlDB.SetConnMaxLifetime(5 * time.Minute)  // Đóng connection sau 5 phút
```

Sau khi áp dụng, lỗi kết nối biến mất và ứng dụng xử lý được nhiều request đồng thời mà không bị quá tải.

9.4.2. Khó Khăn 2 - Xử Lý Transaction Đặt Vé (Race Condition)

Đây là vấn đề phức tạp nhất của dự án. Khi 2 người dùng cùng lúc chọn cùng một ghế và nhấn "Đặt vé" trong cùng một mili-giây, hệ thống không được để cả hai đặt thành công. Tuy nhiên, nếu xử lý không khéo, có thể xảy ra tình trạng **một người bị mất tiền nhưng không có vé** (đã thanh toán nhưng booking bị hủy sau đó).

Minh họa vấn đề (nếu không có transaction):
Thời điểm T1: User A kiểm tra ghế 12 → kết quả: "available"
Thời điểm T1: User B kiểm tra ghế 12 → kết quả: "available" (đọc cùng lúc)
Thời điểm T2: User A tạo booking cho ghế 12 → thành công
Thời điểm T2: User B tạo booking cho ghế 12 → cũng thành công (?!)
→ Kết quả: 2 người cùng giữ 1 ghế - dữ liệu bị sai

Cách khắc phục - Sử dụng Database Transaction với SELECT FOR UPDATE:

```
func (s *BookingService) CreateBooking(userID, showtimeID int, seatIDs []int)
(*Booking, error) {

    // Bắt đầu transaction - tất cả hoặc không có gì
    tx := s.db.Begin()
    if tx.Error != nil {
        return nil, tx.Error
    }

    // SELECT FOR UPDATE: Lock dòng ghế lại,
    // ngăn không cho transaction khác đọc/sửa cho đến khi transaction này kết thúc
    var seats []Seat
    result := tx.Set("gorm:query_option", "FOR UPDATE").
                Where("id IN ? AND showtime_id = ? AND status = 'available'",
                    seatIDs, showtimeID).
                Find(&seats)
```

```
    // Nếu số ghế available ít hơn số ghế yêu cầu → có ghế đã bị đặt
    if len(seats) < len(seatIDs) {
        tx.Rollback() // Hủy toàn bộ, không thay đổi gì
        return nil, errors.New("Một hoặc nhiều ghế đã được đặt bởi người khác")
    }

    // Cập nhật trạng thái ghế → booked
    tx.Model(&Seat{}).Where("id IN ?", seatIDs).Update("status", "booked")

    // Tạo bản ghi booking
    booking := &Booking{ UserID: userID, ShowtimeID: showtimeID, Status:
"confirmed" }
    tx.Create(booking)
    // Commit - lưu tất cả thay đổi vào DB cùng lúc
    if err := tx.Commit().Error; err != nil {
        tx.Rollback()
        return nil, err
    }

    return booking, nil
}
```

Kết quả: Với cơ chế SELECT FOR UPDATE, khi User A đang xử lý, hàng dữ liệu của ghế bị lock lại. User B phải đợi cho đến khi transaction của A kết thúc. Nếu A thành công, ghế chuyển sang *booked*, B kiểm tra lại → thấy không còn *available* → trả về lỗi rõ ràng. Không có trường hợp 2 người cùng đặt thành công 1 ghế.

9.4.3. Khó Khăn 3 - Học Go Trong Khi Làm Dự Án

Vấn đề gặp phải: Một số thành viên trong nhóm chưa có nhiều kinh nghiệm với Go, đặc biệt là các khái niệm như *goroutine*, *channel*, *interface*, và cách Go xử lý lỗi (không có try-catch như Java/JavaScript).

Cách khắc phục:

- Nhóm tổ chức buổi chia sẻ nội bộ ngắn (30 phút) vào đầu tuần, thành viên đã học được gì thì chia sẻ lại.
- Đặt quy ước: mọi đoạn code phức tạp phải có comment giải thích bằng tiếng Việt để đồng đội dễ hiểu.
- Tận dụng Go Playground (https://go.dev/play/) để thử nghiệm nhanh các đoạn code nhỏ trước khi tích hợp vào dự án.
- Khi gặp bug khó, pair programming (ngồi cùng nhau debug) hiệu quả hơn nhiều so với tự mình mò mẫm một mình.

CHƯƠNG 10: HƯỚNG PHÁT TRIỂN

10.1. Nâng Cấp Tính Năng Ứng Dụng

10.1.1. Gửi Vé Qua Email / SMS Sau Khi Thanh Toán

Hiện trạng: Sau khi đặt vé thành công, người dùng chỉ nhận được response JSON. Không có thông báo qua email hay SMS.

Kế hoạch thực hiện:

Gửi Email với Go và dịch vụ SendGrid:

```
import "github.com/sendgrid/sendgrid-go/helpers/mail"

func (s *NotificationService) SendTicketEmail(booking *Booking, user *User) error {
    from := mail.NewEmail("Cinema App", "noreply@cinemaapp.vn")
    to := mail.NewEmail(user.Name, user.Email)
    subject := fmt.Sprintf("Vé xem phim của bạn - %s", booking.MovieTitle)

    // HTML email với thiết kế đẹp, hiển thị QR code vé
    htmlContent := generateTicketEmailHTML(booking)

    message := mail.NewSingleEmail(from, subject, to, "", htmlContent)
    client := sendgrid.NewSendClient(os.Getenv("SENDGRID_API_KEY"))
    client.Send(message)
    return nil
}
```

Gửi SMS với Twilio hoặc ESMS (dịch vụ SMS Việt Nam):

```go
// Gửi SMS xác nhận đơn giản sau khi thanh toán
func (s *NotificationService) SendTicketSMS(phone string, booking *Booking) error
{
    message := fmt.Sprintf(
        "VE PHIM: %s | Rap: %s | %s | Ghe: %s | Ma ve: %s",
        booking.MovieTitle, booking.CinemaName,
        booking.StartTime.Format("15:04 02/01/2006"),
        booking.SeatCodes, booking.TicketCode,
    )
    // Gọi API của ESMS để gửi SMS
    return s.esmsClient.Send(phone, message)
}
```

Tích hợp vào luồng đặt vé:

```go
// Sau khi xác nhận thanh toán thành công
go func() {
    // Chạy trong goroutine riêng để không block response trả về cho người dùng
    s.notificationService.SendTicketEmail(booking, user)
    s.notificationService.SendTicketSMS(user.Phone, booking)
}()
```

Lý do dùng goroutine: Gửi email/SMS có thể mất 1-3 giây. Nếu chờ gửi xong mới trả response, người dùng sẽ cảm thấy hệ thống chậm. Chạy song song trong goroutine giúp người dùng nhận được response ngay lập tức trong khi email/SMS vẫn đang được gửi ở nền.

10.2. Nâng Cấp Kiến Trúc Phần Mềm

10.2.1. Chuyển Đổi Từ Monolithic Sang Microservices

Kiến trúc hiện tại - Monolithic (Nguyên khối):
Toàn bộ dự án hiện tại là một ứng dụng Go duy nhất chạy trên một server, xử lý mọi thứ: xác thực người dùng, quản lý phim, đặt vé, thanh toán... Đây gọi là kiến trúc Monolithic (một khối).

Monolithic phù hợp cho giai đoạn đầu vì đơn giản, dễ phát triển, dễ debug. Tuy nhiên khi hệ thống lớn lên, sẽ phát sinh vấn đề:
- Một lỗi nhỏ trong module Thanh toán có thể làm sập toàn bộ hệ thống kể cả module Xem phim.
- Không thể mở rộng từng phần độc lập: vào dịp lễ, module Đặt vé cần nhiều server hơn, nhưng vì tất cả trong 1 ứng dụng, phải nhân rộng cả hệ thống - tốn kém và lãng phí.
- Khi team lớn hơn (10+ người), nhiều người cùng sửa một codebase gây ra conflict liên tục.

Kiến trúc tương lai - Microservices (Đa dịch vụ):
Tách ứng dụng thành các **service độc lập**, mỗi service chịu trách nhiệm một nghiệp vụ cụ thể, có database riêng, deploy riêng:

Lợi ích cụ thể:

- Fault Isolation (Cô lập lỗi): Nếu Payment Service bị lỗi, người dùng vẫn xem được danh sách phim và suất chiếu bình thường - chỉ chức năng thanh toán bị ảnh hưởng.
- Independent Scaling (Mở rộng độc lập): Dịp Tết khi lượng đặt vé tăng đột biến, chỉ cần nhân số instance của Booking Service lên - không cần đụng đến User Service hay Movie Service.

- Độc lập công nghệ: Booking Service dùng Go vì cần hiệu năng cao, User Service có thể dùng Node.js, Payment Service có thể dùng Java nếu team quen hơn.

Thách thức khi chuyển đổi:

- Giao tiếp giữa các service cần cơ chế mới: HTTP/gRPC cho sync call, Message Queue (Kafka/RabbitMQ) cho async.
- Quản lý transaction phân tán (Distributed Transaction) phức tạp hơn nhiều so với transaction đơn trong một DB.
- Cần đầu tư vào infrastructure: Kubernetes để orchestrate, Service Discovery, Centralized Logging...

10.2.2. Ứng Dụng Redis Để Caching

Redis là gì? Redis (Remote Dictionary Server) là một cơ sở dữ liệu in-memory - nghĩa là nó lưu dữ liệu trực tiếp trên RAM thay vì ổ cứng. Điều này khiến tốc độ đọc/ghi của Redis nhanh hơn PostgreSQL khoảng 100-1000 lần. Redis thường được dùng để caching - lưu tạm kết quả của các truy vấn thường xuyên và tốn tài nguyên.

Vấn đề hiện tại cần caching giải quyết:

Mỗi khi người dùng mở trang chủ xem danh sách phim, hệ thống phải:

1. Kết nối đến PostgreSQL.
2. Thực thi câu query SELECT * FROM movies WHERE status = 'showing' ORDER BY created_at DESC.
3. PostgreSQL đọc từ ổ cứng, trả về kết quả.
4. Go serialize kết quả thành JSON.
5. Trả về cho người dùng.

Nếu có 1,000 người cùng lúc mở trang chủ → 1,000 lần query xuống PostgreSQL → database bị quá tải dù dữ liệu danh sách phim gần như không thay đổi trong 1-2 giờ.

Giải pháp với Redis (Cache-Aside Pattern):

```
func (s *MovieService) GetNowShowingMovies() ([]Movie, error) {
    cacheKey := "movies:now_showing"

    // Bước 1: Kiểm tra cache trước
    cached, err := s.redis.Get(context.Background(), cacheKey).Result()
    if err == nil {
        // Cache HIT - có dữ liệu trong Redis, trả về ngay
        // Không cần đụng đến PostgreSQL
        var movies []Movie
        json.Unmarshal([]byte(cached), &movies)
        return movies, nil
    }

    // Bước 2: Cache MISS - không có trong Redis, truy vấn PostgreSQL
    movies, err := s.movieRepo.FindNowShowing()
    if err != nil {
        return nil, err
    }

    // Bước 3: Lưu kết quả vào Redis để những lần sau dùng lại
    // TTL = 30 phút - sau 30 phút Redis tự xóa, lần sau sẽ query DB lại để lấy dữ liệu
mới
    data, _ := json.Marshal(movies)
    s.redis.Set(context.Background(), cacheKey, data, 30*time.Minute)

    return movies, nil
}

// Khi admin cập nhật thông tin phim, phải xóa cache cũ
```

```
func (s *MovieService) UpdateMovie(movieID int, input UpdateMovieInput) error {
    err := s.movieRepo.Update(movieID, input)
    if err != nil {
        return err
    }

    // Invalidate cache - xóa cache để lần sau load lại từ DB với dữ liệu mới
    s.redis.Del(context.Background(), "movies:now_showing")
    s.redis.Del(context.Background(), fmt.Sprintf("movies:detail:%d", movieID))
    return nil
}
```

Các trường hợp nên dùng Redis trong dự án:

Dữ liệu Cache	**Cache Key**	**TTL (Thời gian sống)**	**Lý do**
Danh sách phim đang chiếu	movies:now_showing	30 phút	Cập nhật không thường xuyên, nhiều người xem
Chi tiết một bộ phim	movies:detail:{id}	1 giờ	Thông tin phim rất ít thay đổi
Danh sách rạp chiếu phim	cinemas:all	2 giờ	Gần như không đổi
Sơ đồ ghế của suất chiếu	seats:showtime:{id}	2 phút	Cần cập nhật nhanh khi có người đặt
Session người dùng / Token blacklist	session:{user_id}	= Thời hạn token	Quản lý đăng xuất, token bị thu hồi

Kết quả kỳ vọng sau khi áp dụng Redis:

- Giảm ~80-90% số lượng query xuống PostgreSQL cho các endpoint đọc (GET).

- Thời gian phản hồi của GET /movies giảm từ ~50-100ms (query DB) xuống còn ~1-5ms (đọc từ RAM).
- PostgreSQL được giải phóng tập trung xử lý các transaction quan trọng như đặt vé, thay vì phải phục vụ hàng nghìn request đọc đơn giản mỗi giây.

Tổng kết hướng phát triển: Những nâng cấp được đề xuất - cổng thanh toán thực tế, thông báo tự động, chuyển đổi sang Microservices và Redis caching - đều được lên kế hoạch theo thứ tự ưu tiên từ dễ đến khó, từ mang lại giá trị người dùng ngay lập tức đến cải thiện khả năng mở rộng dài hạn. Với nền tảng kiến trúc MVC sạch mà nhóm đã xây dựng, việc mở rộng dần từng bước là hoàn toàn khả thi.